TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

“CURSO INTENSIVO DE 5 DÍAS SOBRE AGENTES DE IA CON GOOGLE”

Materia:

Ciencia de Datos

Carrera

Ing. Sistemas Computacionales

Presenta:

MORALES CONTRERAS ALONDRA GEMALI
22350351

Asesor:

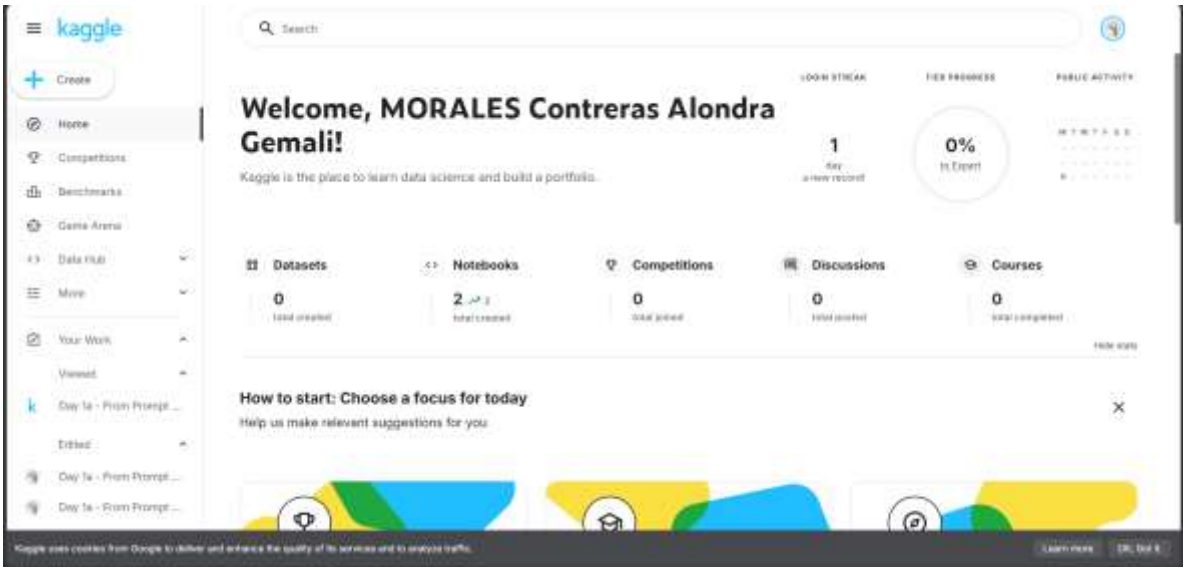
Dr. Julio Aguilar Carmona

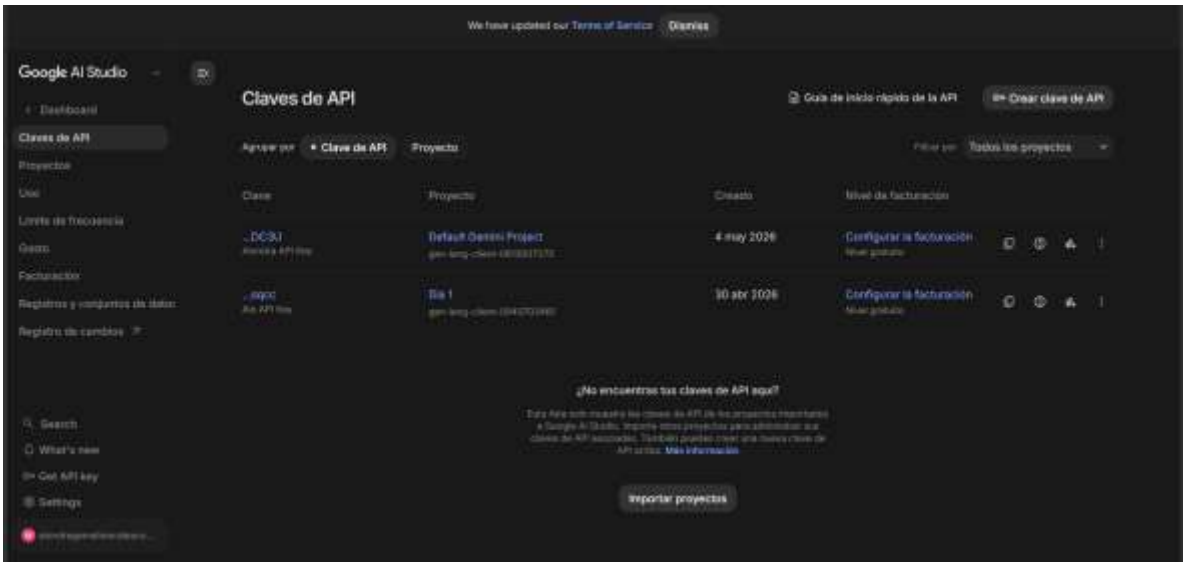
Grado y grupo:

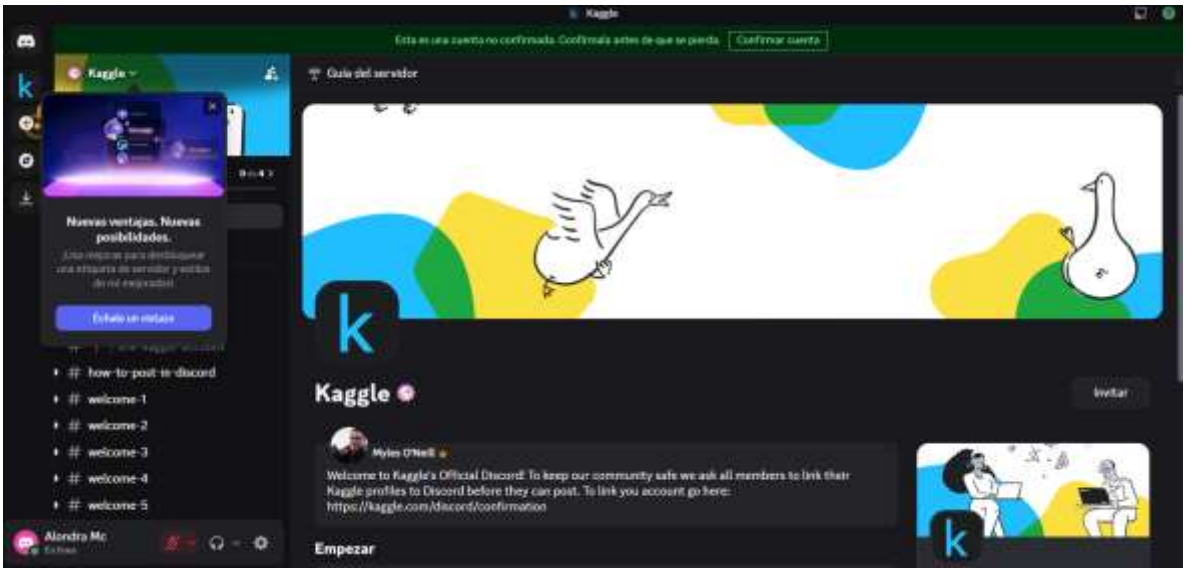
8° “A”

Tuxtepec, Oax. **05 de mayo de 2026**

En esta actividad creé mi cuenta en Kaggle, también me registré en AI Studio y generé mi clave API, y abrí una cuenta en Discord para unirme al servidor de Kaggle y poder colaborar con otros estudiantes.

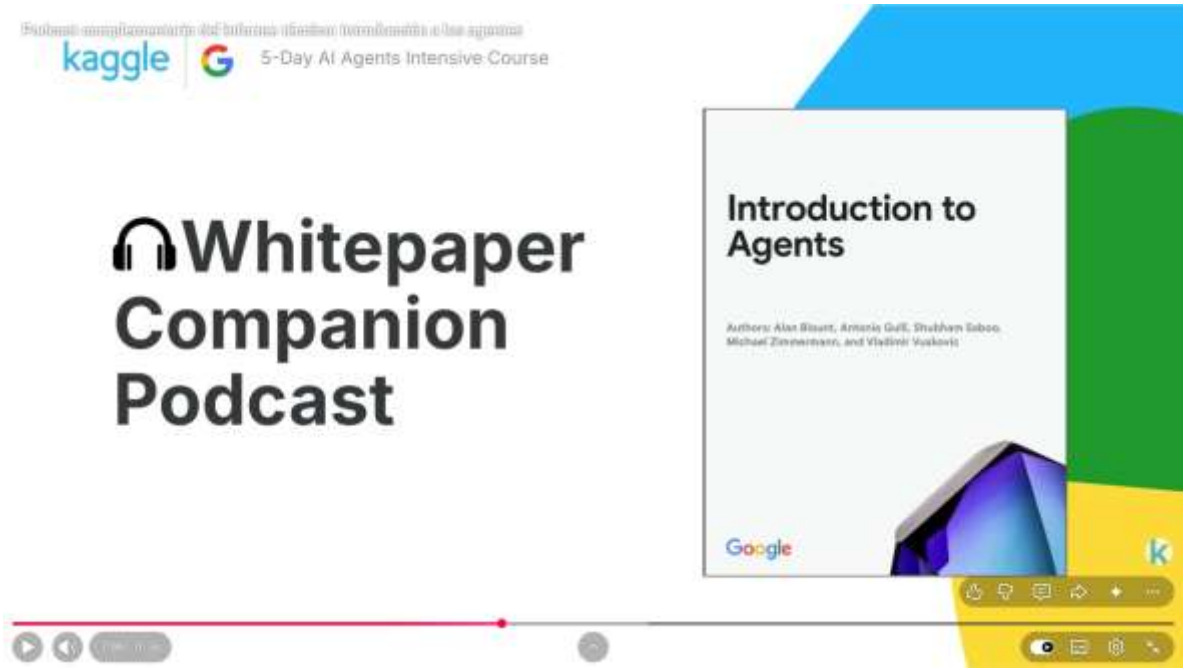






En el Día 1, titulado “Introducción a los agentes”, comprendí qué son los agentes de inteligencia artificial, cuáles son sus capacidades y la importancia de que operen de forma segura. Además, realicé prácticas en las que desarrollé un agente y un sistema compuesto por varios agentes que colaboran entre sí.

Para iniciar, escuché el podcast con el fin de obtener una visión general del tema.

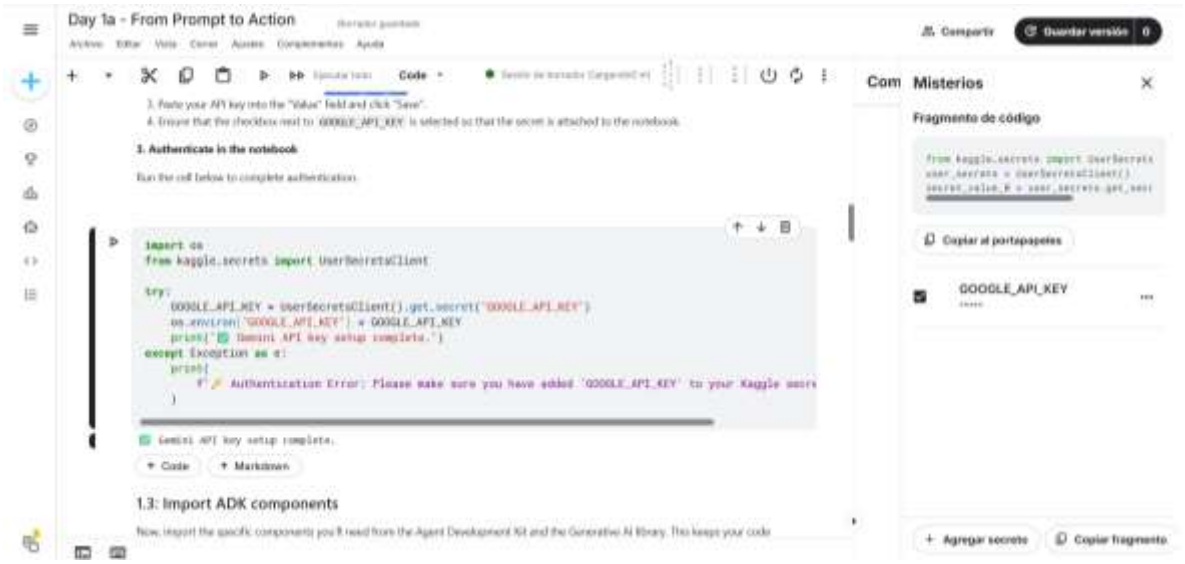


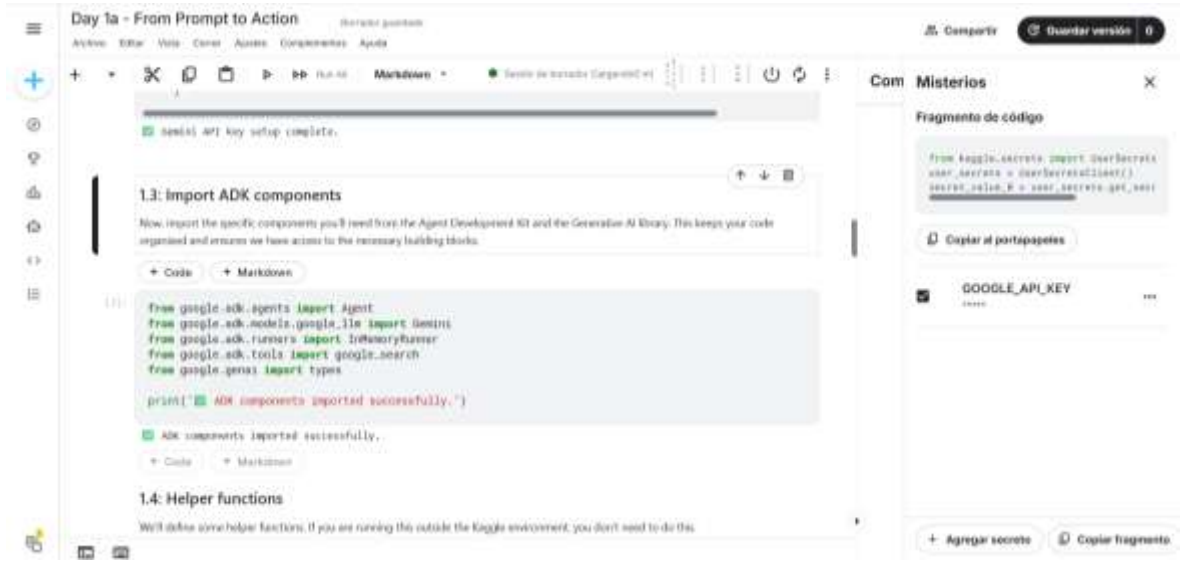
Después leí el documento técnico para profundizar mejor en los conceptos.

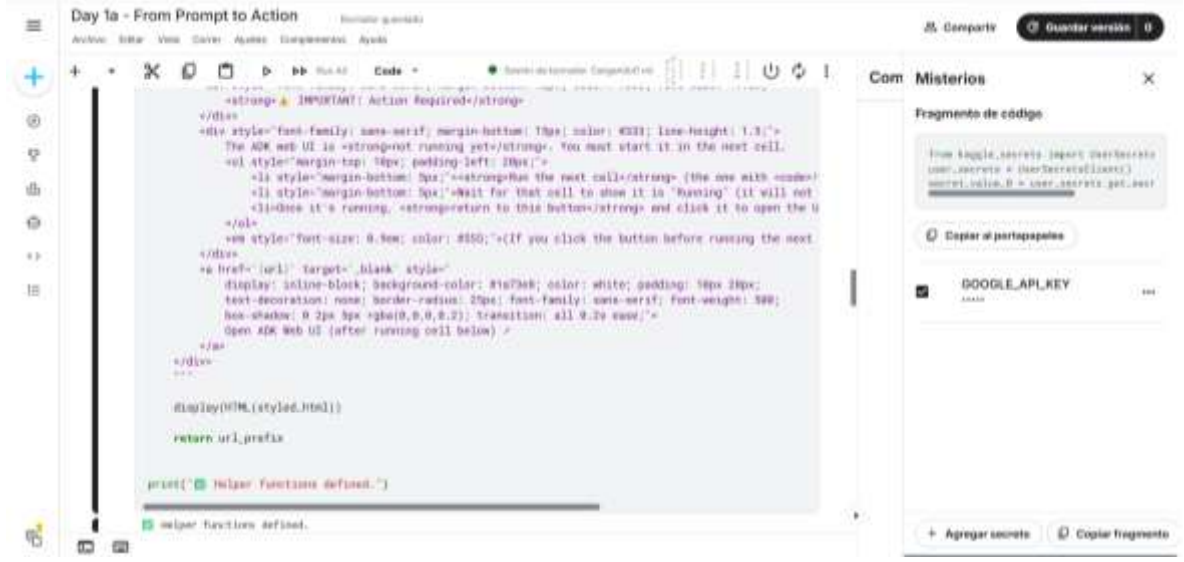
DIA 1

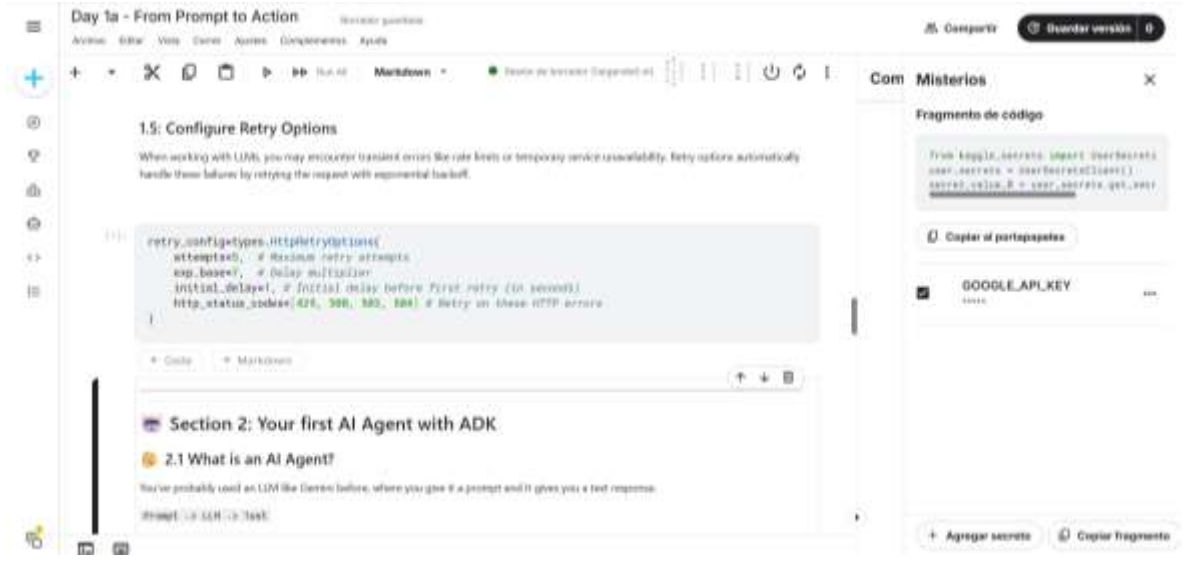
Section 1: Setup

Para poder correr cada código teníamos que meter la clave API que ya habíamos realizado.



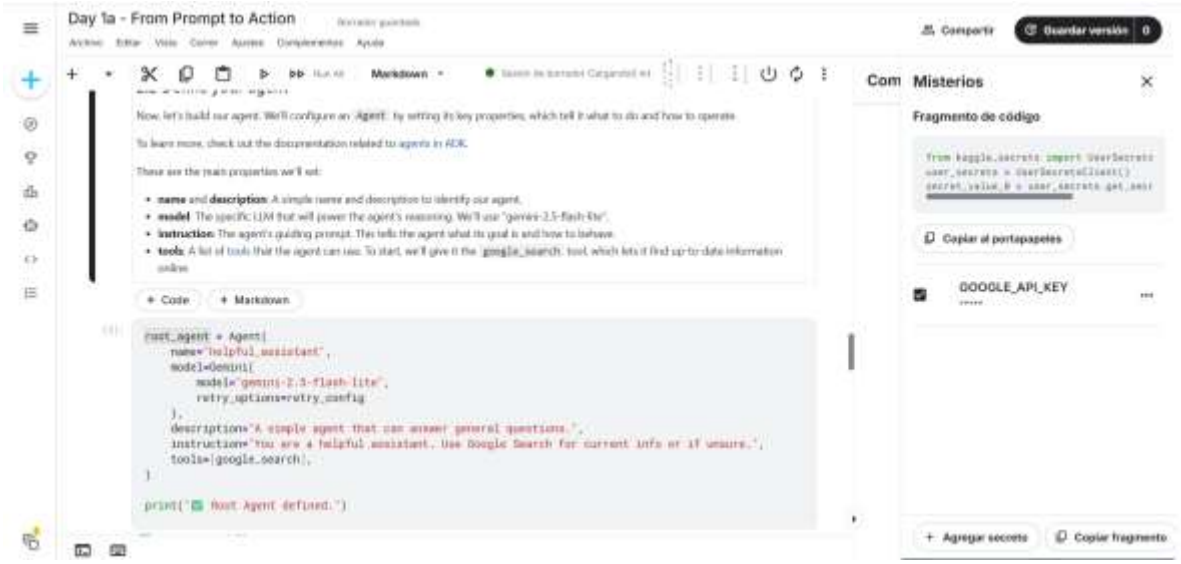

Day 1a - From Prompt to Action
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
Gemini API key setup complete.
1.3: Import ADK components
Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.
+ Code
+ Markdown
from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types
print("✅ ADK components imported successfully.")
ADK components imported successfully.
1.4: Helper functions
We'll define some helper functions. If you are running this outside the Kaggle environment, you don't need to do this.

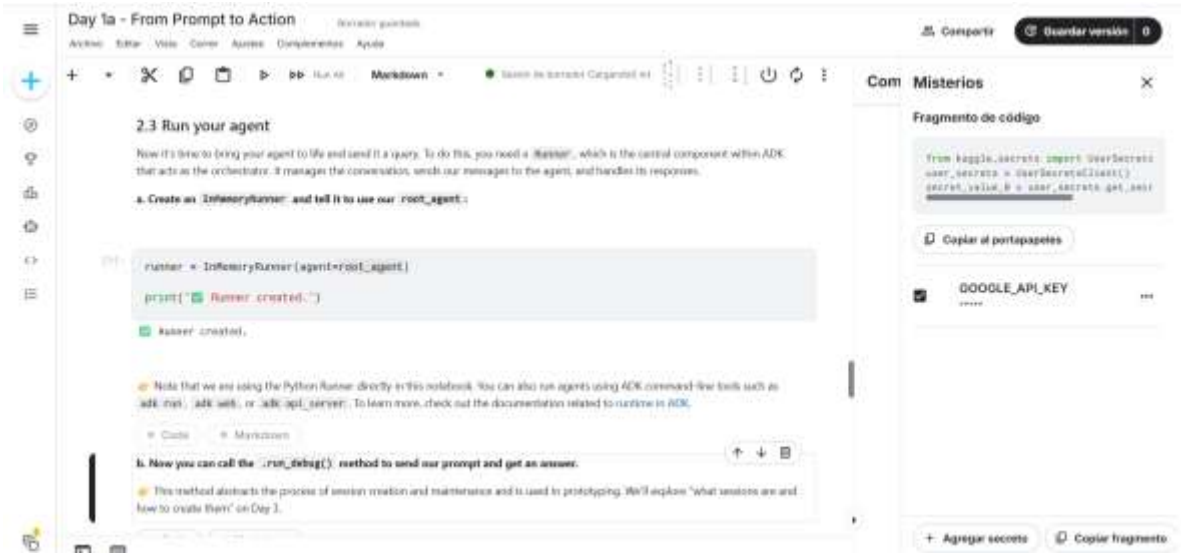

Day 1a - From Prompt to Action
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
⚠️ IMPORTANT: Action Required
The ADK web UI is not running yet. You must start it in the next cell.
Run the next cell (the one with <code>!
Wait for that cell to show it is "Running" (it will not
Once it's running, return to this button and click it to open the U
(If you click the button before running the next
Open ADK Web UI (after running cell below) ↗
display(HTML(styled_html))
return url_prefix
print("✅ Helper functions defined.")
Helper functions defined.

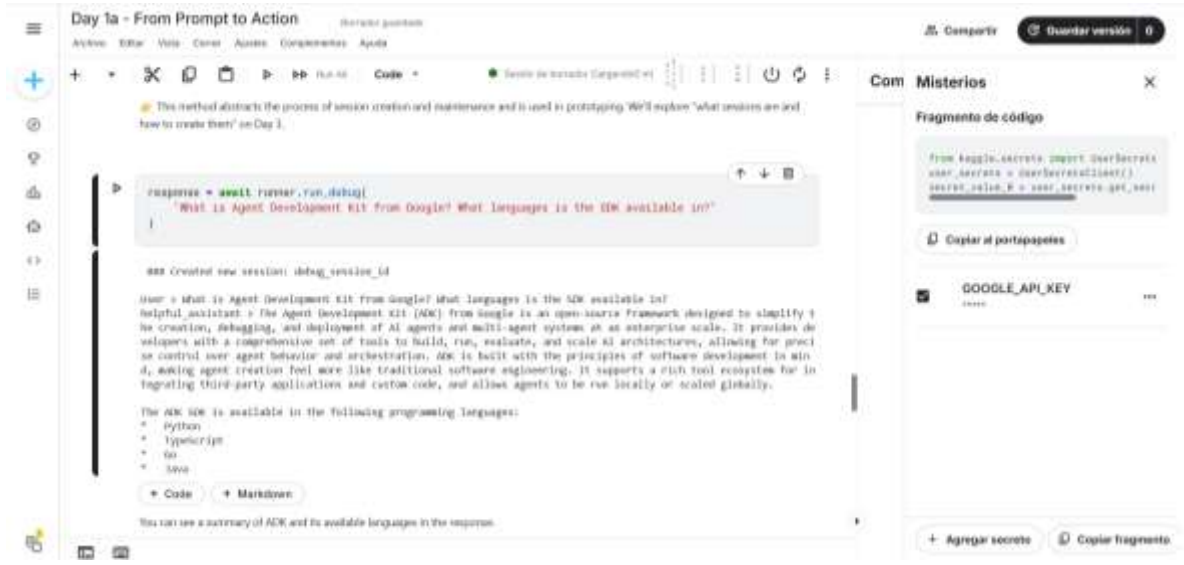

Day 1a - From Prompt to Action
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
1.5: Configure Retry Options
When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.
retry_config=types.HttpRetryOptions(
attempts=5, # Maximum retry attempts
exp_base=7, # Delay multiplier
initial_delay=1, # Initial delay before first retry (in seconds)
http_status_codes=[429, 500, 503, 504] # Retry on these HTTP errors
)
Section 2: Your first AI Agent with ADK
2.1 What is an AI Agent?
You've probably used an LLM like Gemini before, where you give it a prompt and it gives you a text response.
Prompt -> LLM -> Text

Section 2: Your first AI Agent with ADK

Analicé cómo se definían su nombre, el modelo, las instrucciones y las herramientas necesarias para que funcionara correctamente; posteriormente, ejecuté el proceso para comprobar que operara de manera adecuada.

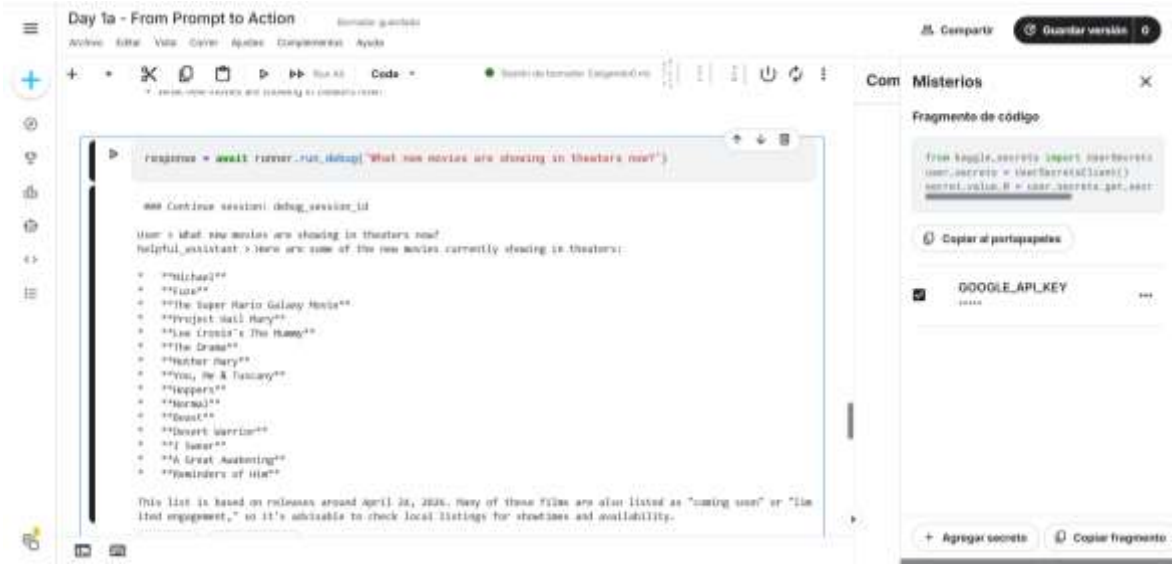





Day 1a - From Prompt to Action
Compartir
Guardar versión
Code
response = await runner.run_debug(
'What is Agent Development Kit from Google? What languages is the SDK available in?'
)
Created new session: debug_session_id
User > What is Agent Development Kit from Google? What languages is the SDK available in?
Helpful_assistant > The Agent Development Kit (ADK) from Google is an open-source framework designed to simplify t
he creation, debugging, and deployment of AI agents and multi-agent systems at an enterprise scale. It provides de
velopers with a comprehensive set of tools to build, run, evaluate, and scale AI architectures, allowing for preci
se control over agent behavior and orchestration. ADK is built with the principles of software development in min
d, making agent creation feel more like traditional software engineering. It supports a rich tool ecosystem for in
tegrating third-party applications and custom code, and allows agents to be run locally or scaled globally.
The ADK SDK is available in the following programming languages:
* Python
* Typescript
* Go
* Java
+ Code
+ Markdown
You can see a summary of ADK and its available languages in the response.
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento

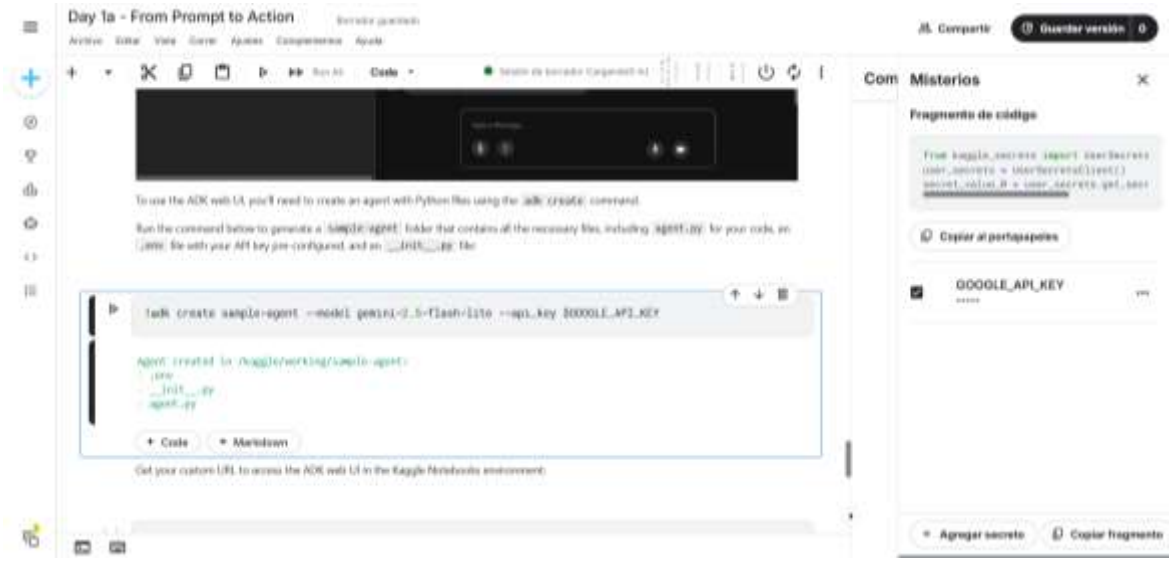

Day 1a - From Prompt to Action
Compartir
Guardar versión
Markdown
2.5 Your Turn!
This is your chance to see the agent in action. Ask it a question that requires current information.
Try one of these, or make up your own:
• What's the weather in London?
• Who won the last soccer world cup?
• What new movies are showing in theaters now?
+ Code
+ Markdown
response = await runner.run_debug("What's the weather in London?")
Continue session: debug_session_id
User > What's the weather in London?
Helpful_assistant > The weather in London, UK is currently partly cloudy with a temperature of 61°F (16°C). The hu
midity is around 43% and it feels like 60°F (20°C). There is a 15% chance of rain.
Looking ahead, the forecast for today, Monday, April 27, 2026, indicates sunny conditions during the day and cloud
y conditions at night, with a 15% chance of rain during the day and 40% at night. Temperatures are expected to ran
ge between 46°F (8°C) and 69°F (21°C). For the upcoming week, temperatures are predicted to be in the range of 46°
F to 71°F.
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento

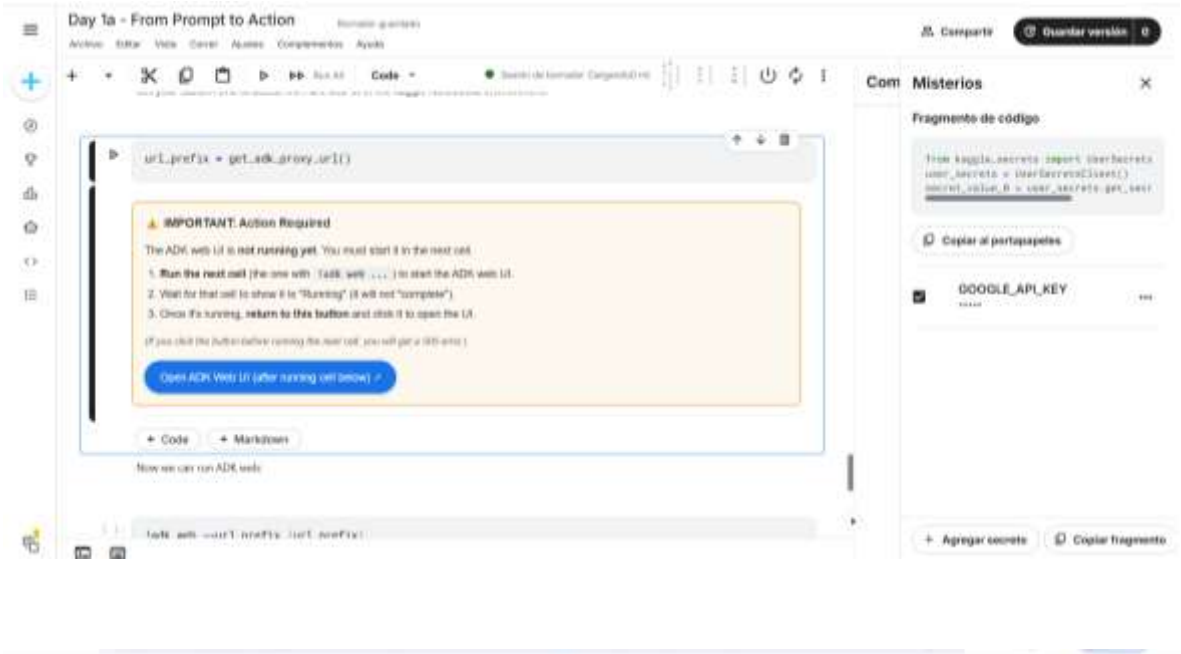

Day 1a - From Prompt to Action
Compartir
Guardar versión
Code
Try one of these, or make up your own:
• What's the weather in London?
• Who won the last soccer world cup?
• What new movies are showing in theaters now?
response = await runner.run_debug("Who won the last soccer world cup?")
Continue session: debug_session_id
User > Who won the last soccer world cup?
Helpful_assistant > Argentina won the last soccer World Cup, which was held in Qatar in 2022. They defeated France
in a dramatic final that ended 3-3 after extra time and was decided by a penalty shootout, which Argentina won 4-
2. This was Argentina's third World Cup title, and their first since 1986. The match is widely considered one of t
he greatest World Cup finals ever played.
+ Code
+ Markdown
Section 3: Try the ADK Web Interface
Overview
ADK includes a built-in web interface for interactively chatting with, testing, and debugging your agents.
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento



Section 3: Try the ADK Web Interface

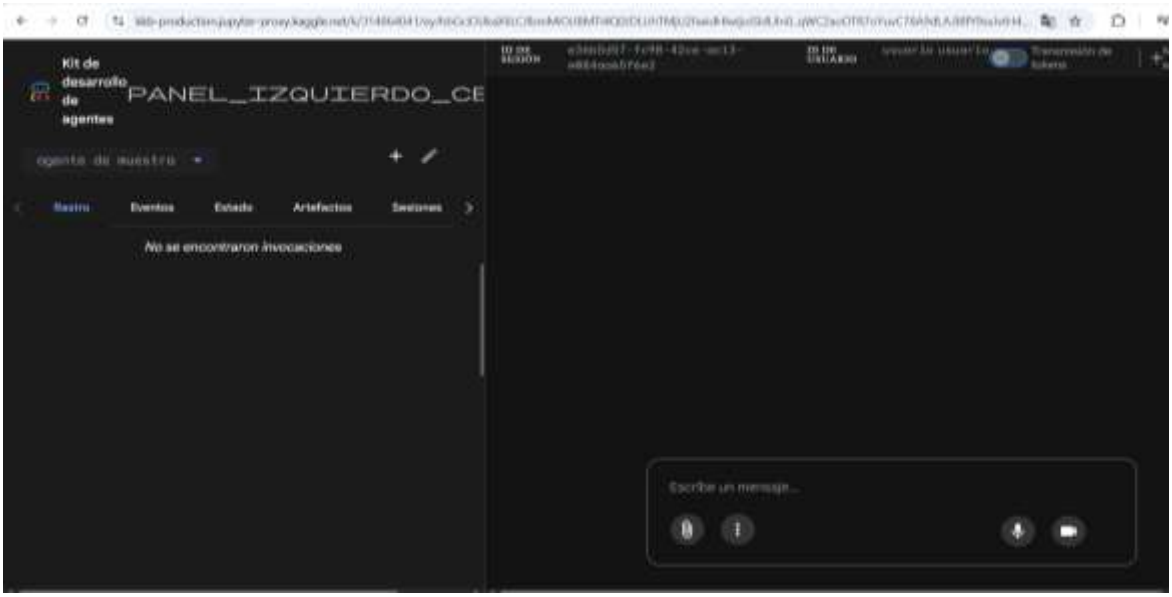
Examiné la creación del agente utilizando el comando **adk create** y, posteriormente, su ejecución con **adk web**, lo que permitió interactuar con él desde la interfaz web y entender su funcionamiento de manera práctica.

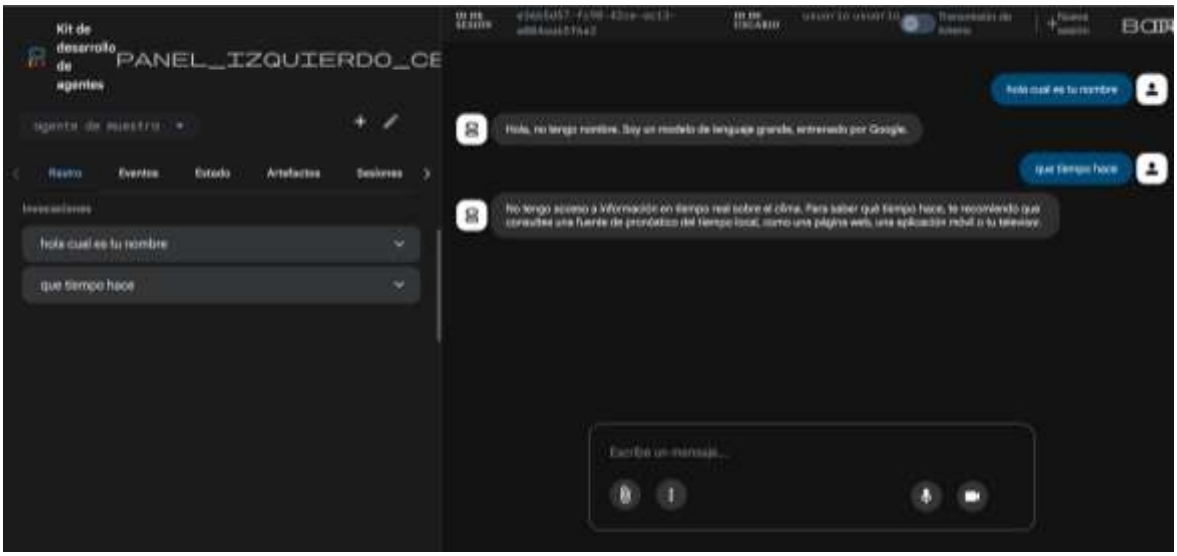




jupyter

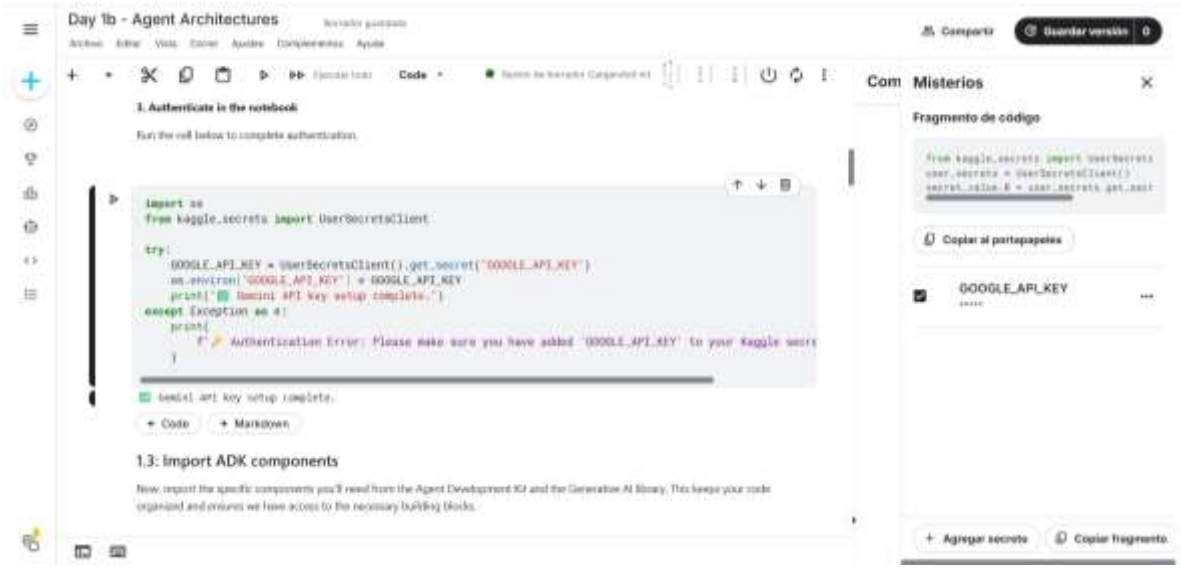
500: Error interno del servidor

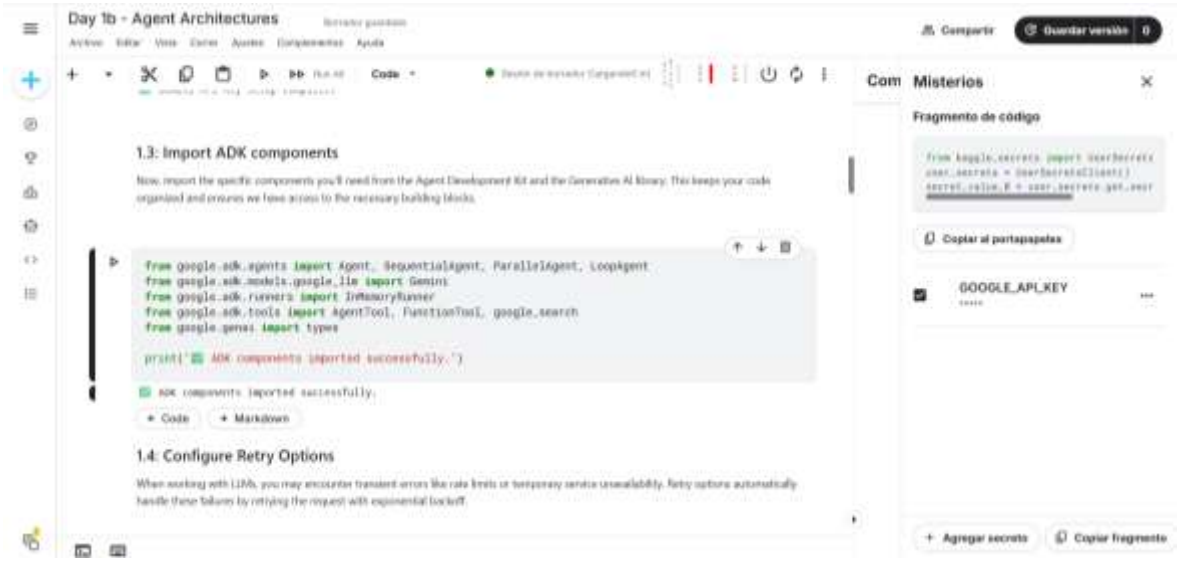

Kit de desarrollo de agentes
PANEL_IZQUIERDO_CE
agente de muestra
Rastro
Eventos
Estado
Artefactos
Sesiones
No se encontraron invocaciones
Escribe un mensaje...

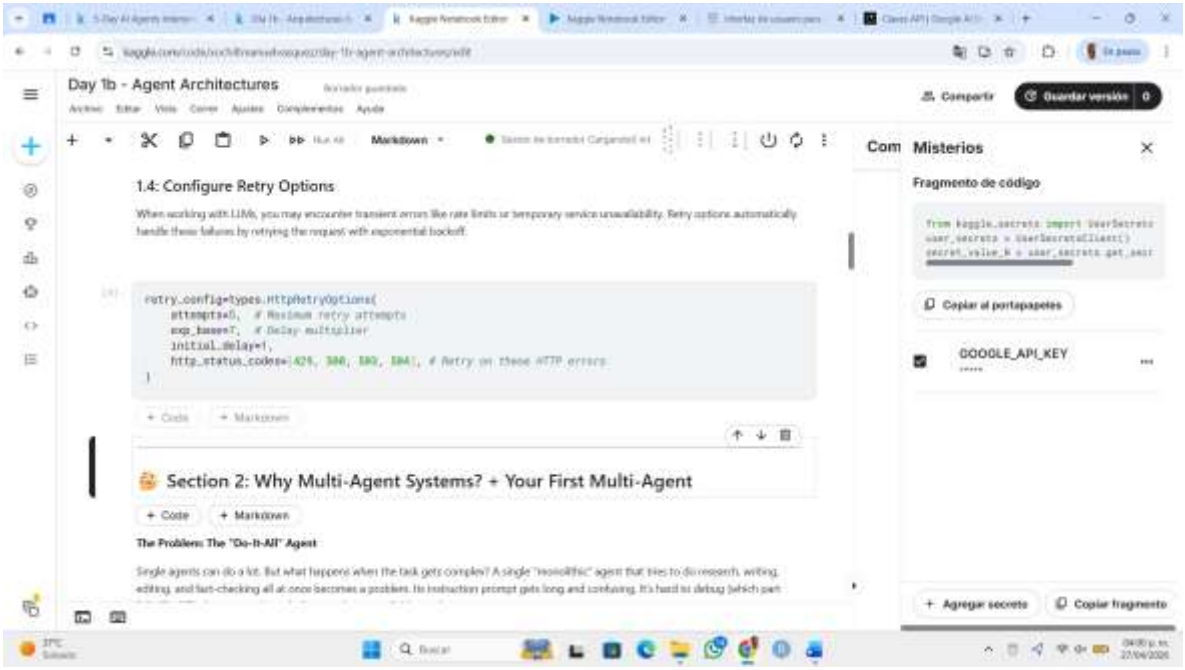

Kit de desarrollo de agentes
PANEL_IZQUIERDO_CE
agente de muestra
Rastro
Eventos
Estado
Artefactos
Sesiones
hola cual es tu nombre
que tiempo hace
Hola, no tengo nombre. Soy un modelo de lenguaje grande, entrenado por Google.
Escribe un mensaje...

DIA 1

Section 1: Setup

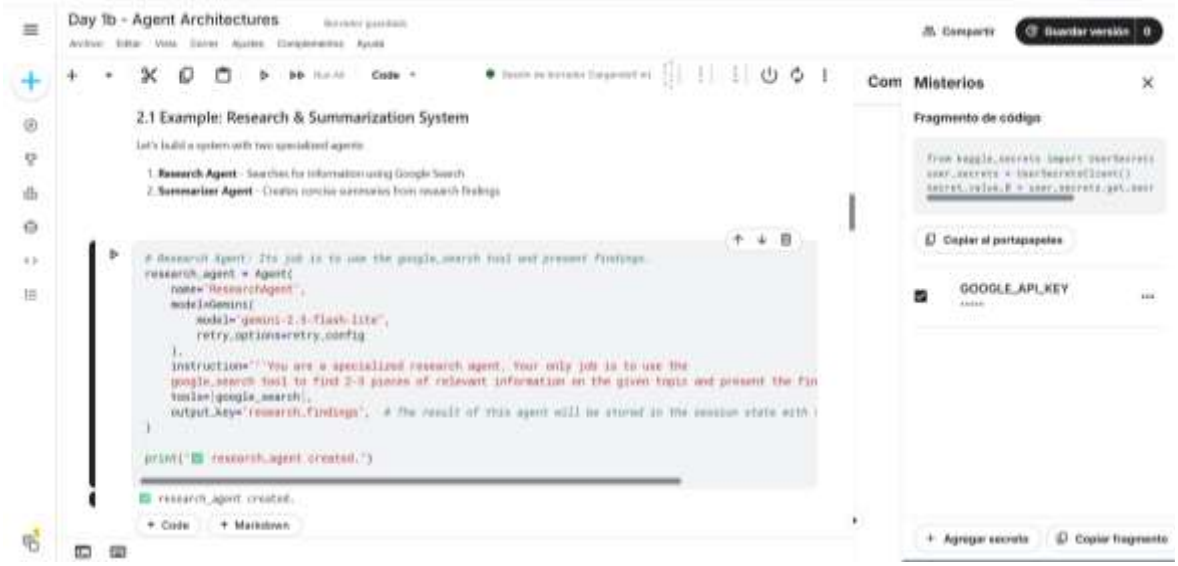


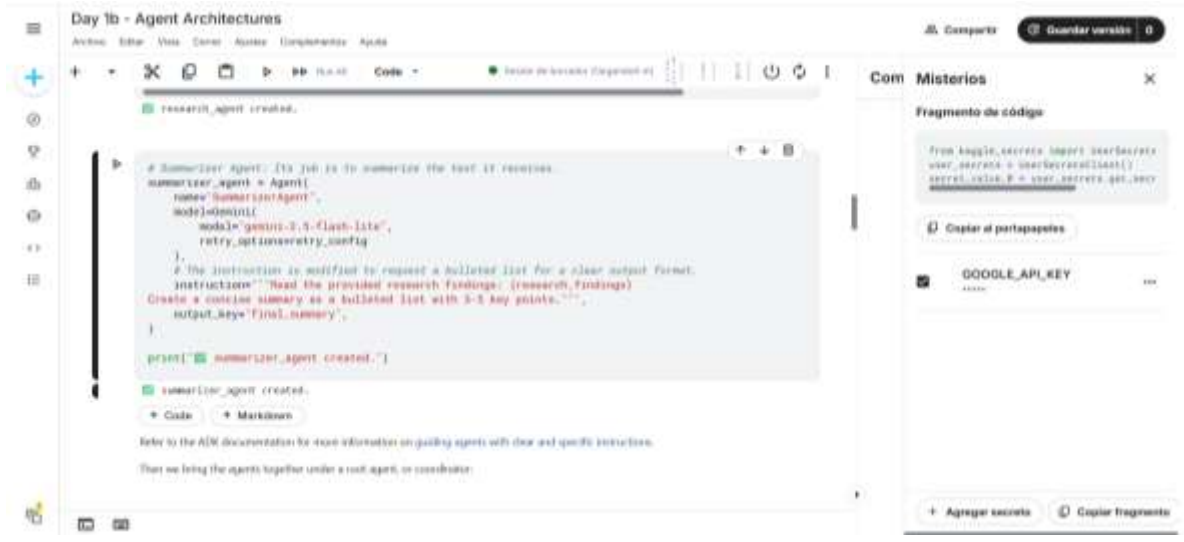


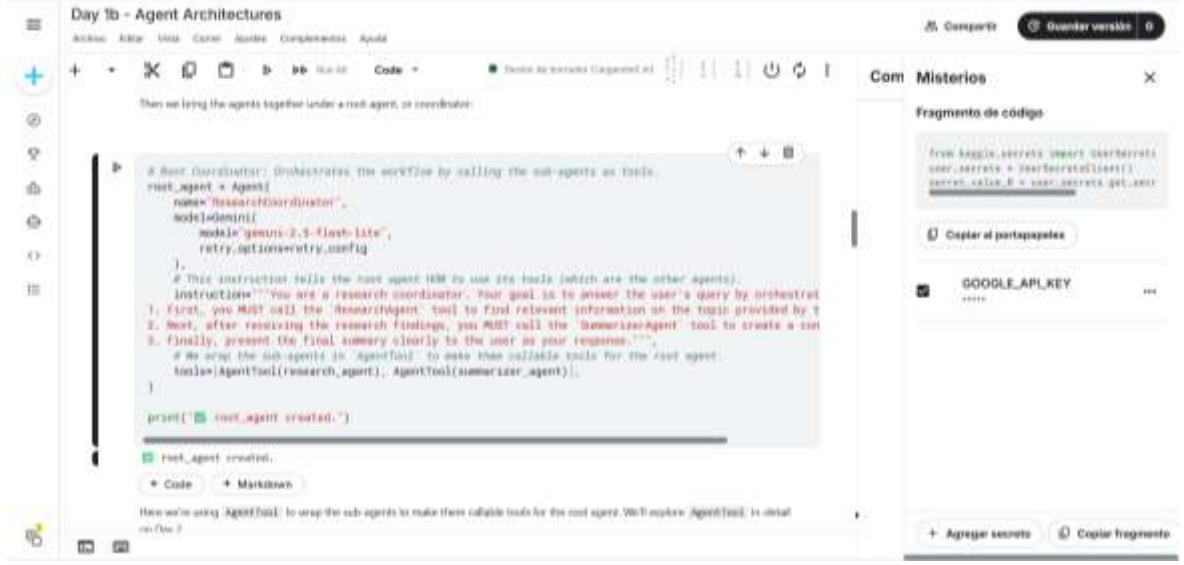


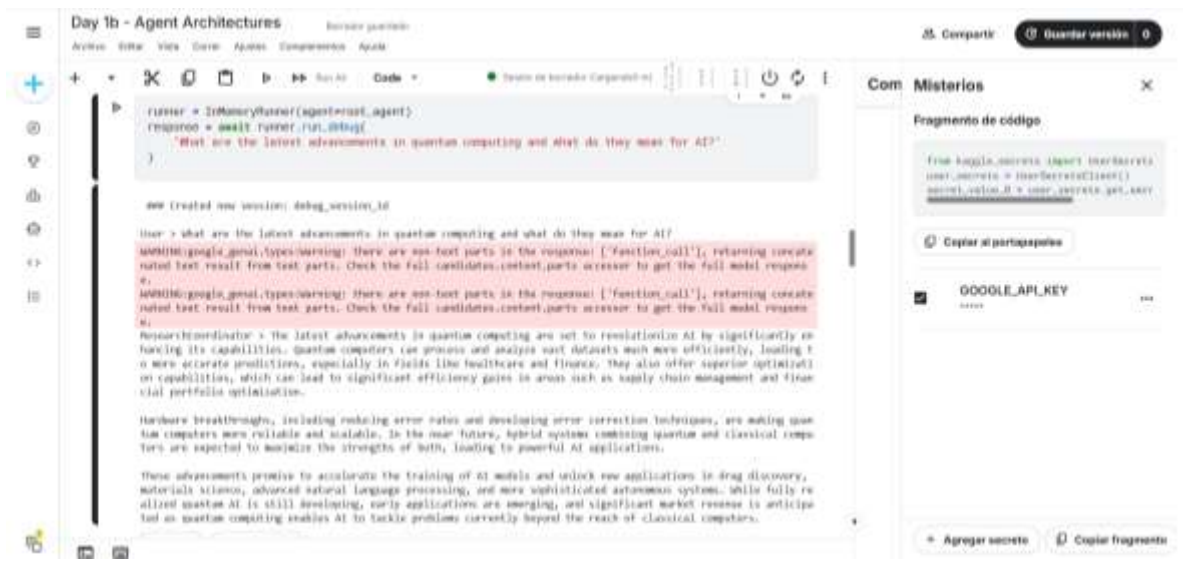
Section 2: Why Multi-Agent Systems? + Your First Multi-Agent

Observé la creación de dos agentes especializados.




Day 1b - Agent Architectures
Compartir
Guardar versión
Code
research_agent created.
# Summarizer Agent: Its job is to summarize the text it receives.
summarizer_agent = Agent(
name="SummarizerAgent",
model=Gemini(
model="gemini-2.5-flash-lite",
retry_options=retry_config
),
# The instruction is modified to request a bulleted list for a clear output format.
instruction="""Read the provided research findings: {research_findings}
Create a concise summary as a bulleted list with 3-5 key points.""",
output_key="final_summary",
)
print("✅ summarizer_agent created.")
summarizer_agent created.
+ Code
+ Markdown
Refer to the ADK documentation for more information on guiding agents with clear and specific instructions.
Then we bring the agents together under a root agent, or coordinator:
Com
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

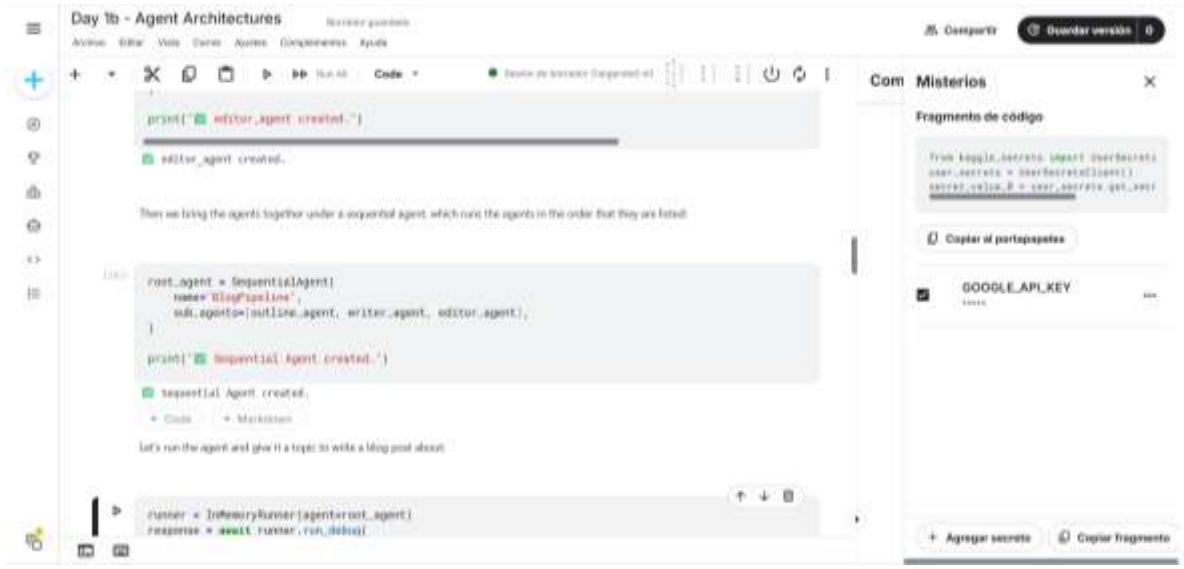

Day 1b - Agent Architectures
Compartir
Guardar versión
Code
Then we bring the agents together under a root agent, or coordinator:
# Root Coordinator: Orchestrates the workflow by calling the sub-agents as tools.
root_agent = Agent(
name="ResearchCoordinator",
model=Gemini(
model="gemini-2.5-flash-lite",
retry_options=retry_config
),
# This instruction tells the root agent HOW to use its tools (which are the other agents).
instruction="""You are a research coordinator. Your goal is to answer the user's query by orchestrat
1. First, you MUST call the `ResearchAgent` tool to find relevant information on the topic provided by t
2. Next, after receiving the research findings, you MUST call the `SummarizerAgent` tool to create a con
3. Finally, present the final summary clearly to the user as your response.""",
# We wrap the sub-agents in `AgentTool` to make them callable tools for the root agent.
tools=[AgentTool(research_agent), AgentTool(summarizer_agent)],
)
print("✅ root_agent created.")
root_agent created.
+ Code
+ Markdown
Here we're using `AgentTool` to wrap the sub-agents to make them callable tools for the root agent. We'll explore `AgentTool` in detail
Com
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

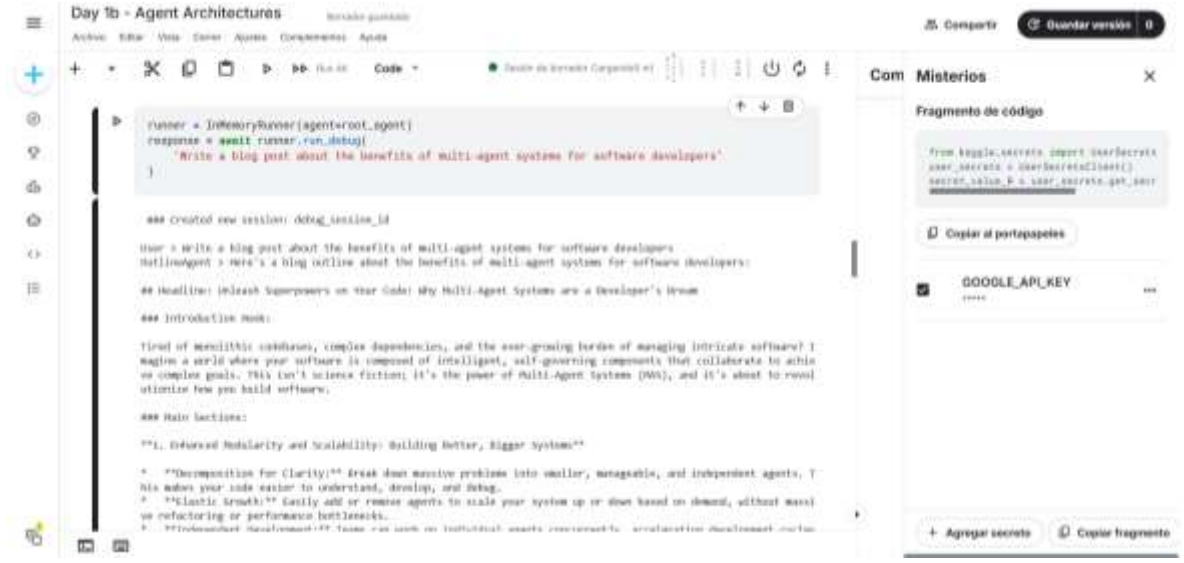

Day 1b - Agent Architectures
Compartir
Guardar versión
Code
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
"What are the latest advancements in quantum computing and what do they mean for AI?"
)
Created new session: debug_session_id
User > what are the latest advancements in quantum computing and what do they mean for AI?
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concate
nated text result from text parts. Check the full candidates.content.parts accessor to get the full model respons
e.
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concate
nated text result from text parts. Check the full candidates.content.parts accessor to get the full model respons
e.
ResearchCoordinator > The latest advancements in quantum computing are set to revolutionize AI by significantly en
hancing its capabilities. Quantum computers can process and analyze vast datasets much more efficiently, leading t
o more accurate predictions, especially in fields like healthcare and finance. They also offer superior optimizati
on capabilities, which can lead to significant efficiency gains in areas such as supply chain management and finan
cial portfolio optimization.
Hardware breakthroughs, including reducing error rates and developing error correction techniques, are making quan
tum computers more reliable and scalable. In the near future, hybrid systems combining quantum and classical comp
uters are expected to maximize the strengths of both, leading to powerful AI applications.
These advancements promise to accelerate the training of AI models and unlock new applications in drug discovery,
materials science, advanced natural language processing, and more sophisticated autonomous systems. While fully re
alized quantum AI is still developing, early applications are emerging, and significant market revenue is anticipa
ted as quantum computing enables AI to tackle problems currently beyond the reach of classical computers.
Com
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

Section 3: Sequential Workflows - The Assembly Line

Entendí cómo se construye un sistema compuesto por tres agentes (outline, writer y editor), organizados de manera secuencial para generar y perfeccionar automáticamente un blog, observando que cada uno cumple una función específica dentro del proceso.

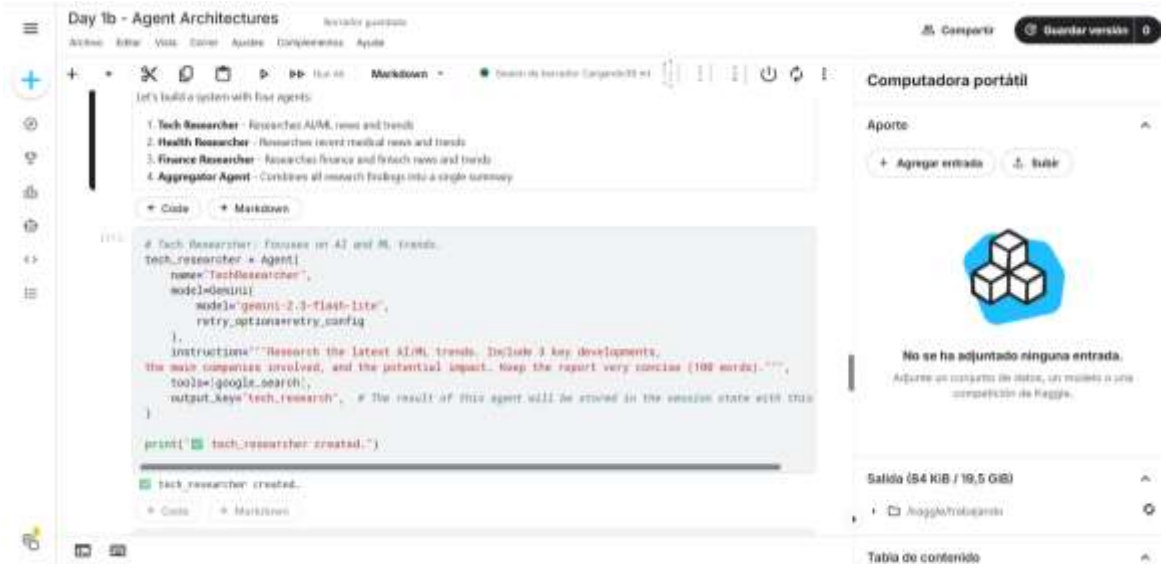

Day 1b - Agent Architectures
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
writer_agent created.
editor_agent created.
Then we bring the agents together under a sequential agent, which runs the agents in the order that they are listed:

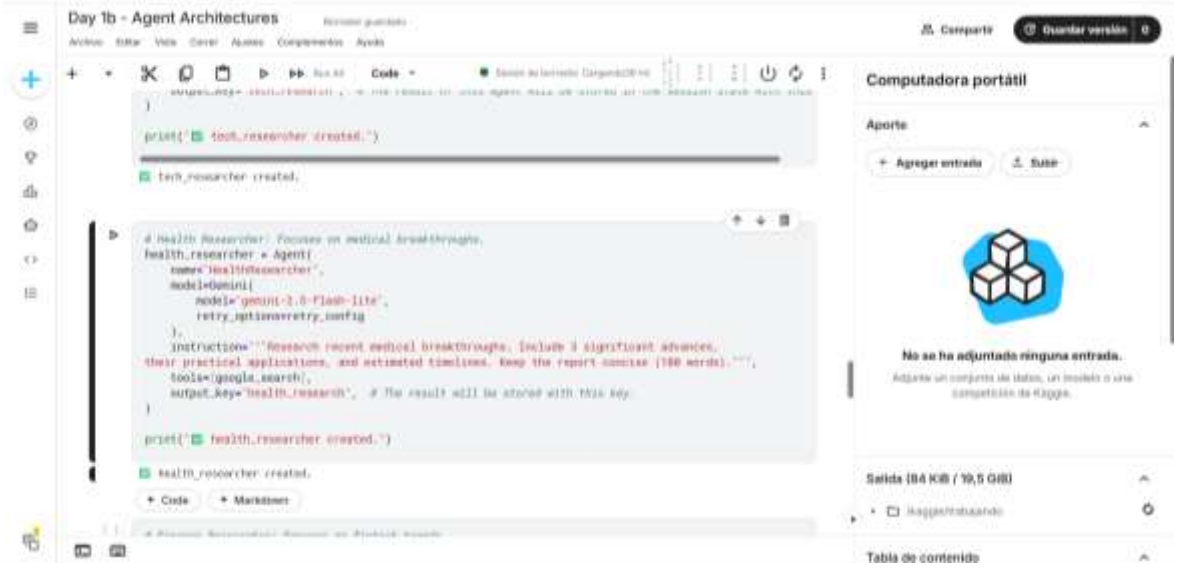

Day 1b - Agent Architectures
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
editor_agent created.
Then we bring the agents together under a sequential agent, which runs the agents in the order that they are listed:
Sequential Agent created.
Let's run the agent and give it a topic to write a blog post about:

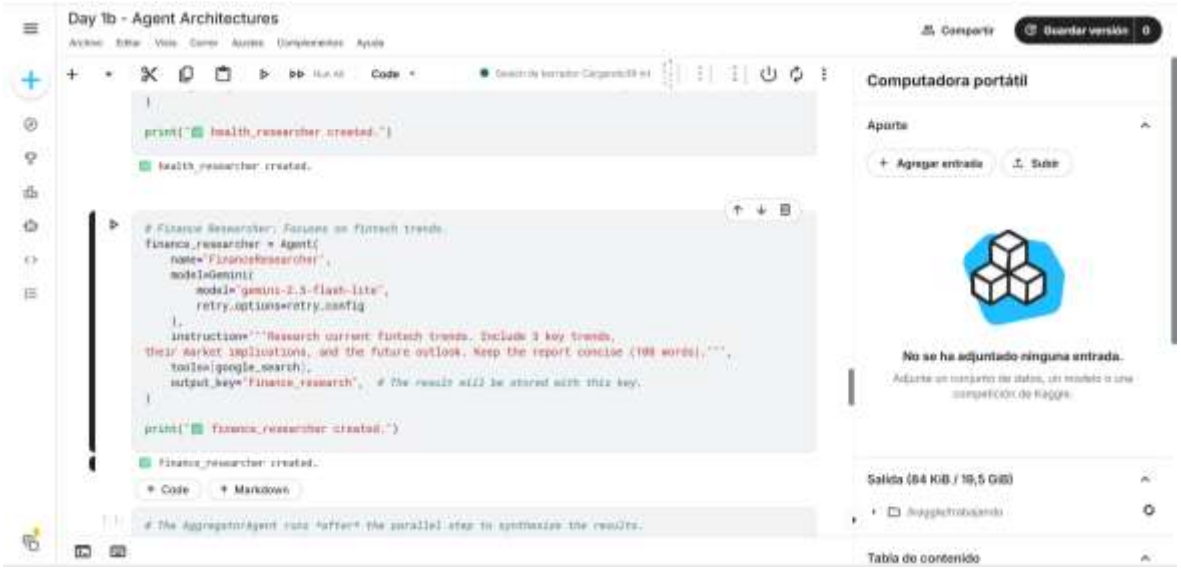

Day 1b - Agent Architectures
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
Write a blog post about the benefits of multi-agent systems for software developers
*** Creating new session: debug_session_id
User > Write a blog post about the benefits of multi-agent systems for software developers
OutlineAgent > Here's a blog outline about the benefits of multi-agent systems for software developers:
Headline: Unleash Superpowers on Your Code: Why Multi-Agent Systems are a Developer's Dream
*** Introductive Hook:
Tired of monolithic codebases, complex dependencies, and the ever-growing burden of managing intricate software? I
magine a world where your software is composed of intelligent, self-governing components that collaborate to achie
ve complex goals. This isn't science fiction; it's the power of Multi-Agent Systems (MAS), and it's about to revol
utionize how you build software.
*** Main Sections:
1. Enhanced Modularity and Scalability: Building Better, Bigger Systems
* **Decomposition for Clarity:** Break down massive problems into smaller, manageable, and independent agents. T
his makes your code easier to understand, develop, and debug.
* **Elastic Growth:** Easily add or remove agents to scale your system up or down based on demand, without massi
ve refactoring or performance bottlenecks.

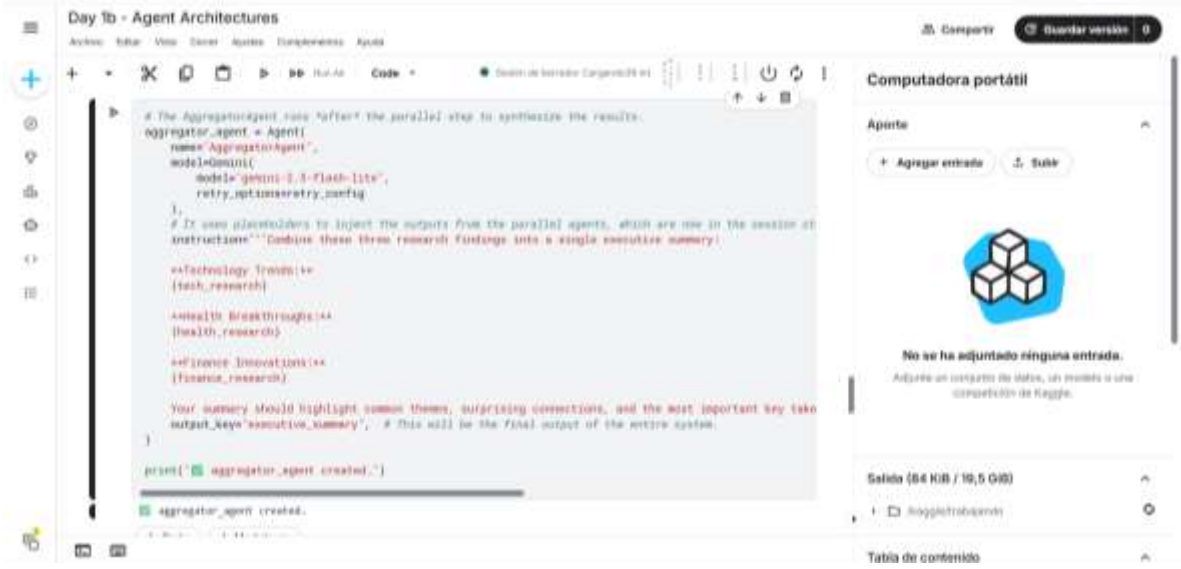
Section 4: Parallel Workflows - Independent Researchers

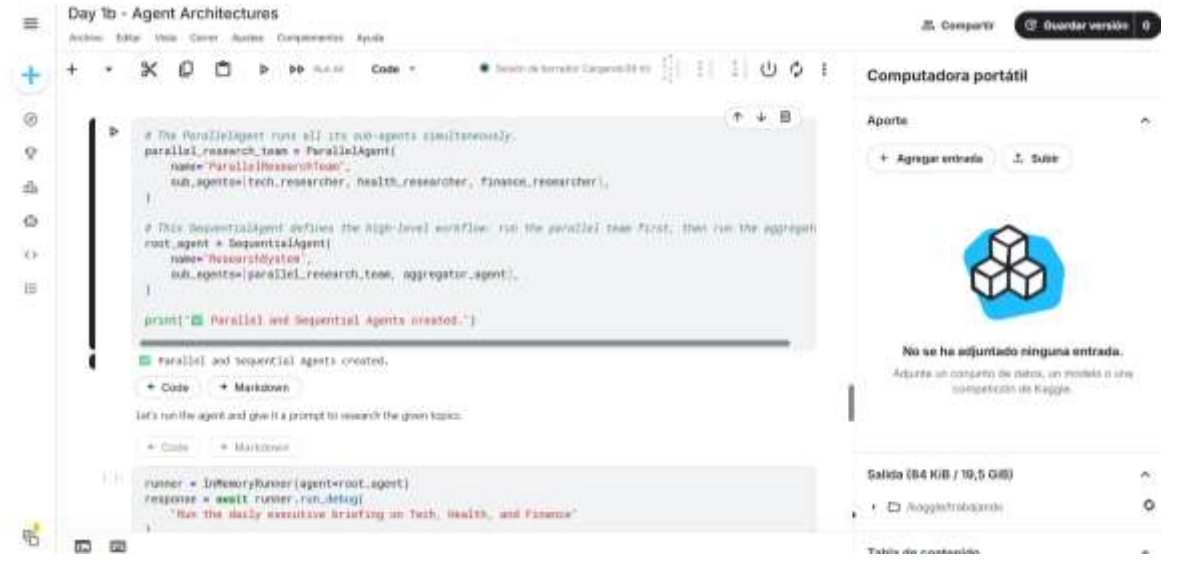
Analicé la creación de múltiples agentes que investigan distintos temas de forma paralela, junto con un agente adicional encargado de unificar toda la información en un único resumen, comprendiendo cómo se integra el trabajo distribuido en un resultado final coherente.

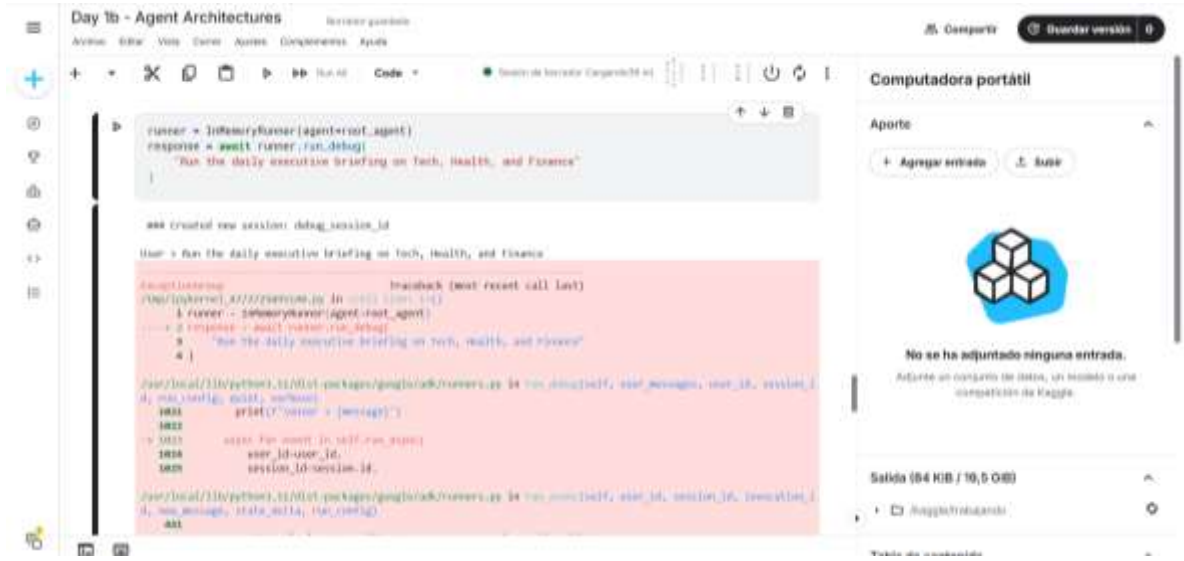


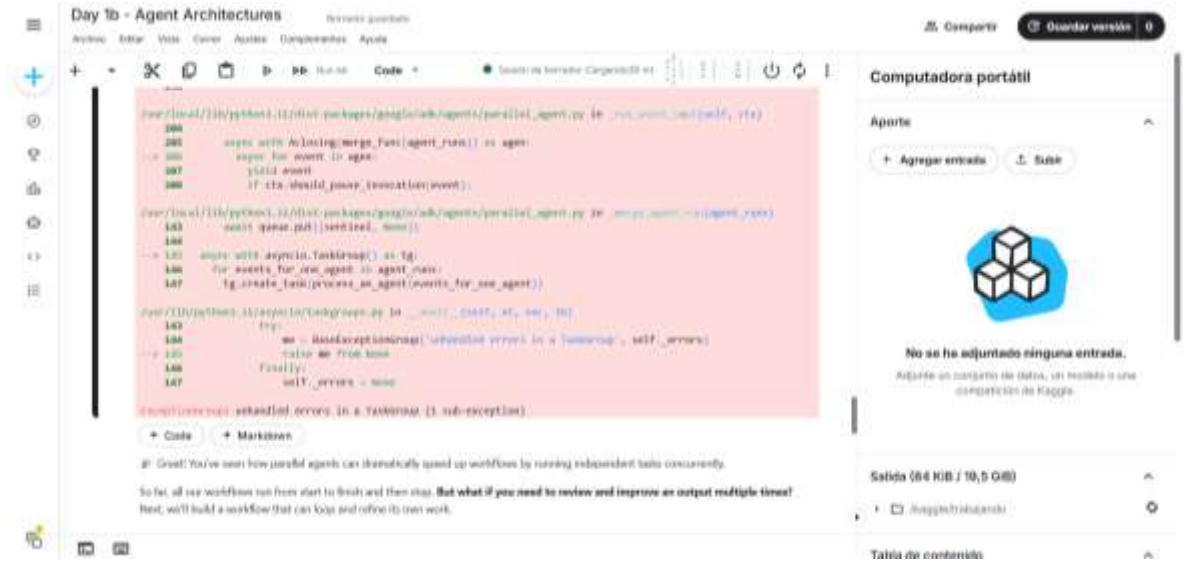



Day 1b - Agent Architectures
Compartir
Guardar versión
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
Tabla de contenido
health_researcher created.
finance_researcher created.
+ Code
+ Markdown


Day 1b - Agent Architectures
Compartir
Guardar versión
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
Tabla de contenido
aggregator_agent created.

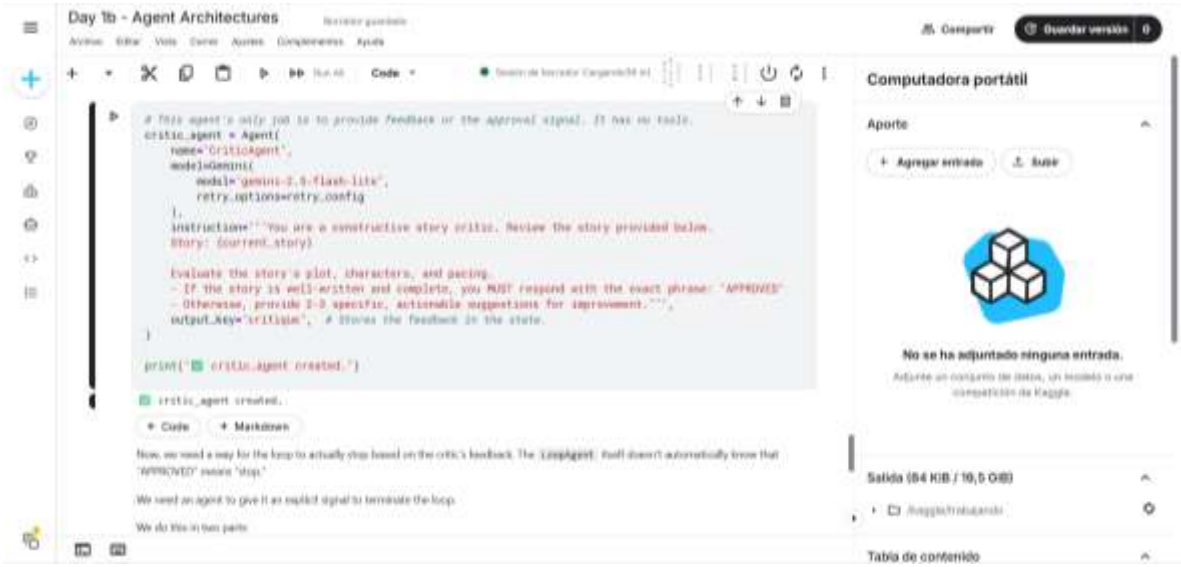

Day 1b - Agent Architectures
Compartir
Guardar versión
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
Parallel and Sequential Agents created.
+ Code
+ Markdown
Let's run the agent and give it a prompt to research the given topics.

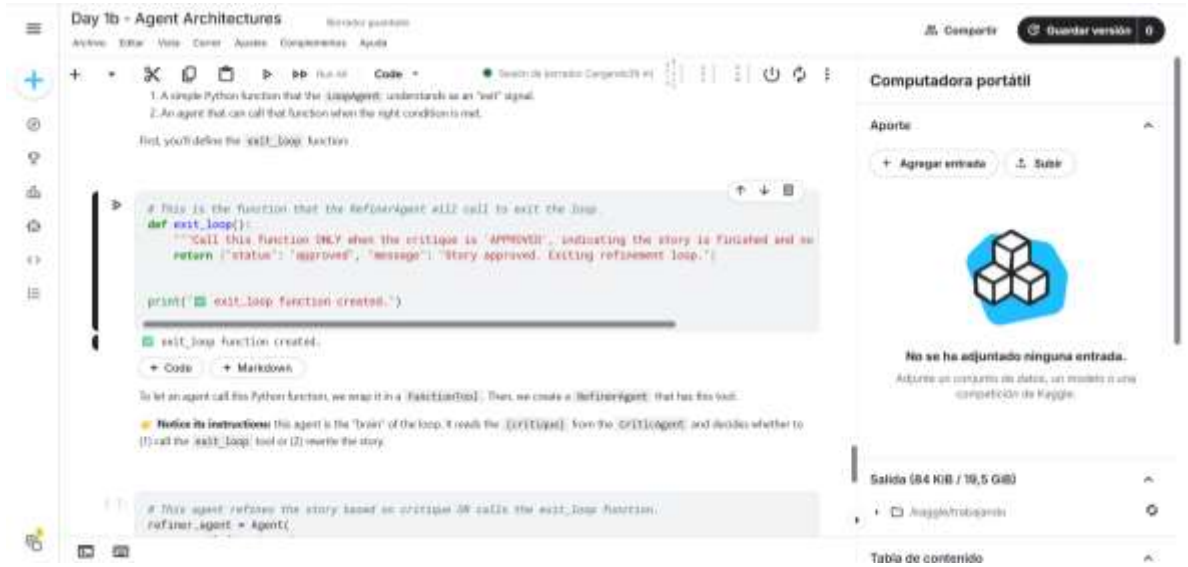

Day 1b - Agent Architectures
Archivo Editar Vista Cuenta Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
"Run the daily executive briefing on Tech, Health, and Finance"
)
*** Created new session: debug_session_id
User > Run the daily executive briefing on Tech, Health, and Finance
Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_debug(self, user_messages, user_id, session_id, run_config, quiet, verbose)
1031 print(f"user > {message}")
1032
-> 1033 async for event in self.run_async(
1034 user_id=user_id,
1035 session_id=session.id,
/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_async(self, user_id, session_id, invocation_id, new_message, state_delta, run_config)
401
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle
Salida (84 KiB / 19,5 GiB)
/kaggle/working
Tabla de contenido

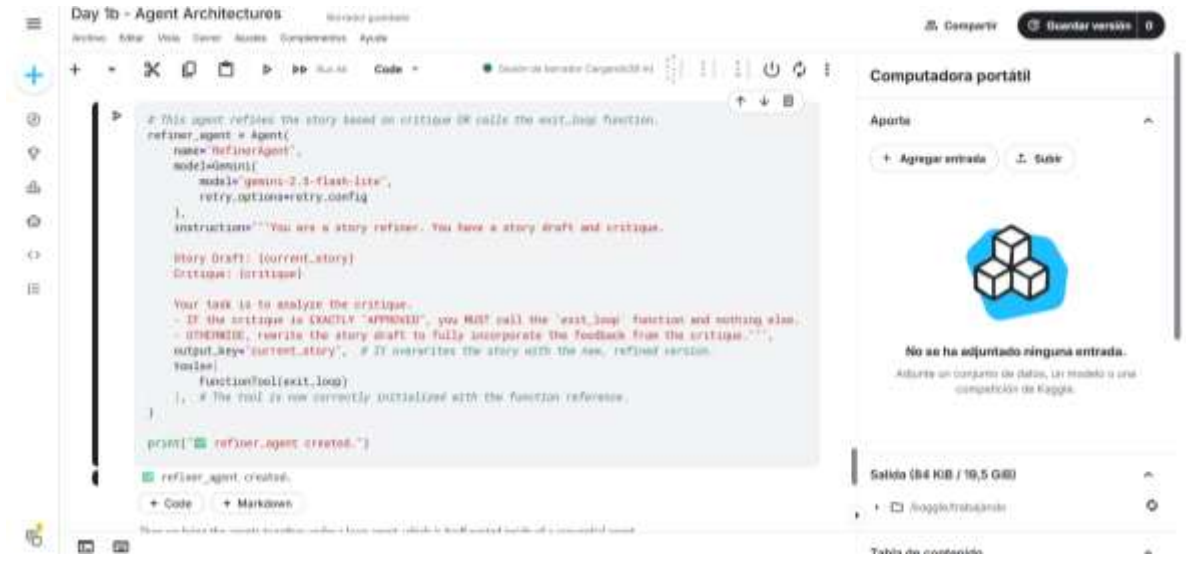

Day 1b - Agent Architectures
Archivo Editar Vista Cuenta Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
/usr/local/lib/python3.11/dist-packages/google/adk/agents/parallel_agent.py in _run_async_impl(self, ctx)
204
205 async with Aclosing(merge_func(agent_runs)) as agen:
-> 206 async for event in agen:
207 yield event
208 if ctx.should_pause_invocation(event):
/usr/local/lib/python3.11/dist-packages/google/adk/agents/parallel_agent.py in _merge_agent_runs(agent_runs)
143 await queue.put(sentinel, None)
144
-> 145 async with asyncio.TaskGroup() as tg:
146 for events_for_one_agent in agent_runs:
147 tg.create_task(process_an_agent(events_for_one_agent))
/usr/lib/python3.11/asyncio/taskgroups.py in __exit__(self, et, exc, tb)
143 try:
144 me = BaseExceptionGroup('unhandled errors in a TaskGroup', self._errors)
-> 145 raise me from None
146 finally:
147 self._errors = None
ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+ Code
+ Markdown
Great! You've seen how parallel agents can dramatically speed up workflows by running independent tasks concurrently.
So far, all our workflows run from start to finish and then stop. But what if you need to review and improve an output multiple times?
Next, we'll build a workflow that can loop and refine its own work.
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle
Salida (84 KiB / 19,5 GiB)
/kaggle/working
Tabla de contenido

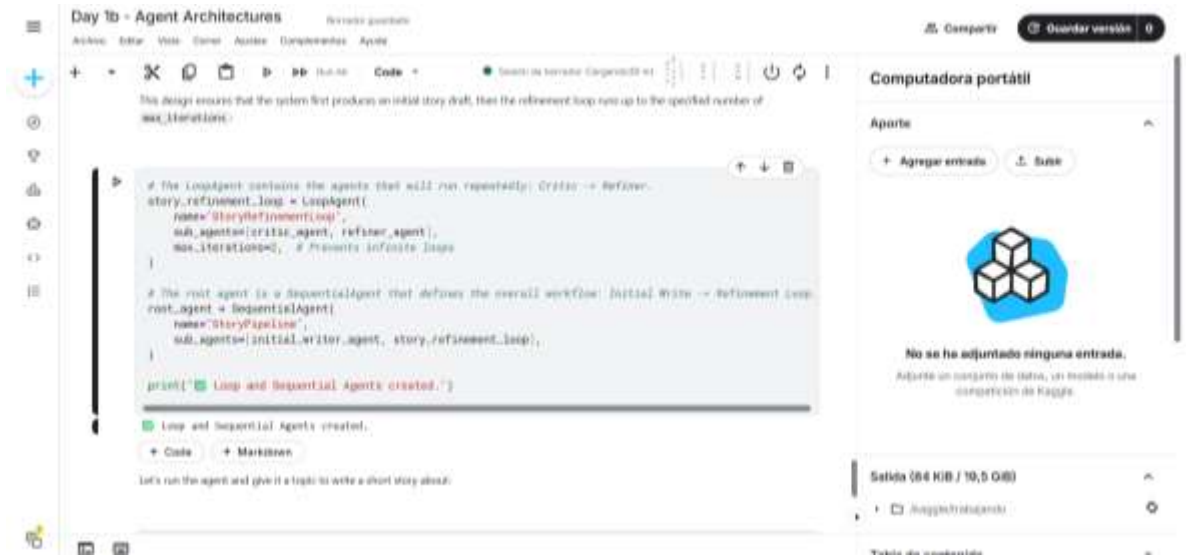
Section 5: Loop Workflows - The Refinement Cycle

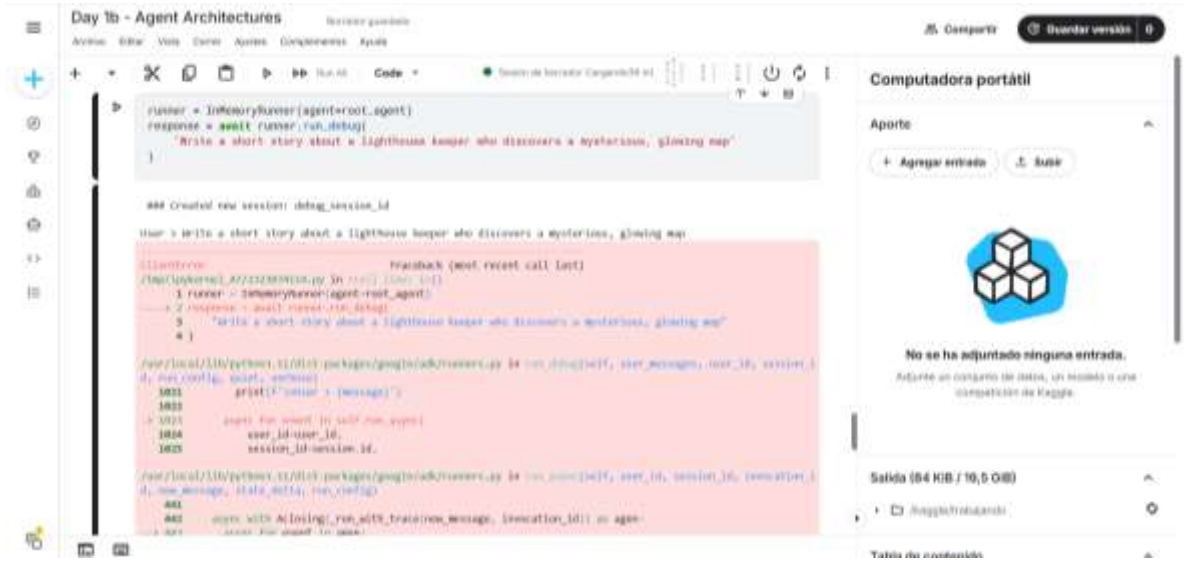
Comprendí la implementación de un sistema que utiliza un *LoopAgent* y un *SequentialAgent*, en el cual un agente redacta una historia, otro la revisa y un tercero la mejora de manera iterativa hasta lograr una versión final optimizada.

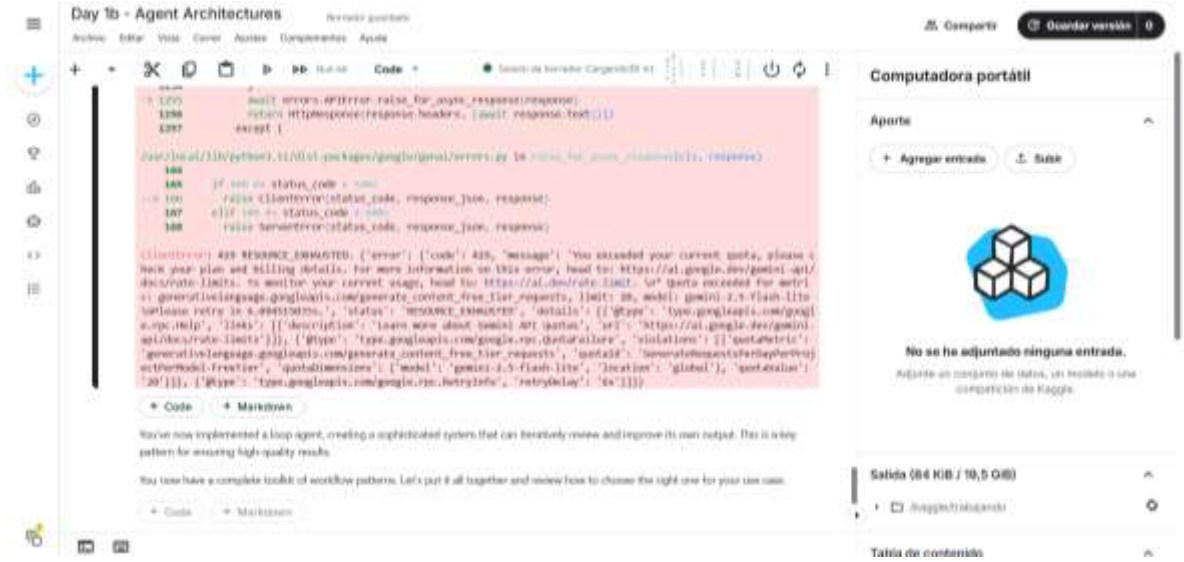



Day 1b - Agent Architectures
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
1. A simple Python function that the LoopAgent understands as an "exit" signal.
2. An agent that can call that function when the right condition is met.
First, you'll define the exit_loop function.
# This is the function that the RefinerAgent will call to exit the loop.
def exit_loop():
"""Call this function ONLY when the critique is 'APPROVED', indicating the story is finished and no
return {"status": "approved", "message": "Story approved. Exiting refinement loop."}
print("✅ exit_loop function created.")
✅ exit_loop function created.
+ Code
+ Markdown
To let an agent call this Python function, we wrap it in a FunctionTool. Then, we create a RefinerAgent that has this tool.
Notice its instructions: this agent is the "brain" of the loop. It reads the {critique} from the CriticAgent and decides whether to (1) call the exit_loop tool or (2) rewrite the story.
# This agent refines the story based on critique OR calls the exit_loop function.
refiner_agent = Agent(
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
/kaggle/trabajando
Tabla de contenido


Day 1b - Agent Architectures
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
# This agent refines the story based on critique OR calls the exit_loop function.
refiner_agent = Agent(
name="RefinerAgent",
model=Gemini(
model="gemini-2.5-flash-lite",
retry_options=retry_config
),
instruction="""You are a story refiner. You have a story draft and critique.
Story Draft: {current_story}
Critique: {critique}
Your task is to analyze the critique.
- IF the critique is EXACTLY "APPROVED", you MUST call the 'exit_loop' function and nothing else.
- OTHERWISE, rewrite the story draft to fully incorporate the feedback from the critique.""",
output_key="current_story", # It overwrites the story with the new, refined version.
tools=[
FunctionTool(exit_loop)
], # The tool is now correctly initialized with the function reference.
)
print("✅ refiner_agent created.")
✅ refiner_agent created.
+ Code
+ Markdown
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
/kaggle/trabajando

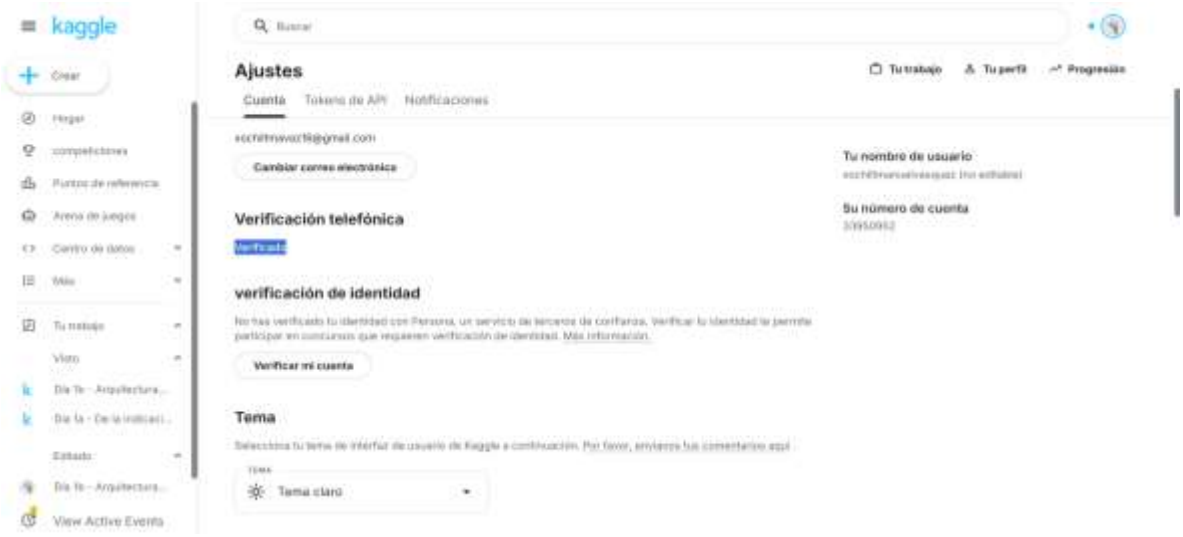

Day 1b - Agent Architectures
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
This design ensures that the system first produces an initial story draft, then the refinement loop runs up to the specified number of max_iterations.
# The LoopAgent contains the agents that will run repeatedly: Critic -> Refiner.
story_refinement_loop = LoopAgent(
name="StoryRefinementLoop",
sub_agents=[critic_agent, refiner_agent],
max_iterations=2, # Prevents infinite loops
)
# The root agent is a SequentialAgent that defines the overall workflow: Initial Write -> Refinement Loop
root_agent = SequentialAgent(
name="StoryPipeline",
sub_agents=[initial_writer_agent, story_refinement_loop],
)
print("✅ Loop and Sequential Agents created.")
✅ Loop and Sequential Agents created.
+ Code
+ Markdown
Let's run the agent and give it a topic to write a short story about.
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Adjunte un conjunto de datos, un modelo o una competición de Kaggle.
Salida (84 KiB / 19,5 GiB)
/kaggle/trabajando

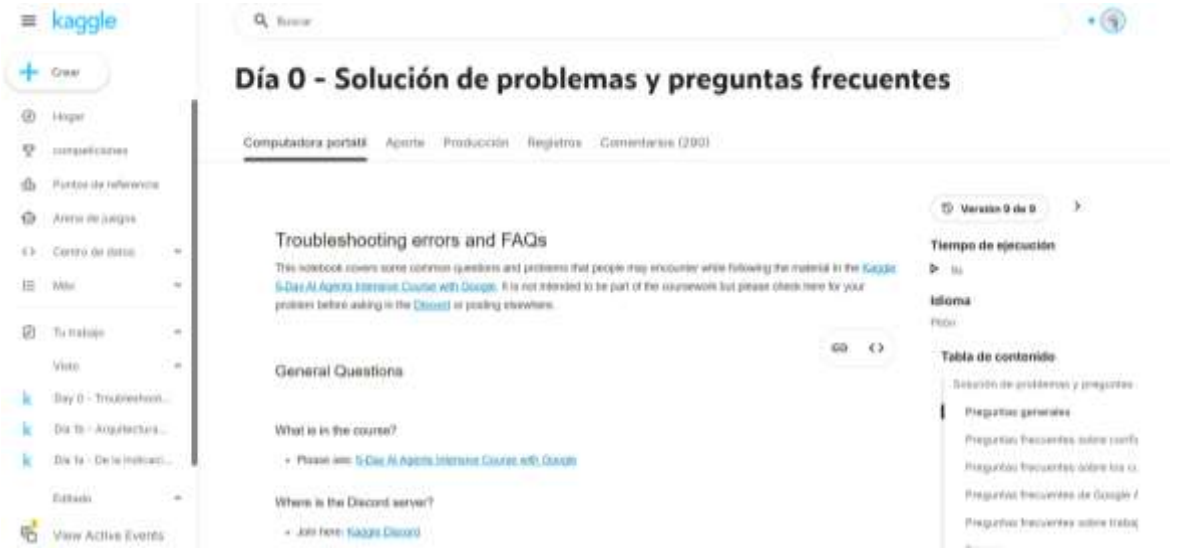

Day 1b - Agent Architectures
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
'Write a short story about a lighthouse keeper who discovers a mysterious, glowing map'
)
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Salida (84 KiB / 10,5 GiB)


Day 1b - Agent Architectures
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
Computadora portátil
Aporte
+ Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
+ Code
+ Markdown
You've now implemented a loop agent, creating a sophisticated system that can iteratively review and improve its own output. This is a key pattern for ensuring high-quality results.
You now have a complete toolkit of workflow patterns. Let's put it all together and review how to choose the right one for your use case.
Salida (84 KiB / 10,5 GiB)

Section 6: Summary - Choosing the Right Pattern

Comprendí los distintos tipos de agentes (Sequential, Parallel, Loop y LLM-based) y su aplicación en la orquestación de sistemas multiagente, identificando en qué situaciones conviene utilizar cada uno según el tipo de problema. Asimismo, verifiqué mi cuenta de Kaggle mediante mi número telefónico para poder acceder y aprovechar los codelabs del curso.

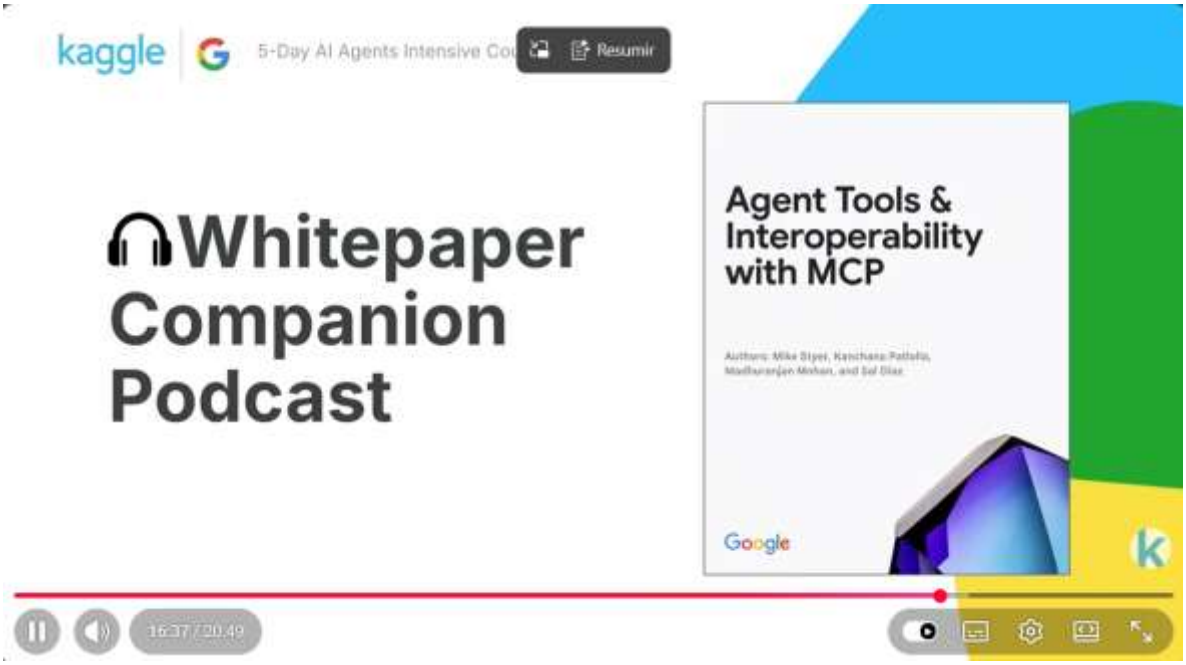




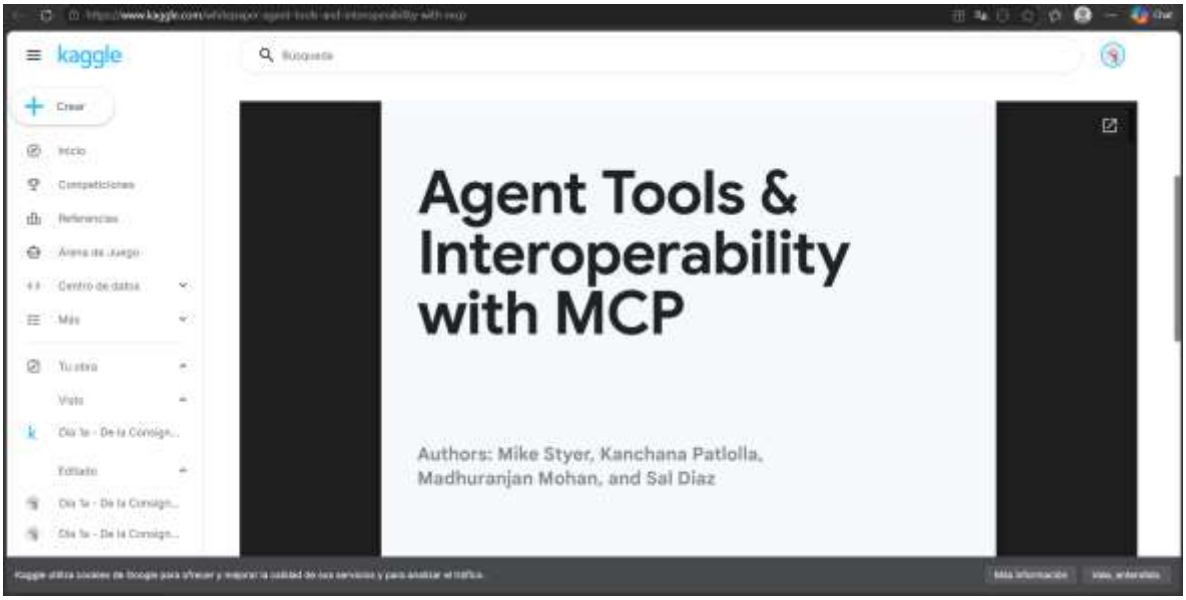
DIA 2

En el Día 2, titulado “**Herramientas para agentes e interoperabilidad con el Protocolo de Contexto de Modelo (MCP)”**, aprendí a integrar herramientas externas en mis agentes y a comprender el funcionamiento del MCP. Además, desarrollé herramientas en Python y trabajé con flujos en los que el agente puede pausar su ejecución y continuar posteriormente con la aprobación de un usuario.

Para comenzar, escuché el episodio del podcast de la Unidad 2, el cual ofreció un resumen de los conceptos clave relacionados con la introducción a los agentes.



Leí el documento técnico sobre herramientas de agente e interoperabilidad con MCP, para comprender cómo los agentes se conectan con sistemas externos.

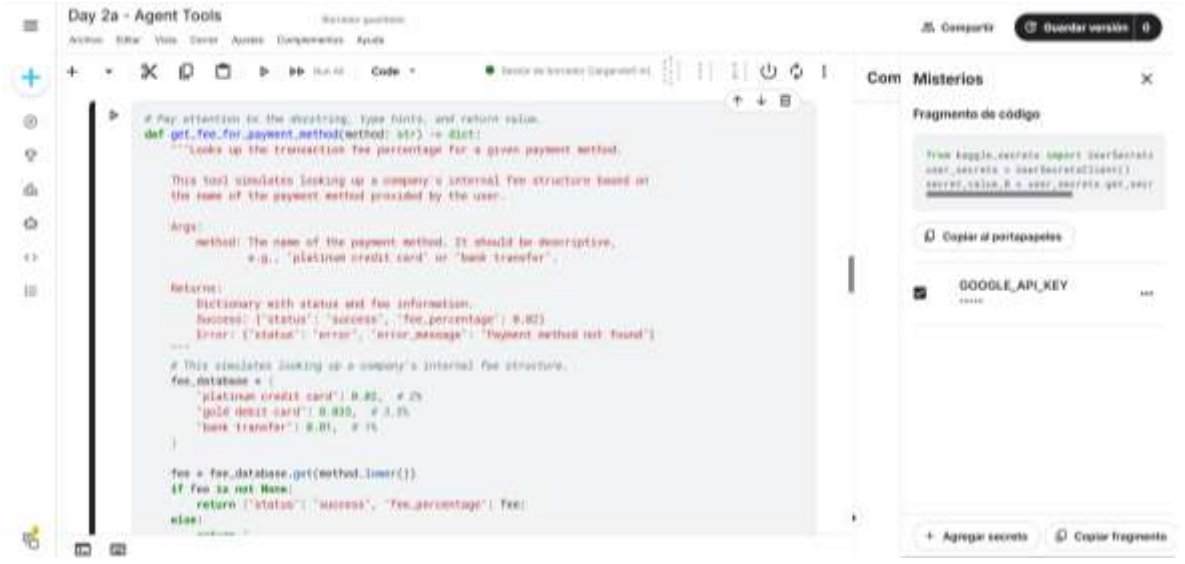


Section 1: Setup

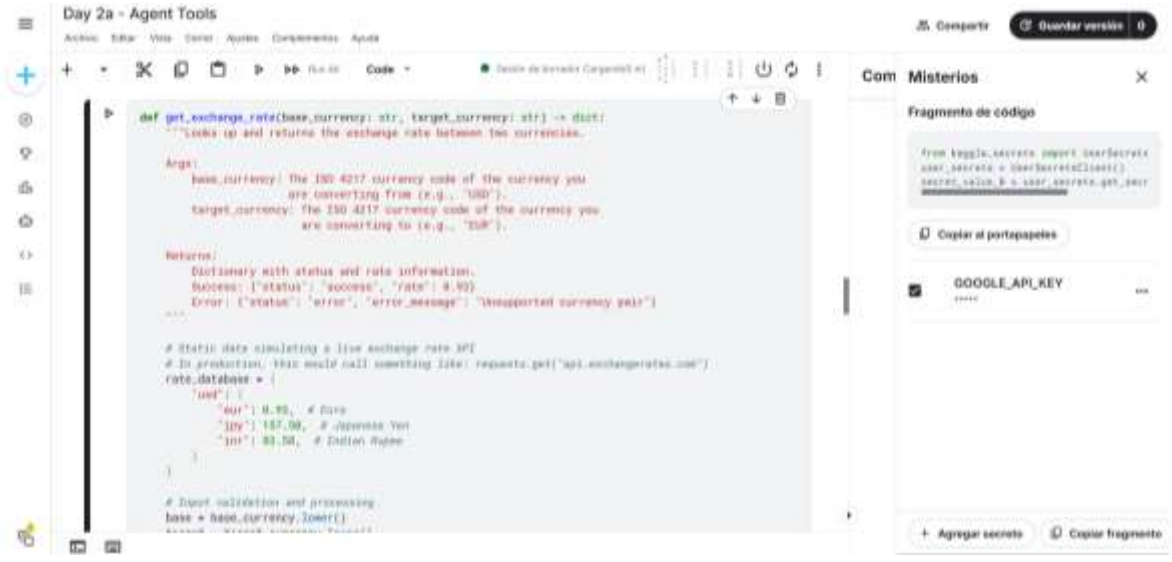
Utilicé el entorno que ya había configurado previamente, lo que me permitió omitir los pasos iniciales. Gracias a esto, pude enfocarme desde el inicio en las actividades del laboratorio y en comprender el uso de las herramientas y funciones del sistema.

Section 2: What are Custom Tools?

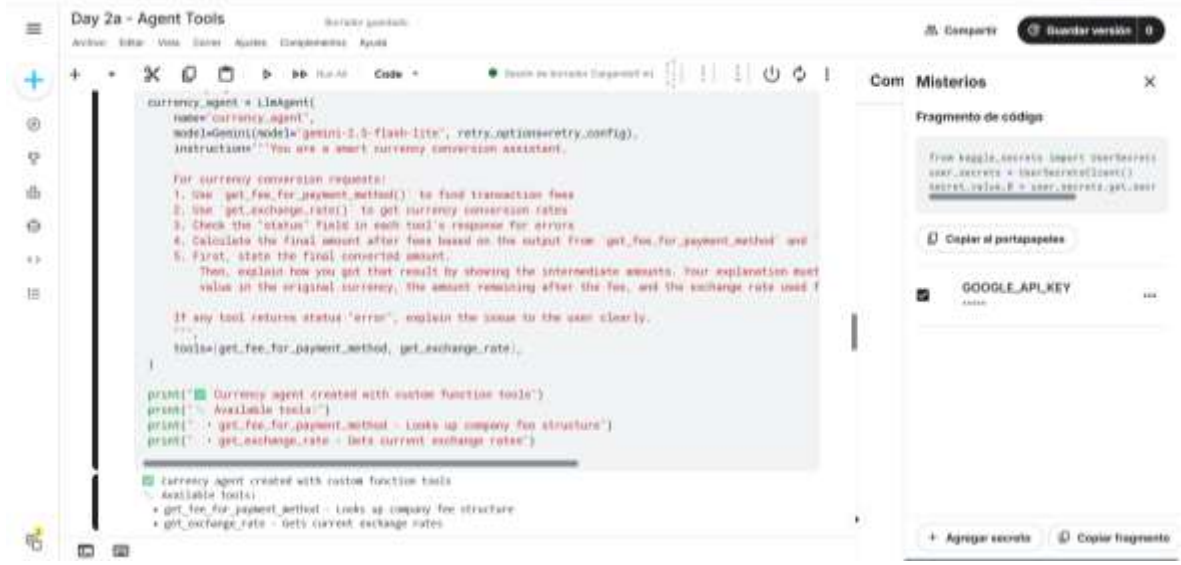
Entendí cómo definir herramientas personalizadas en Python e integrarlas en un agente, el cual es capaz de realizar conversiones de moneda y calcular comisiones utilizando lógica propia y generando respuestas estructuradas.

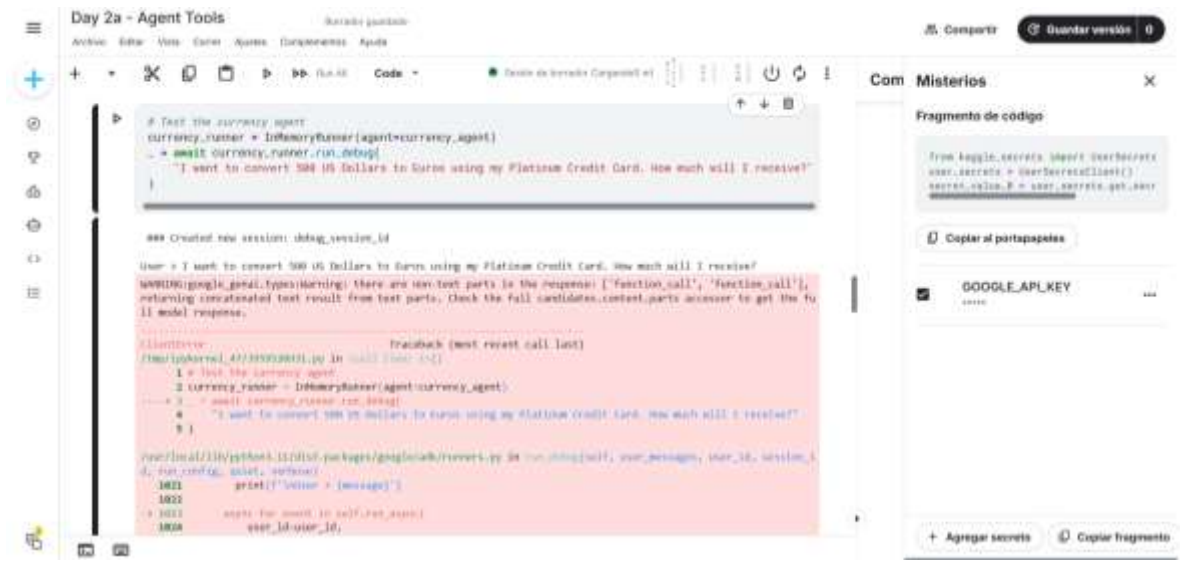


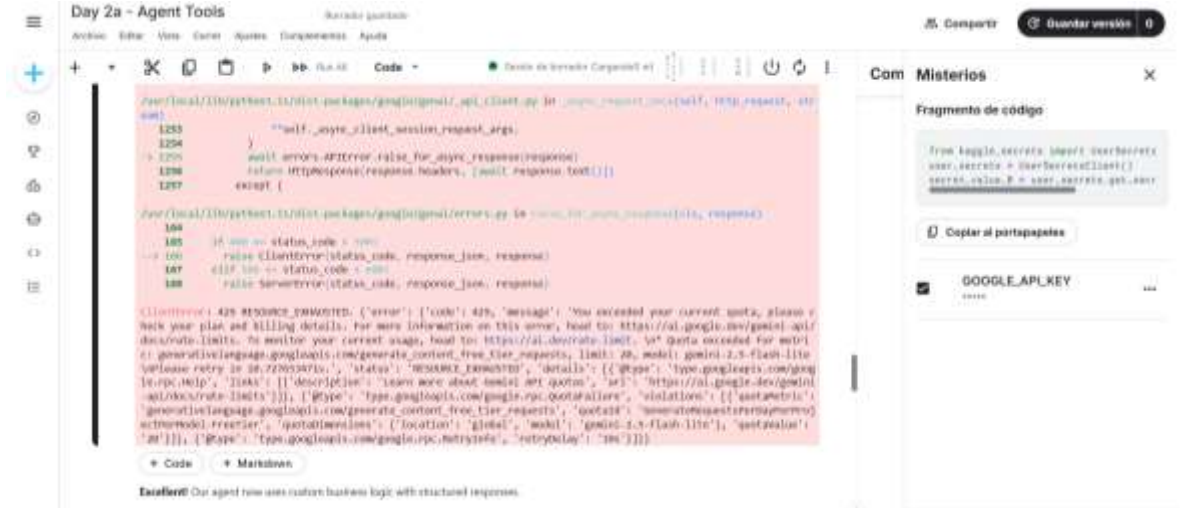

Day 2a - Agent Tools
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
Code
Markdown


Day 2a - Agent Tools
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
Code
Markdown


Day 2a - Agent Tools
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
Code
Markdown

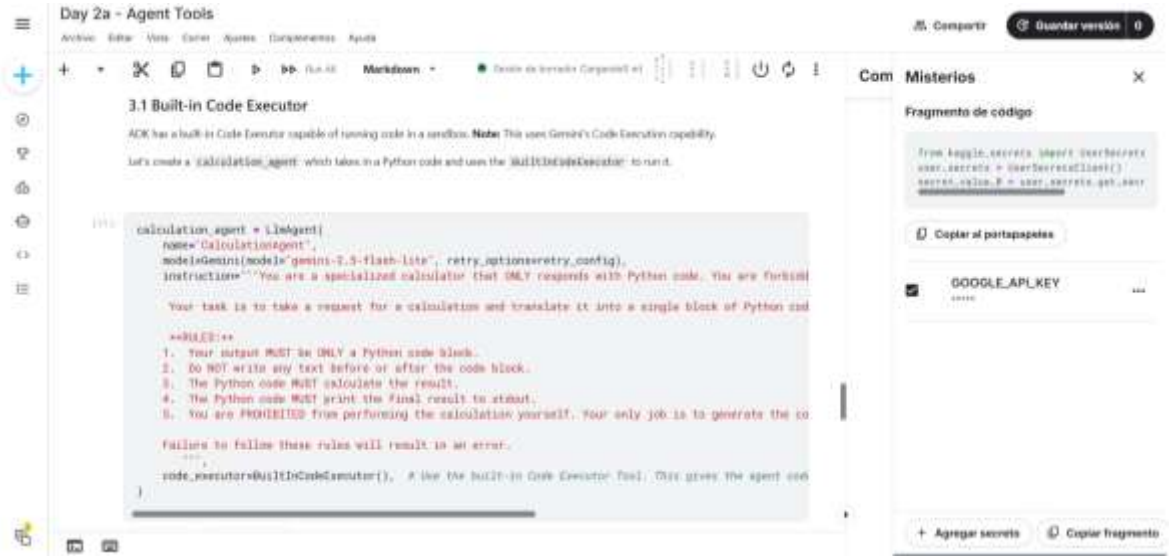

Day 2a - Agent Tools
Compartir
Guardar versión
Code
currency_agent = LlmAgent(
name='currency_agent',
model=Gemini(model='gemini-2.5-flash-lite', retry_options=retry_config),
instructions='''You are a smart currency conversion assistant.
For currency conversion requests:
1. Use `get_fee_for_payment_method()` to find transaction fees
2. Use `get_exchange_rate()` to get currency conversion rates
3. Check the 'status' field in each tool's response for errors
4. Calculate the final amount after fees based on the output from `get_fee_for_payment_method` and
5. First, state the final converted amount.
Then, explain how you got that result by showing the intermediate amounts. Your explanation must
value in the original currency, the amount remaining after the fee, and the exchange rate used f
If any tool returns status 'error', explain the issue to the user clearly.
'''
tools=[get_fee_for_payment_method, get_exchange_rate],
)
print('✅ Currency agent created with custom function tools')
print('\n Available tools:')
print(' • get_fee_for_payment_method - Looks up company fee structure')
print(' • get_exchange_rate - Gets current exchange rates')
✅ Currency agent created with custom function tools
Available tools:
• get_fee_for_payment_method - Looks up company fee structure
• get_exchange_rate - Gets current exchange rates
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

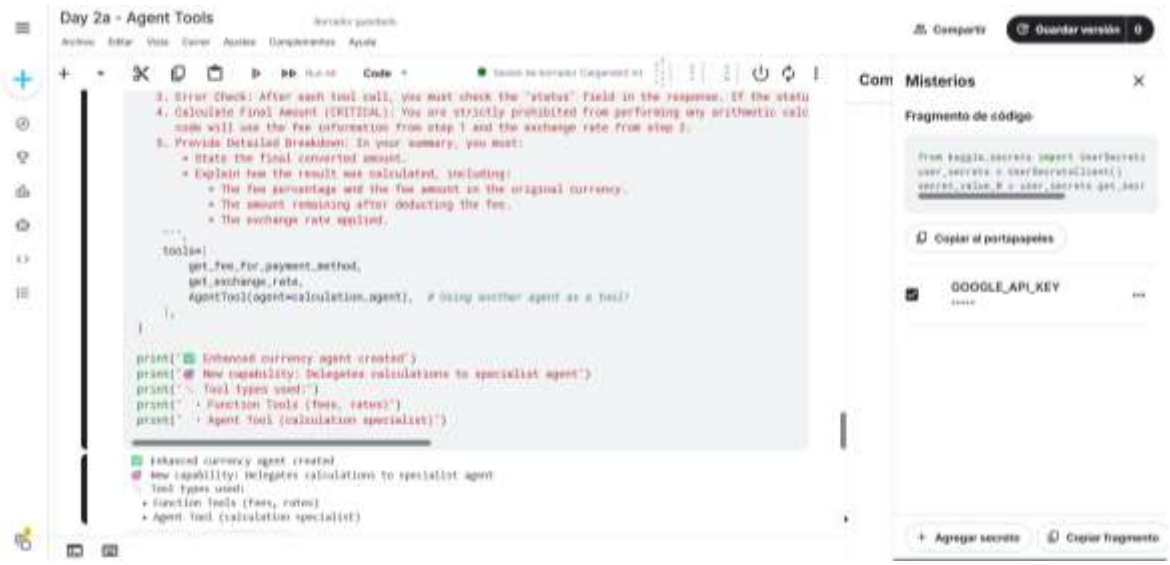

Day 2a - Agent Tools
Compartir
Guardar versión
Code
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
"I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)
Created new session: debug_session_id
User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call', 'function_call'],
returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the fu
ll model response.
ClientError
Traceback (most recent call last)
/tmp/ipykernel_47/3959180731.py in <cell line: 1>()
1 # Test the currency agent
2 currency_runner = InMemoryRunner(agent=currency_agent)
----> 3 _ = await currency_runner.run_debug(
4 "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
5)
/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_debug(self, user_messages, user_id, session_i
d, run_config, quiet, verbose)
1021 print(f'User > {message}')
1022
1023 async for event in self.run_async(
1024 user_id=user_id,
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

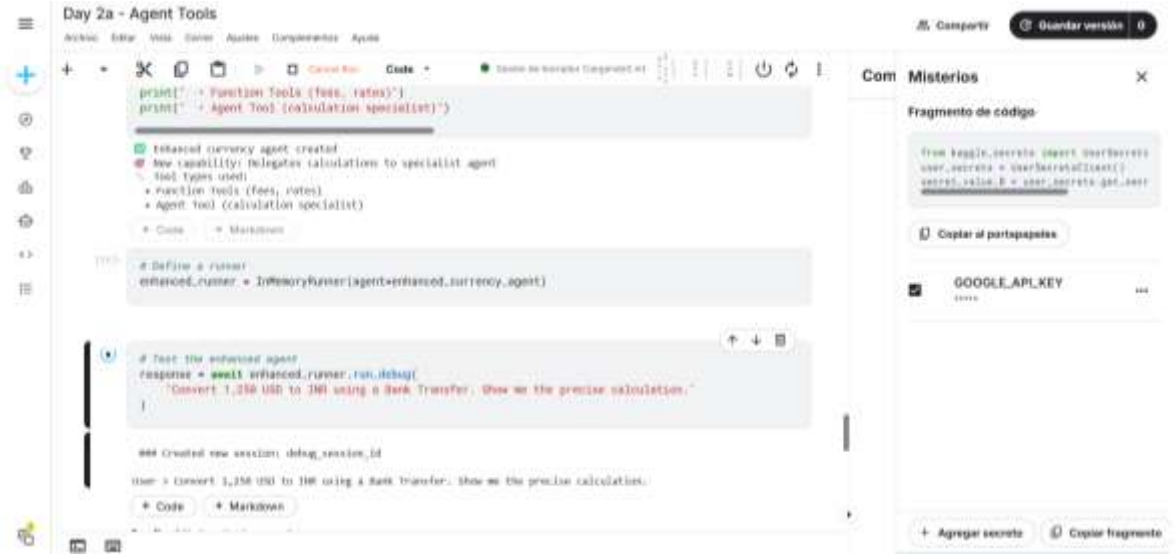

Day 2a - Agent Tools
Compartir
Guardar versión
Code
/usr/local/lib/python3.11/dist-packages/google/genai/_api_client.py in async_request_once(self, http_request, str
eam)
1253 **self._async_client_session_request_args,
1254)
1255 await errors.APIError.raise_for_async_response(response)
1256 return HttpResponse(response.headers, [await response.text()])
1257 except (
/usr/local/lib/python3.11/dist-packages/google/genai/errors.py in raise_for_async_response(cls, response)
184
185 if 400 <= status_code < 500:
--> 186 raise ClientError(status_code, response_json, response)
187 elif 500 <= status_code < 600:
188 raise ServerError(status_code, response_json, response)
ClientError: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please c
heck your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/
docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric
: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-2.5-flash-lite
\nPlease retry in 10.729534791s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/goog
le.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini
-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric':
'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProj
ectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.5-flash-lite'}, 'quotaValue':
'20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '10s'}]}}
+ Code
+ Markdown
Excellent! Our agent now uses custom business logic with structured responses.
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY

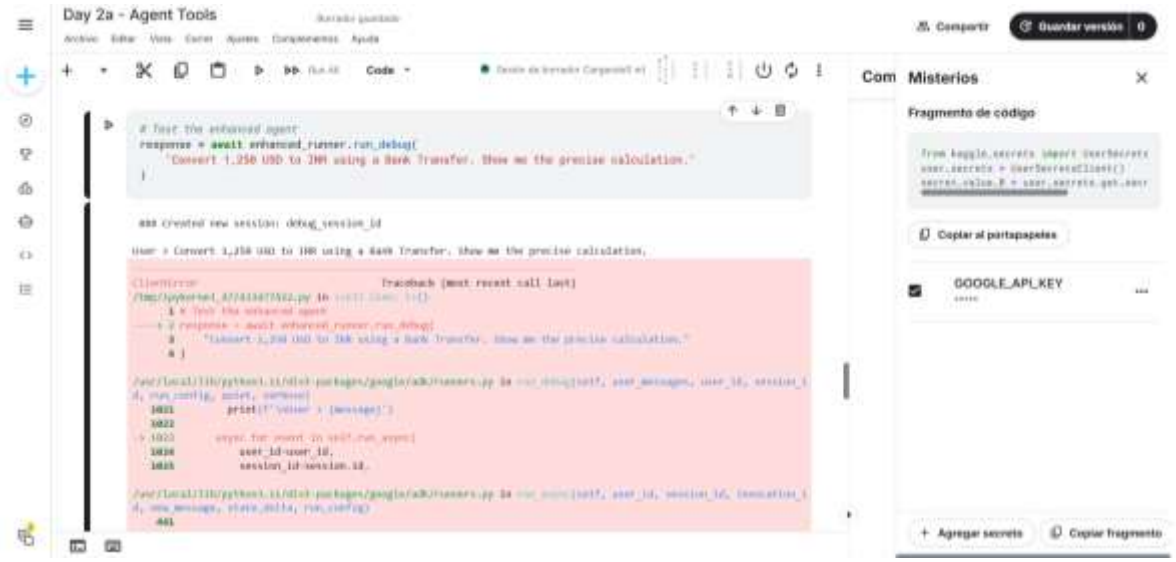
Section 3: Improving Agent Reliability with Code

Comprendí la implementación de un agente que utiliza el **Code Executor** integrado de ADK, el cual permite ejecutar código en Python dentro de un entorno seguro y obtener resultados de manera automática.

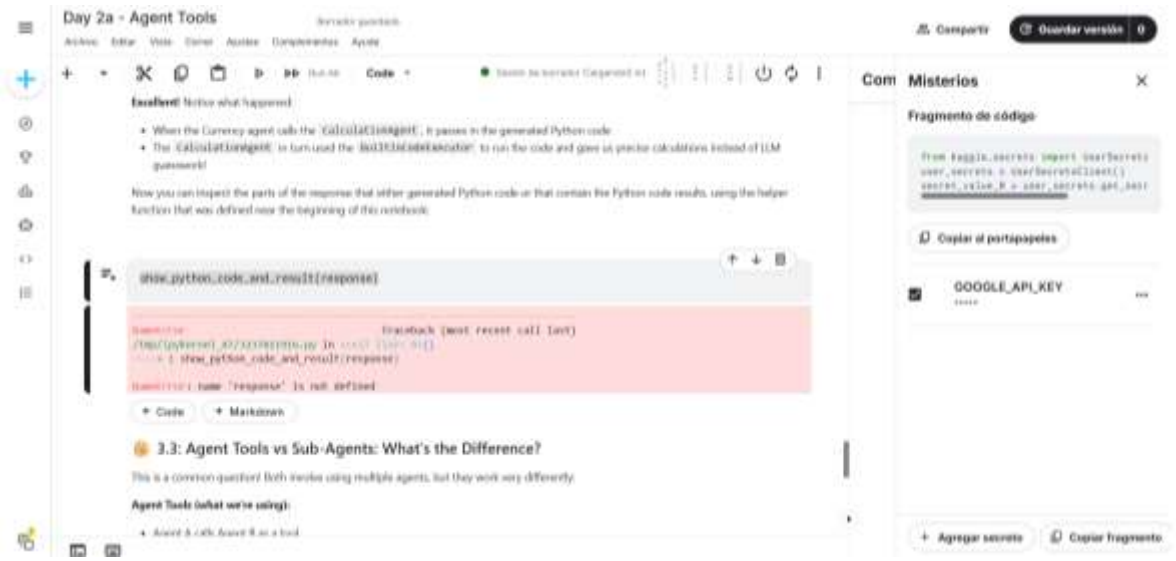





Day 2a - Agent Tools
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
print(' ✓ Function Tools (fees, rates)')
print(' ✓ Agent Tool (calculation specialist)')
Enhanced currency agent created
New capability: Delegates calculations to specialist agent
Tool types used:
• Function Tools (fees, rates)
• Agent Tool (calculation specialist)
+ Code
+ Markdown
# Define a runner
enhanced_runner = InMemoryRunner(agent=enhanced_currency_agent)
# Test the enhanced agent
response = await enhanced_runner.run_debug(
'Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.'
)
Created new session: debug_session_id
User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.
+ Code
+ Markdown
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento


Day 2a - Agent Tools
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
# Test the enhanced agent
response = await enhanced_runner.run_debug(
'Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.'
)
Created new session: debug_session_id
User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.
ClientError Traceback (most recent call last)
/tmp/ipykernel_4714347552.py in <cell line: 1>()
1 # Test the enhanced agent
----> 2 response = await enhanced_runner.run_debug(
3 'Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.'
4)
/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_debug(self, user_messages, user_id, session_id, run_config, quiet, verbose)
1021 print(f"\nUser > {message}")
1022
1023 async for event in self.run_async(
1024 user_id=user_id,
1025 session_id=session.id,
/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_async(self, user_id, session_id, invocation_id, new_message, state_delta, run_config)
441
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento


Day 2a - Agent Tools
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
1256 return httpx.Response(response.headers, [await response.text()])
1257 except (
/usr/local/lib/python3.11/dist-packages/google/genai/errors.py in raise_for_async_response(http_response)
164
165 if 400 <= status_code < 500:
166 raise ClientError(status_code, response_json, response)
167 elif 500 <= status_code < 600:
168 raise ServerError(status_code, response_json, response)
ClientError: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-2.5-flash-lite\nPlease retry in 23.7\n\n', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.5-flash-lite'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '23s'}]}}
+ Code
+ Markdown
Excellent! Notice what happened:
• When the Currency agent calls the CalculationAgent, it passes in the generated Python code
• The CalculationAgent, in turn used the BuiltInCodeExecutor to run the code and gave us precise calculations instead of LLM guesswork!
Now you can inspect the parts of the response that either generated Python code or that contain the Python code results, using the helper function that was defined near the beginning of this notebook:
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento

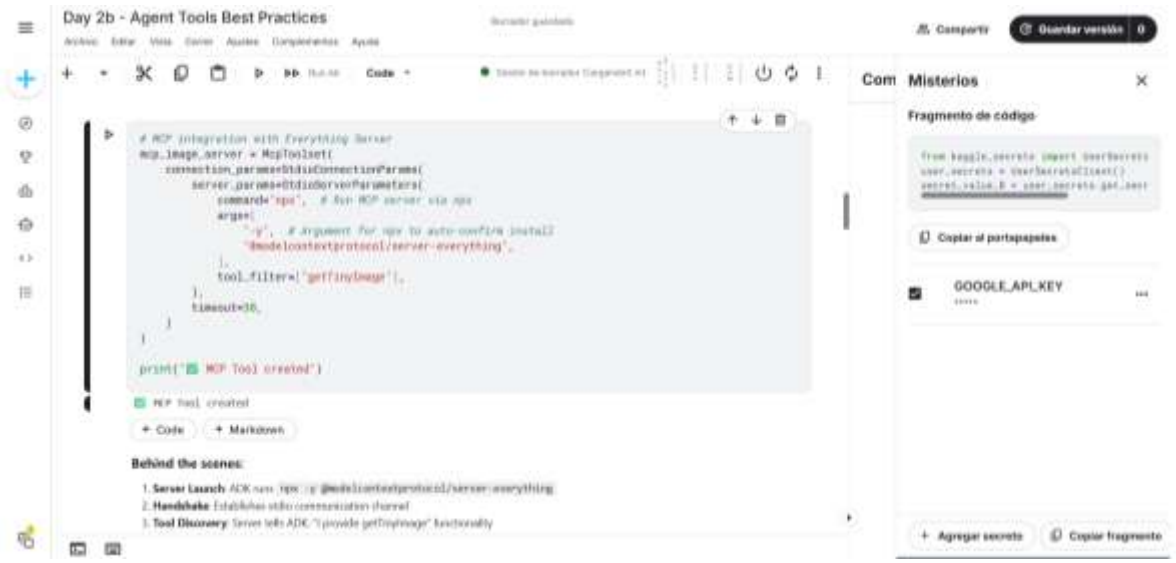


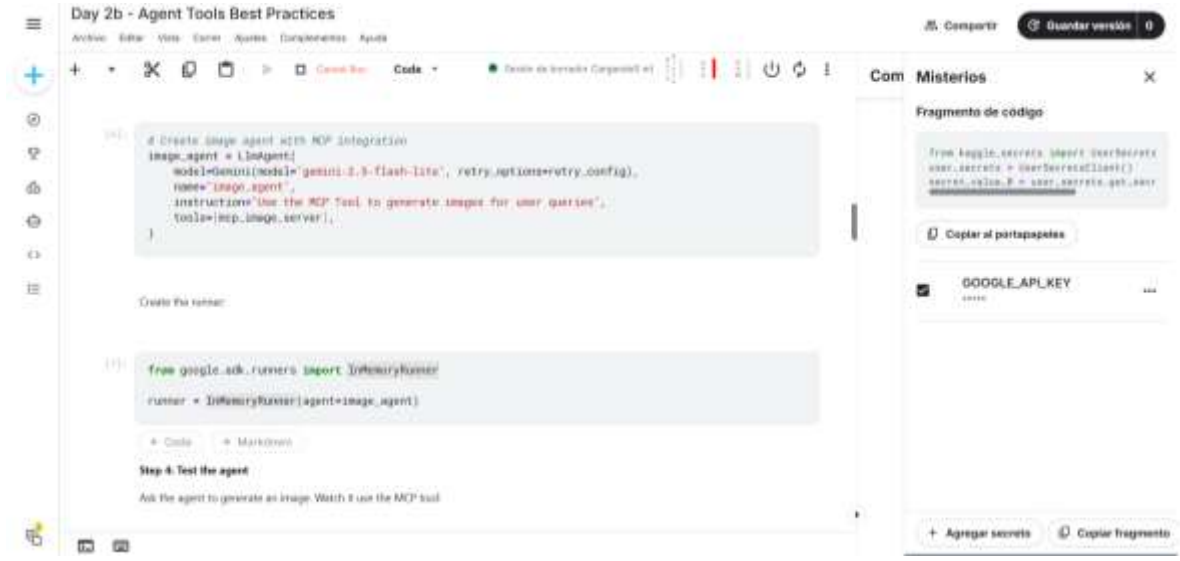
Section 1: Setup

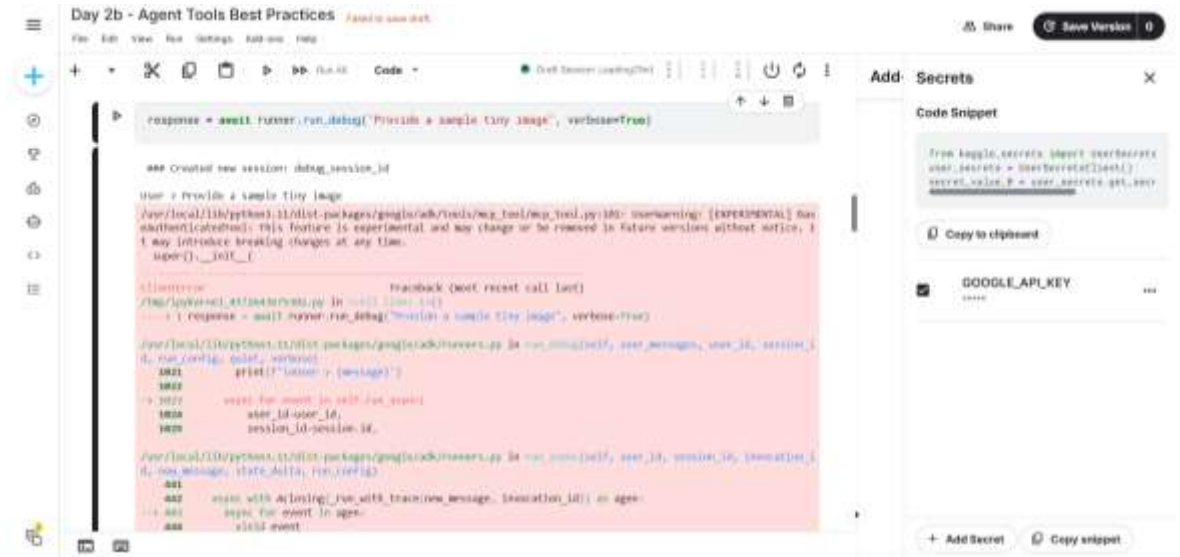
Verifiqué la configuración inicial del entorno de trabajo para poder utilizar los agentes en Kaggle. También revisé la instalación de las dependencias necesarias, la configuración de la clave API y la importación de los componentes requeridos para asegurar el correcto funcionamiento del sistema. En mi caso, no fue necesario volver a configurar la clave, ya que ya estaba lista; únicamente confirmé que todo estuviera en orden para continuar con las actividades.

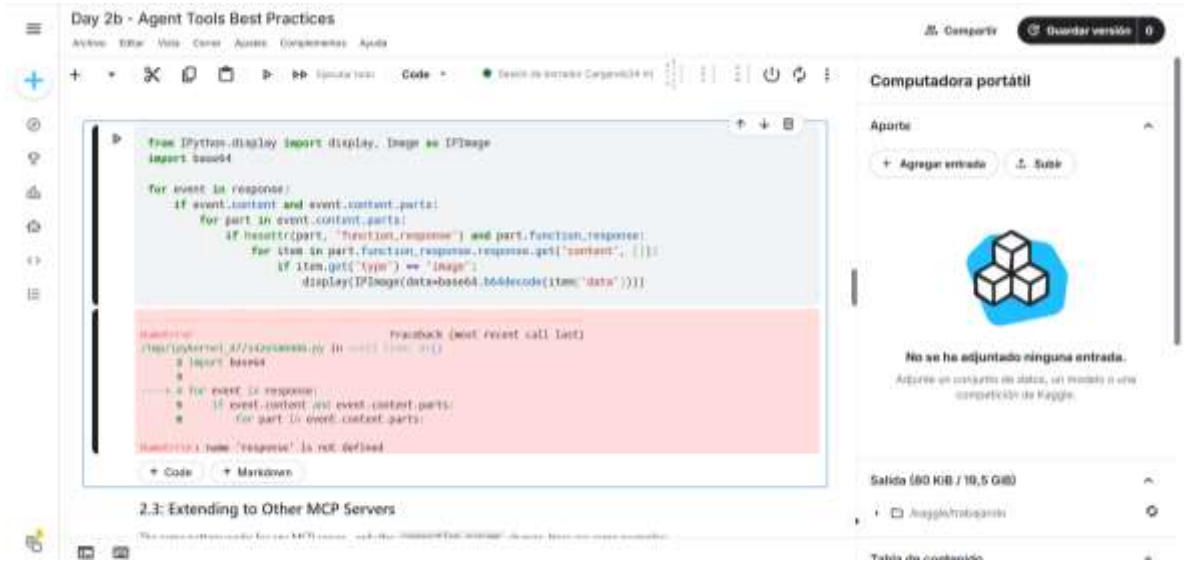
Section 2: Model Context Protocol

Comprendí que MCP permite conectar un agente a servicios externos mediante servidores preconfigurados, sin necesidad de crear APIs manualmente, lo que facilita el acceso a herramientas y datos en tiempo real.



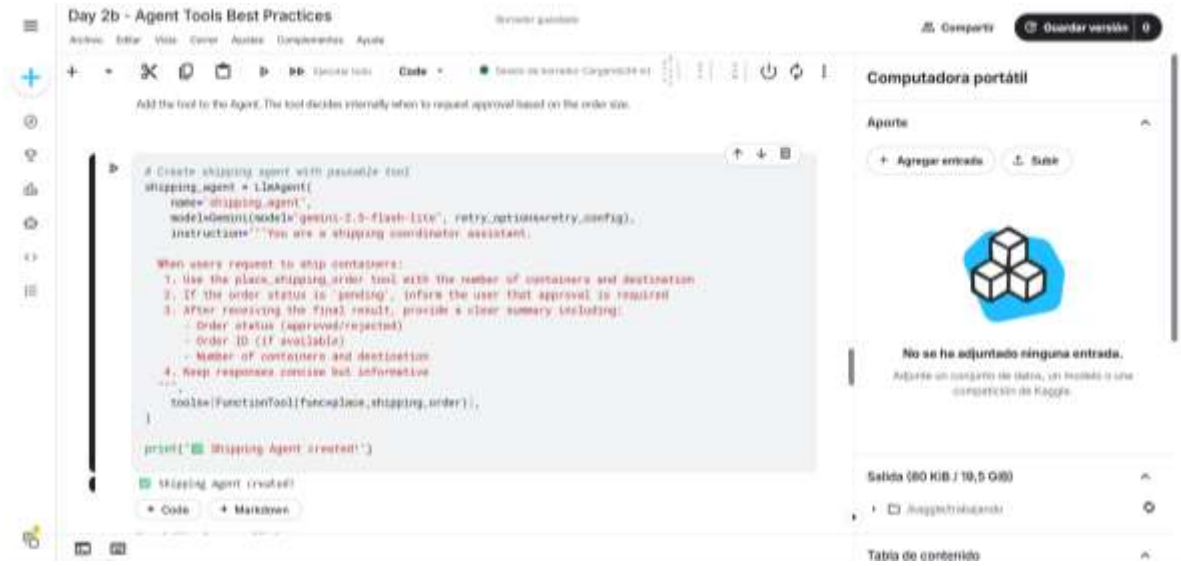

Day 2b - Agent Tools Best Practices
Compartir
Guardar versión
Code
# Create image agent with MCP integration
image_agent = LlmAgent(
model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
name="image_agent",
instruction="Use the MCP Tool to generate images for user queries",
tools=[mcp_image_server],
)
Create the runner
from google.adk.runners import InMemoryRunner
runner = InMemoryRunner(agent=image_agent)
Step 4: Test the agent
Ask the agent to generate an image. Watch it use the MCP tool
Secretos
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secrets
Copiar fragmento

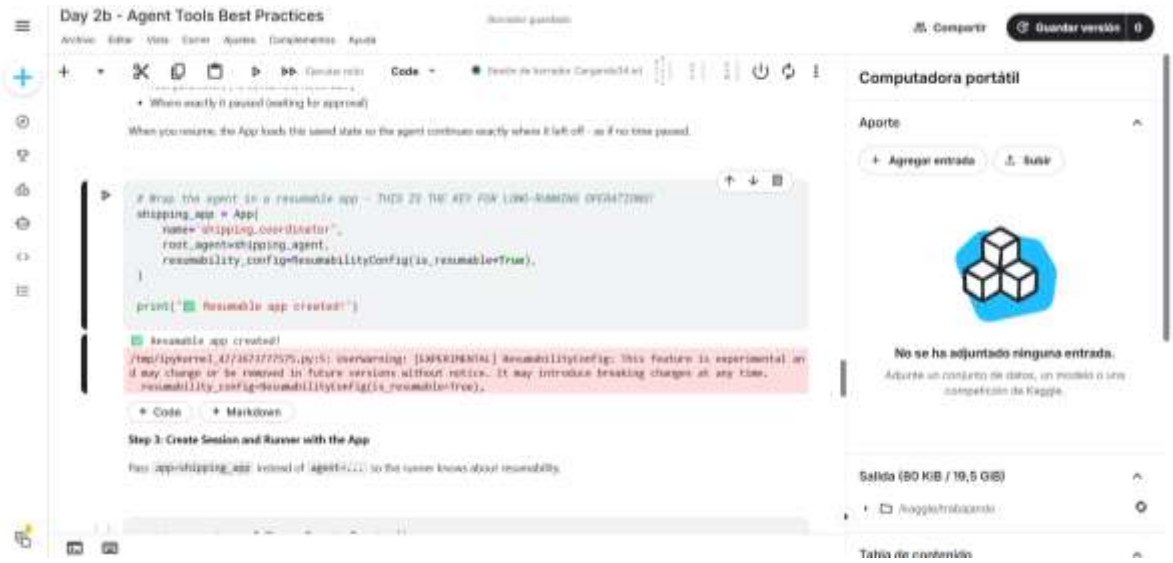

Day 2b - Agent Tools Best Practices
Share
Save Version
Code
response = await runner.run_debug("Provide a sample tiny image", verbose=True)
Secrets
Code Snippet
Copy to clipboard
GOOGLE_API_KEY
Add Secret
Copy snippet

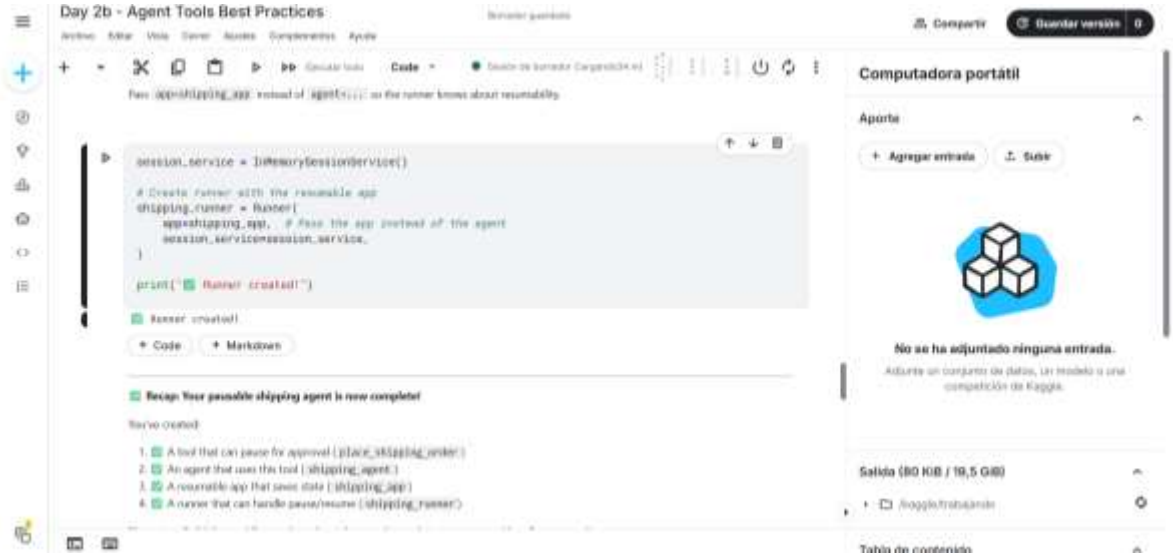

Day 2b - Agent Tools Best Practices
Compartir
Guardar versión
Code
from IPython.display import display, Image as IPImage
import base64
for event in response:
if event.content and event.content.parts:
for part in event.content.parts:
if hasattr(part, "function_response") and part.function_response:
for item in part.function_response.response.get("content", []):
if item.get("type") == "image":
display(IPImage(data=base64.b64decode(item["data"])))
Code
Markdown
2.3: Extending to Other MCP Servers
Computadora portátil
Aporte
Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Salida (80 KiB / 19,5 GiB)

Section 3: Long-Running Operations (Human-in-the-Loop)

Ejecuté un agente con herramientas de ejecución prolongada, en el que el proceso puede pausarse para solicitar la aprobación de un usuario y, posteriormente, reanudarse o cancelarse según la decisión tomada.

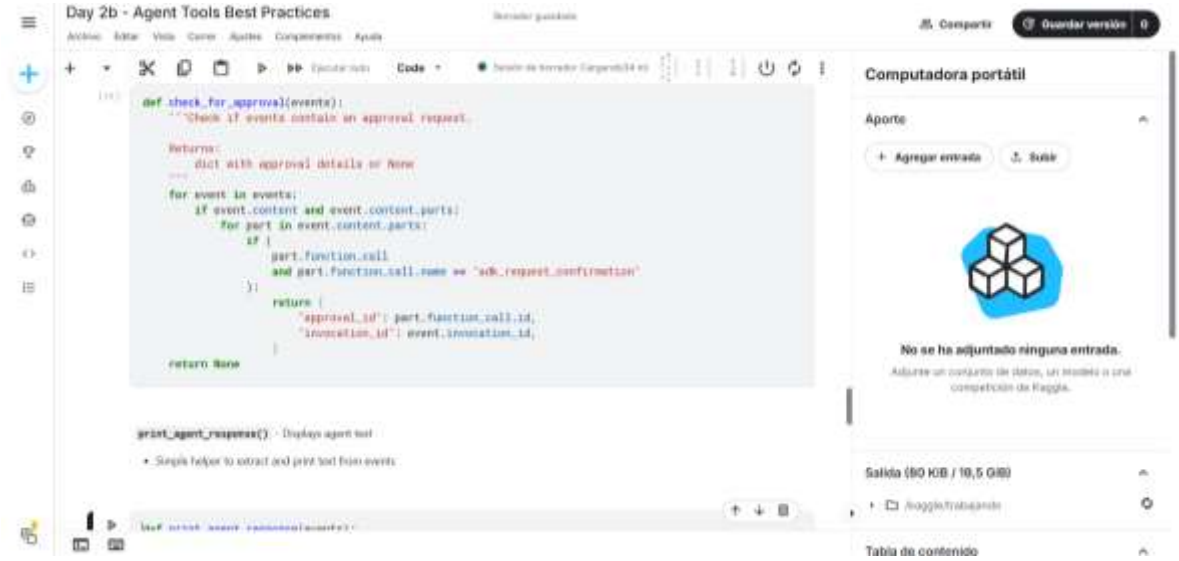


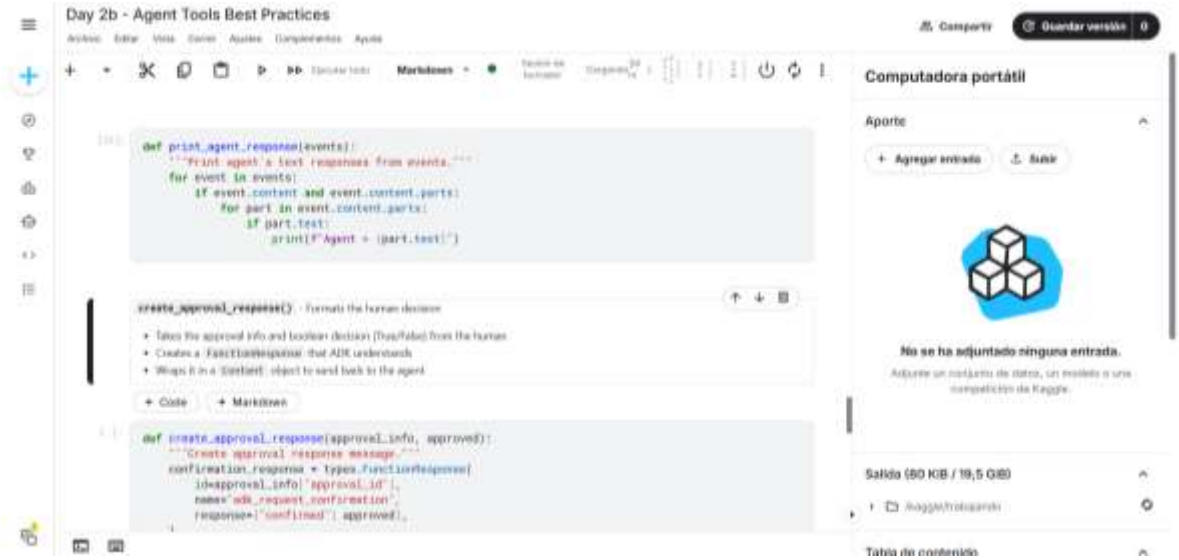


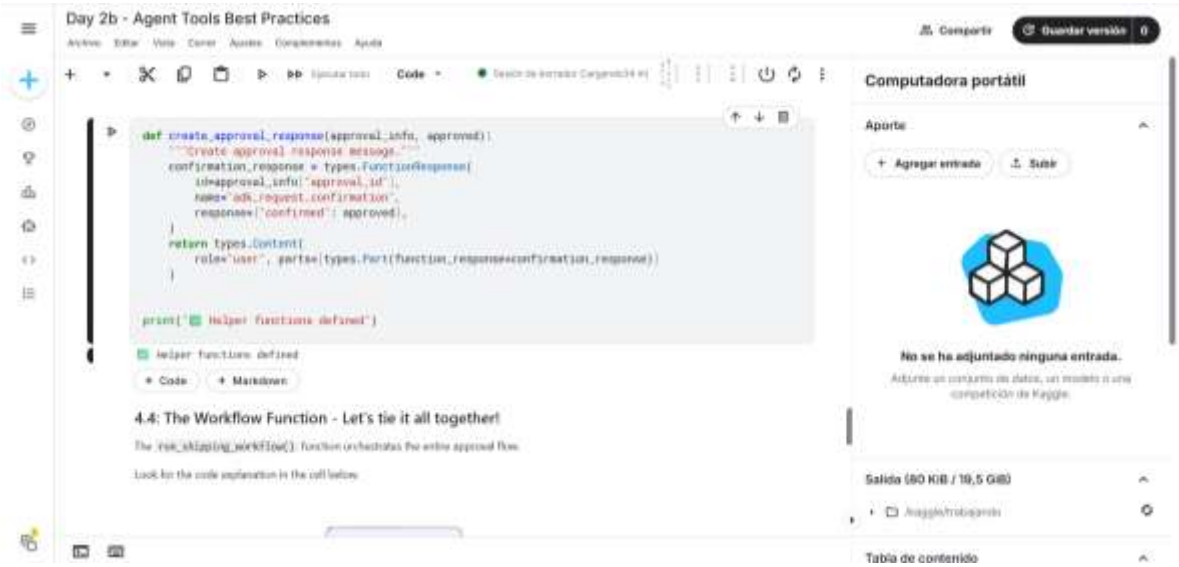


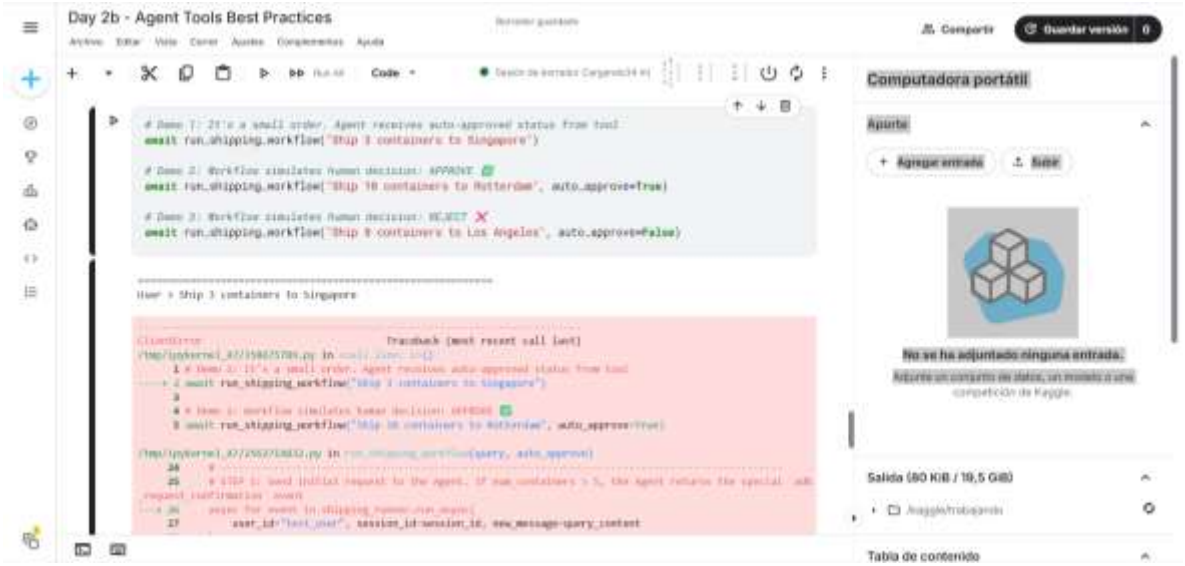
Section 4: Building the Workflow

Comprendí la configuración de un flujo en el que el agente puede detener su ejecución para recibir aprobación humana y, posteriormente, continuarla, utilizando eventos y sesiones para gestionar de manera adecuada tanto la pausa como la reanudación del proceso.


Day 2b - Agent Tools Best Practices
Computadora portátil
Aporte
Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Salida (80 KiB / 19,5 GiB)
Tabla de contenido


Day 2b - Agent Tools Best Practices
Computadora portátil
Aporte
Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Salida (80 KiB / 19,5 GiB)
Tabla de contenido

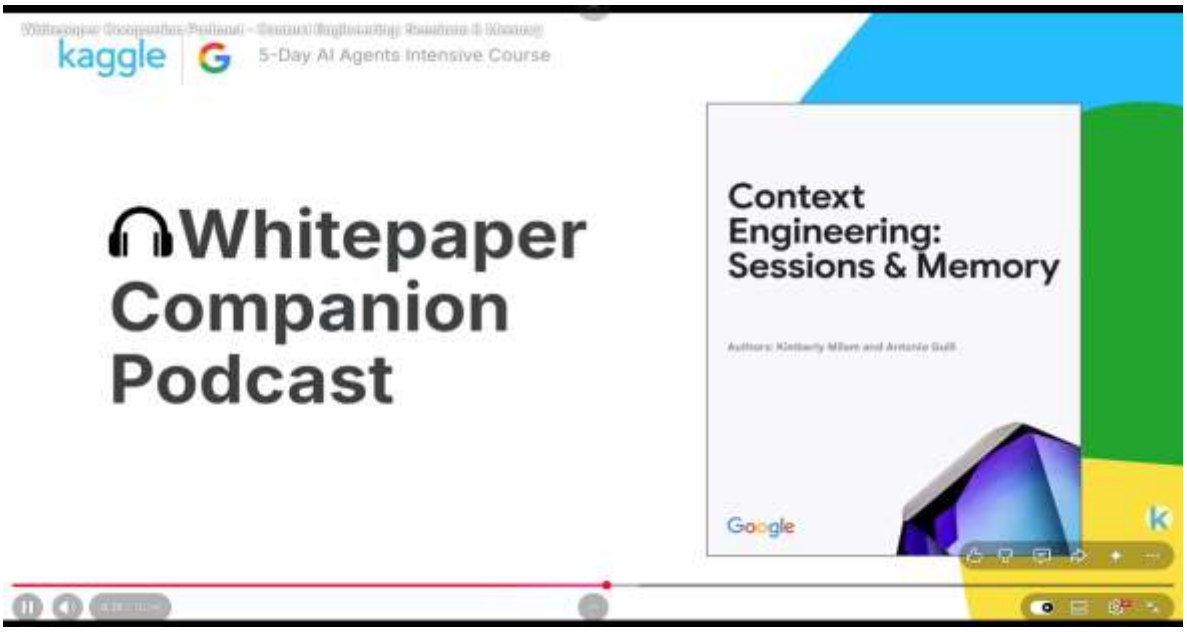

Day 2b - Agent Tools Best Practices
Computadora portátil
Aporte
Agregar entrada
Subir
No se ha adjuntado ninguna entrada.
Salida (80 KiB / 19,5 GiB)
Tabla de contenido



DIA 3

En el Día 3, **“Introducción a los agentes”**, aprendí qué son los agentes de inteligencia artificial, sus capacidades y por qué es importante que operen de manera segura.

Además, escuché un audio resumen que me ayudó a reforzar los conceptos principales, explicados de forma más clara y sencilla para facilitar su comprensión.



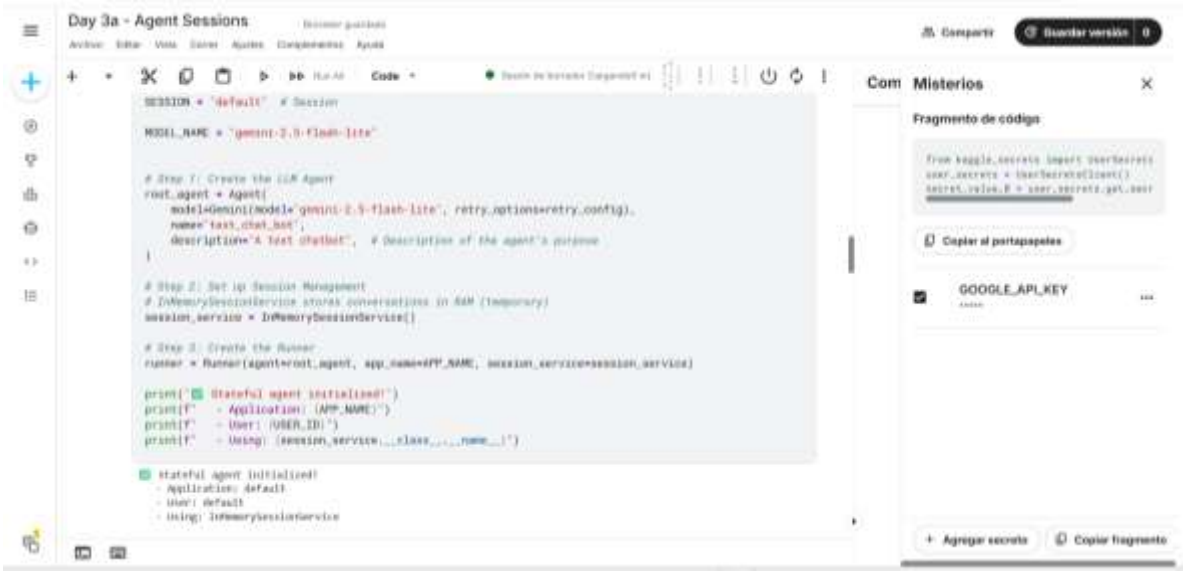
Leí el documento “Ingeniería de contexto: sesiones y memoria” para reforzar lo aprendido sobre cómo los agentes usan sesiones y memoria para mantener el contexto.

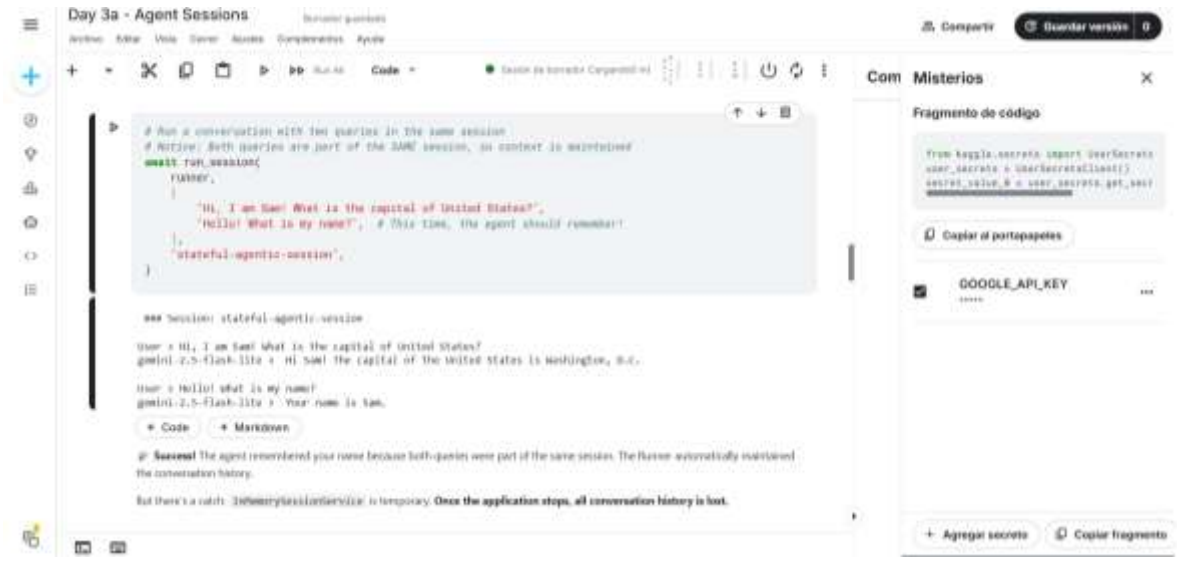
Section 1: Setup

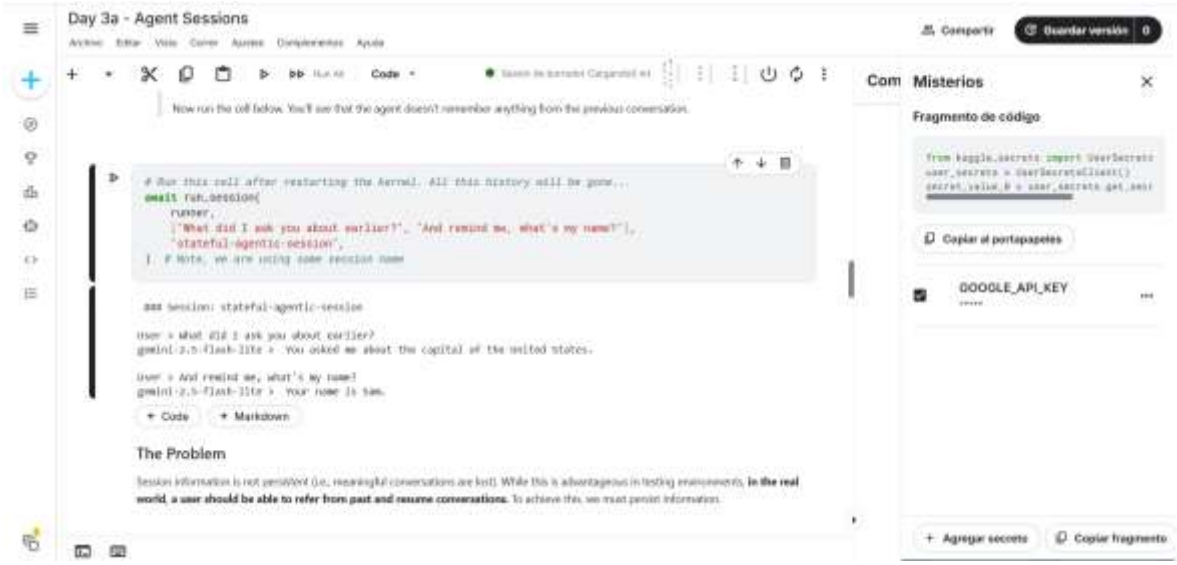
En esta parte no hice la configuración inicial porque ya estaba lista; solo ejecuté nuevas secciones con funciones de sesiones y reintentos para mejorar el sistema.

Section 2: Session Management

En esta sección comprendí el funcionamiento de las sesiones en agentes basados en LLM y cómo permiten conservar el contexto de una conversación mediante eventos y estados. Esto lo comprobé al trabajar con un agente capaz de recordar información dentro de una misma sesión.

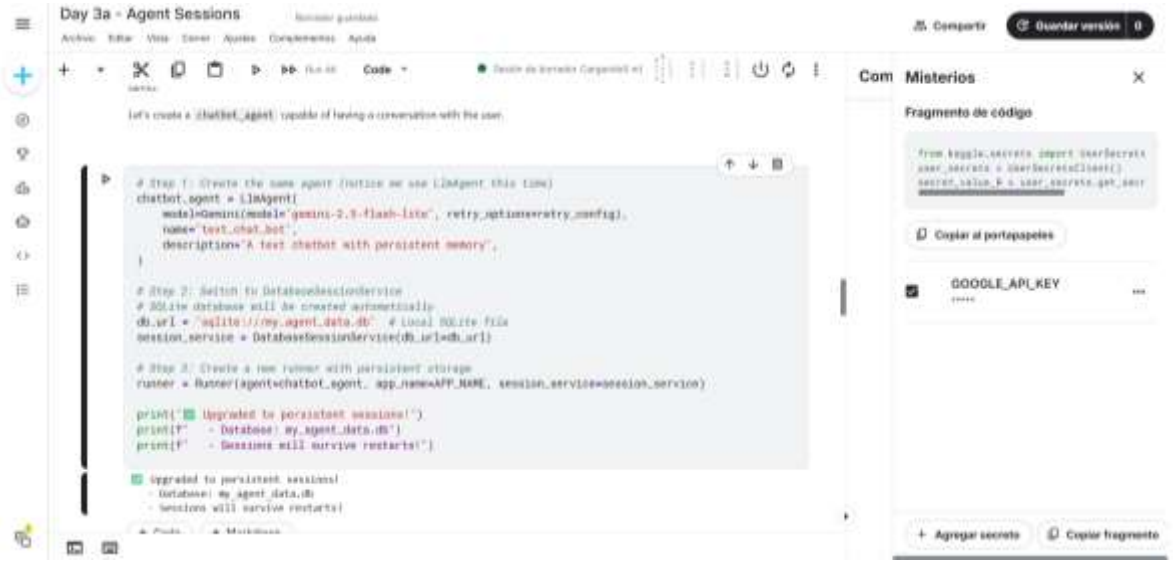


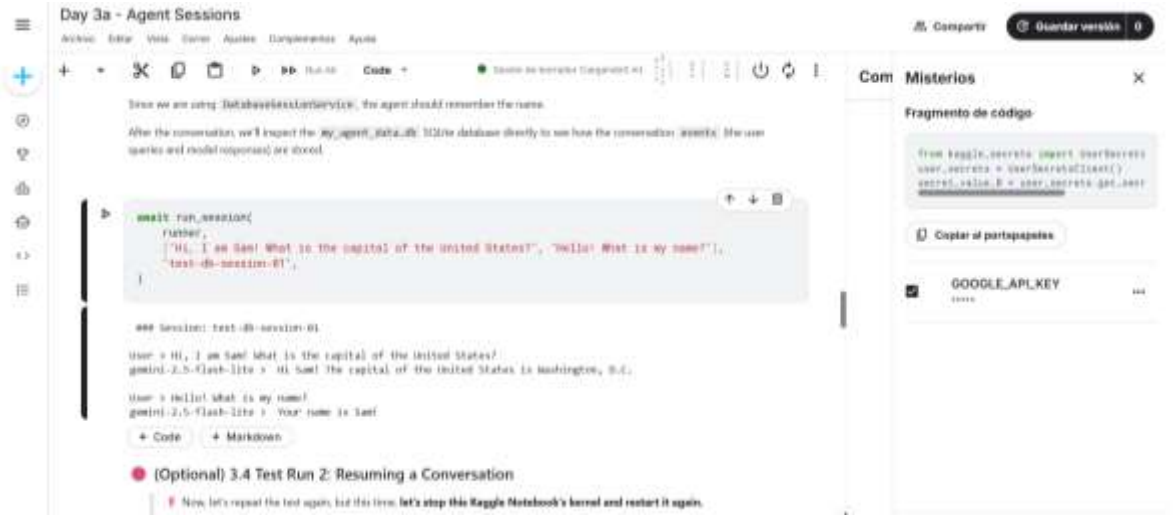




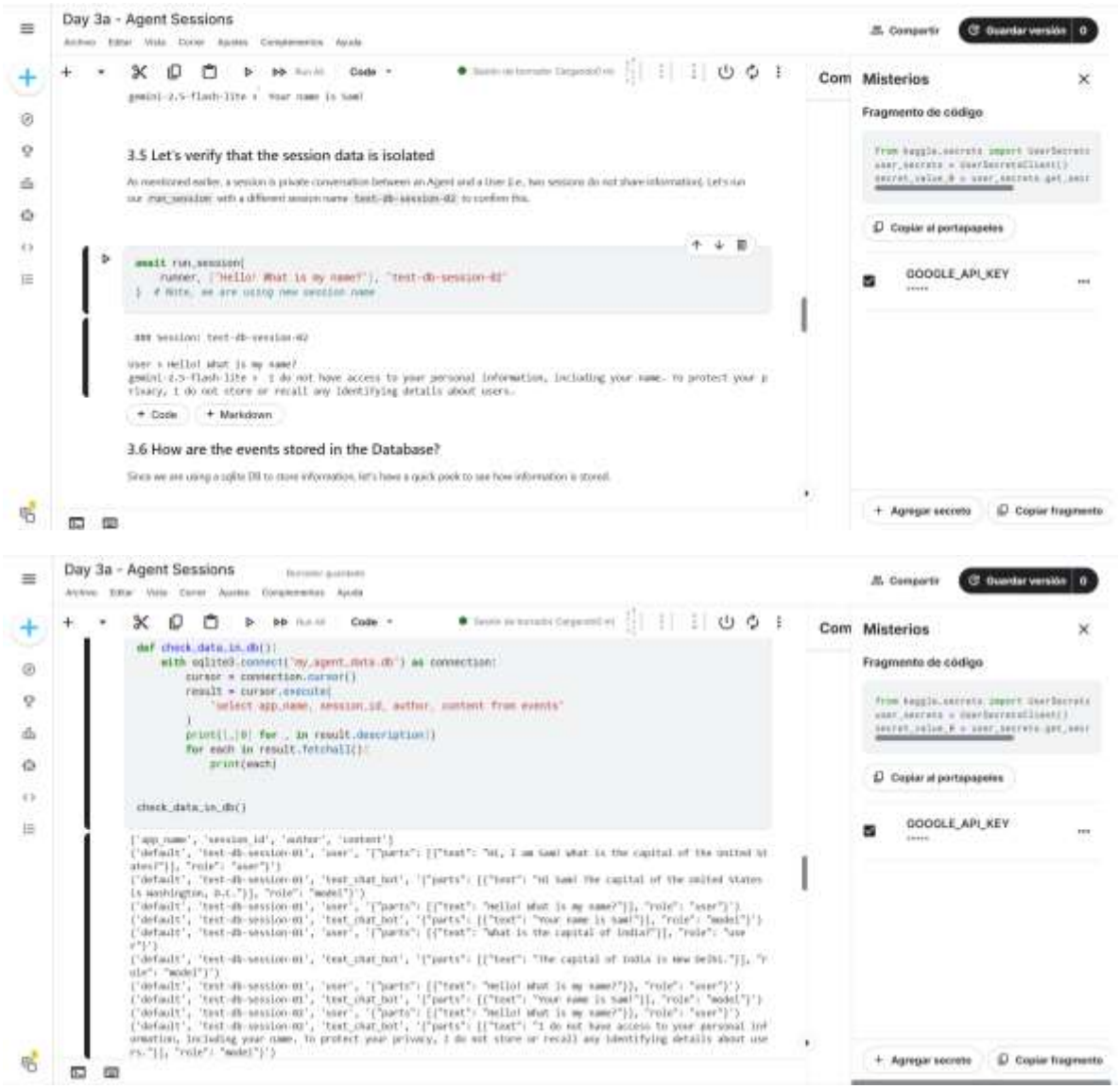
Section 3: Persistent Sessions with DatabaseSessionService

En esta sección comprendí cómo utilizar sesiones persistentes con **DatabaseSessionService** para que el agente conserve la información incluso después de reiniciarse. Asimismo, verifiqué cómo se almacenan los datos en una base de datos SQLite.


Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

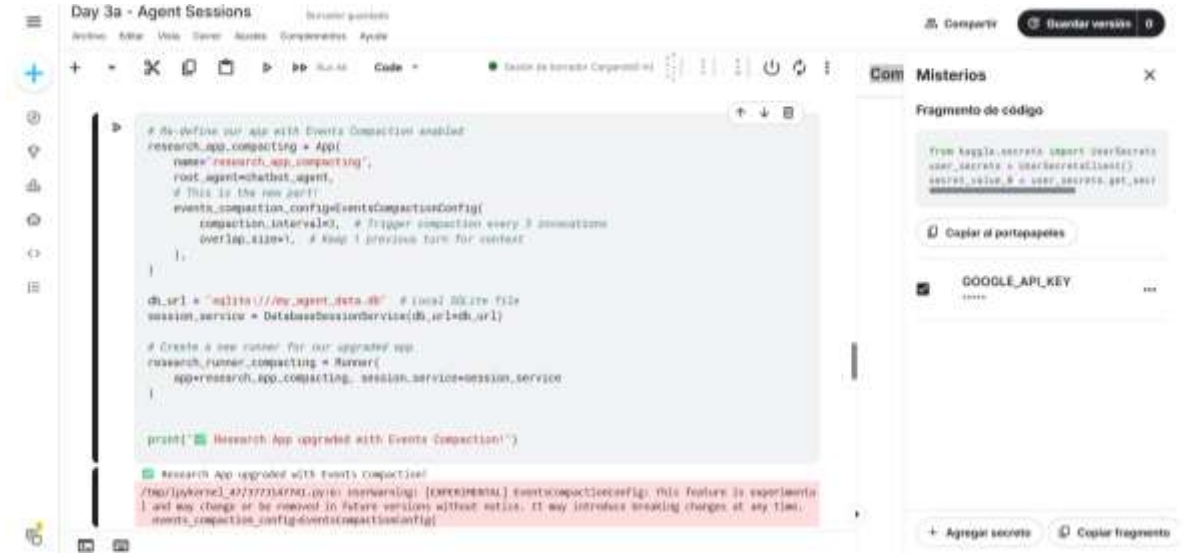

Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
(Optional) 3.4 Test Run 2: Resuming a Conversation
Agregar secreto
Copiar fragmento


Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
3.5 Let's verify that the session data is isolated
Agregar secreto
Copiar fragmento



Section 4: Context Compaction

Comprendí cómo el sistema puede resumir automáticamente el historial de conversaciones mediante **Context Compaction** en ADK, reduciendo la cantidad de información almacenada para optimizar el rendimiento y disminuir costos. Además, observé cómo los eventos antiguos son reemplazados por resúmenes y cómo este proceso puede verificarse dentro de la base de datos.

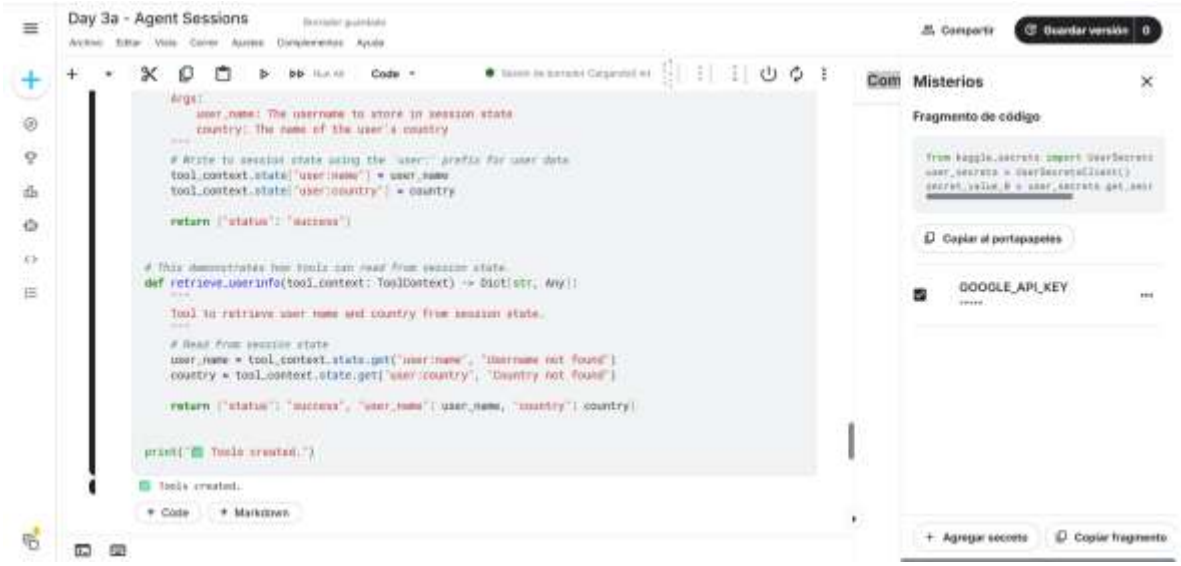

Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

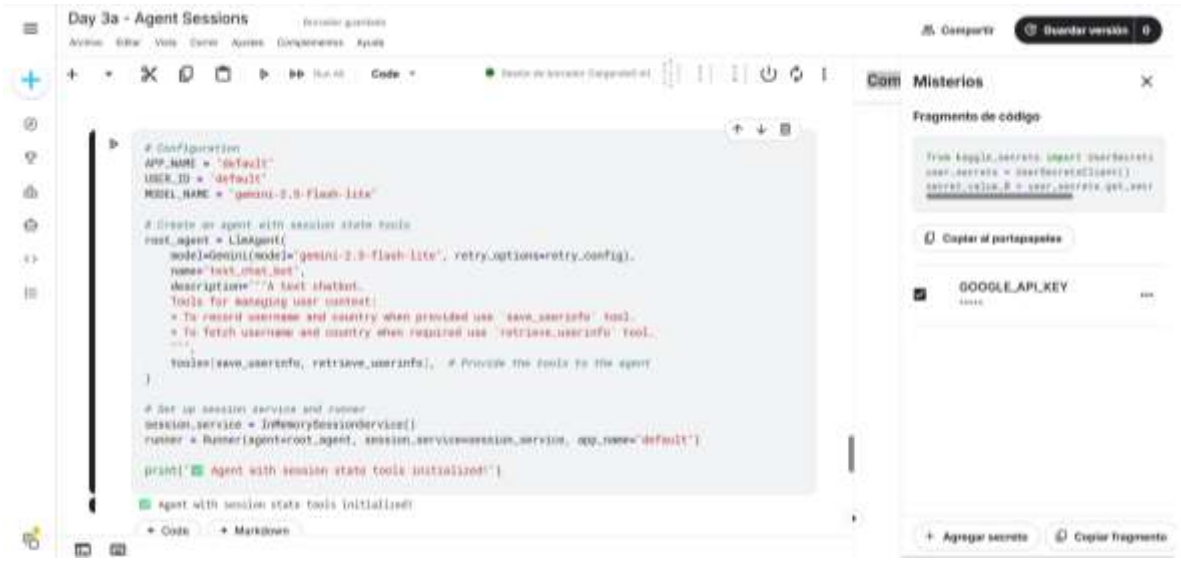

Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

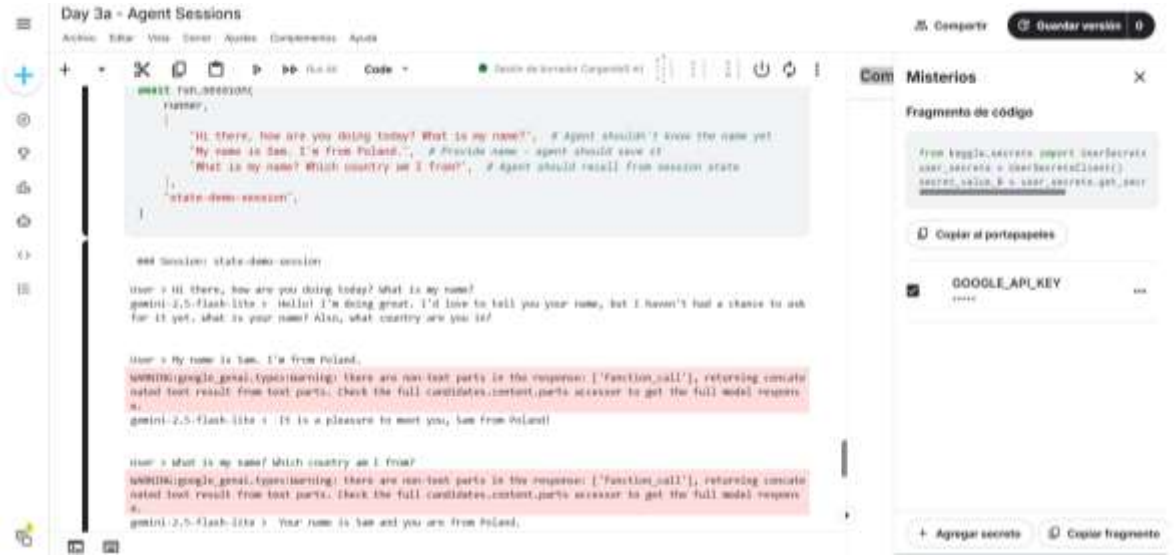

Day 3a - Agent Sessions
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

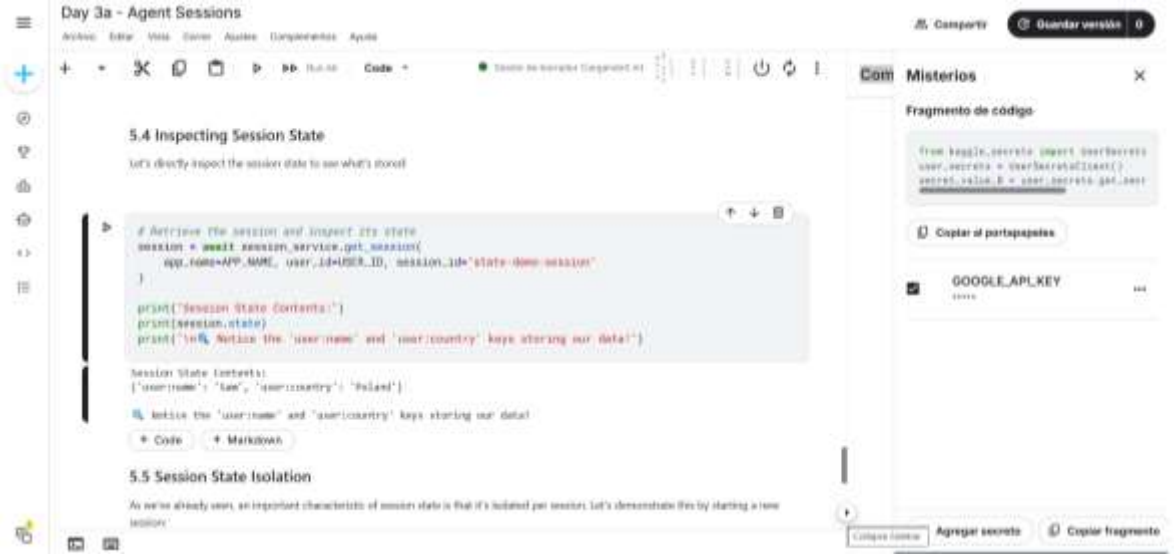
Section 5: Working with Session State

Ejecuté las herramientas de manejo de estado de sesión para observar cómo el agente guarda y recupera información como el nombre y el país del usuario, sin realizar ninguna modificación al código.

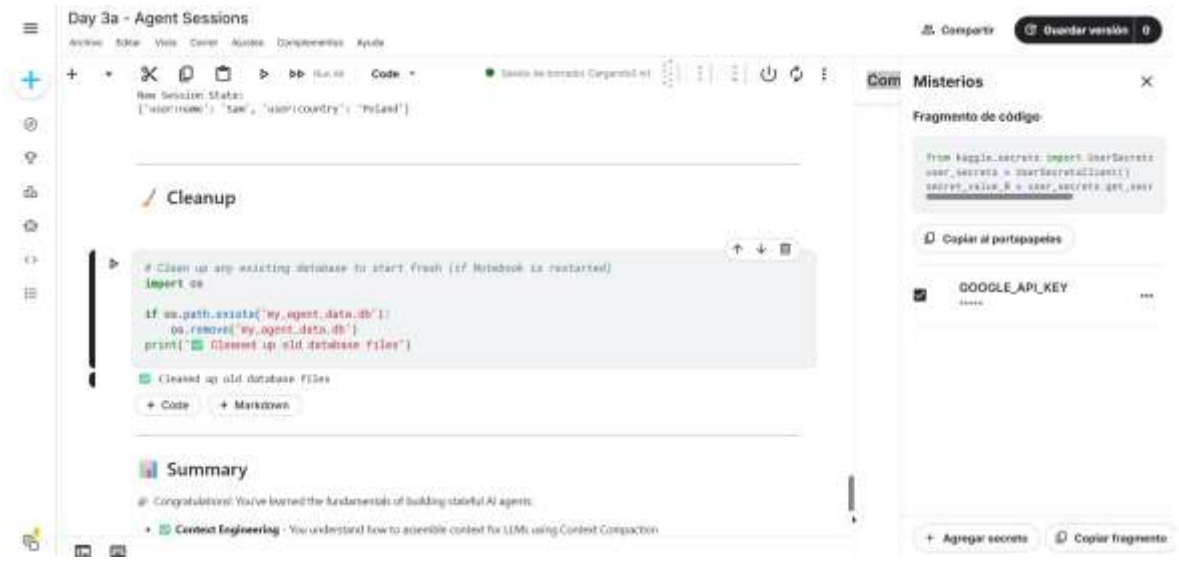





Day 3a - Agent Sessions
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
await run_session(
runner,
'Hi there, how are you doing today? What is my name?', # Agent shouldn't know the name yet
'My name is Sam. I'm from Poland.', # Provide name - agent should save it
'What is my name? Which country am I from?', # Agent should recall from session state
],
'state-demo-session',
)
Session: state-demo-session
User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing great. I'd love to tell you your name, but I haven't had a chance to ask for it yet. What is your name? Also, what country are you in?
User > My name is Sam. I'm from Poland.
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
gemini-2.5-flash-lite > It is a pleasure to meet you, Sam from Poland!
User > What is my name? Which country am I from?
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
gemini-2.5-flash-lite > Your name is Sam and you are from Poland.
Comentarios
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento


Day 3a - Agent Sessions
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
5.4 Inspecting Session State
Let's directly inspect the session state to see what's stored!
# Retrieve the session and inspect its state
session = await session_service.get_session(
app_name=APP_NAME, user_id=USER_ID, session_id='state-demo-session'
)
print('Session State Contents:')
print(session.state)
print('\n🔍 Notice the 'user:name' and 'user:country' keys storing our data!')
Session State Contents:
{'user:name': 'Sam', 'user:country': 'Poland'}
🔍 Notice the 'user:name' and 'user:country' keys storing our data!
+ Code
+ Markdown
5.5 Session State Isolation
As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:
Comentarios
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento


Day 3a - Agent Sessions
Archivo Editar Vista Correr Ajustes Complementos Ayuda
Compartir
Guardar versión
Code
5.5 Session State Isolation
As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:
Start a completely new session - the agent won't know our name
await run_session(
runner,
['Hi there, how are you doing today? What is my name?'],
'new-isolated-session',
)
Expected: The agent won't know the name because this is a different session
Session: new-isolated-session
User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing great, thank you for asking. I can't seem to recall your name. Could you please tell me what it is?
+ Code
+ Markdown
5.6 Cross-Session State Sharing
While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session ('new-isolated-
Comentarios
Misterios
Fragmento de código
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento



Section 1: Setup

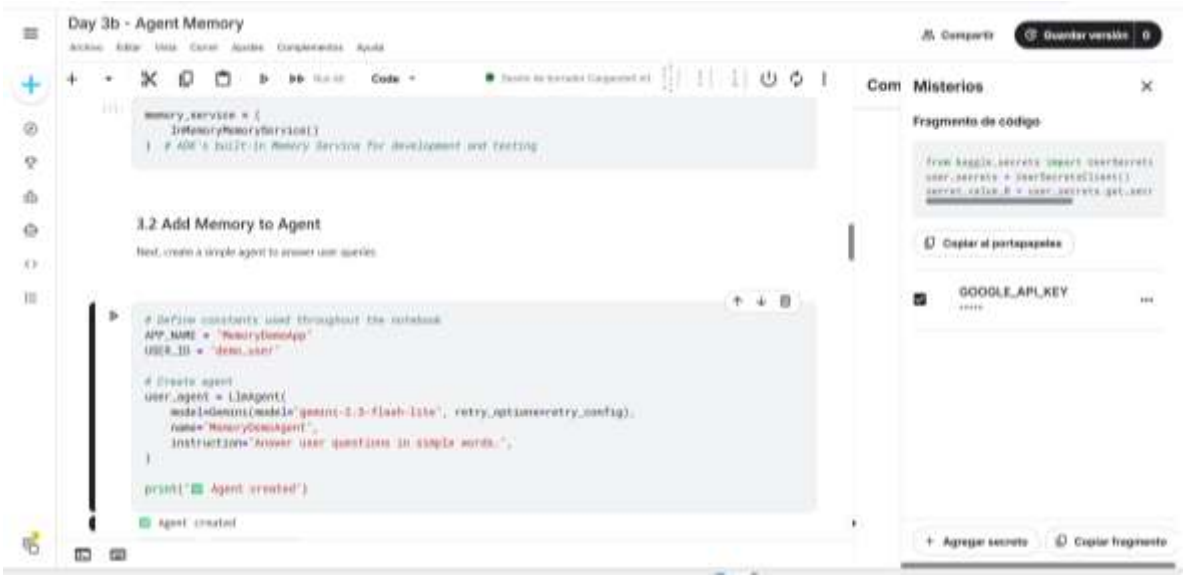
En esta sección ya no realicé la configuración desde cero, ya que en los días anteriores había preparado el entorno y la API key. Por eso, únicamente ejecuté las celdas de instalación, configuración, importación de componentes y funciones auxiliares para verificar que todo funcionara correctamente en el notebook.

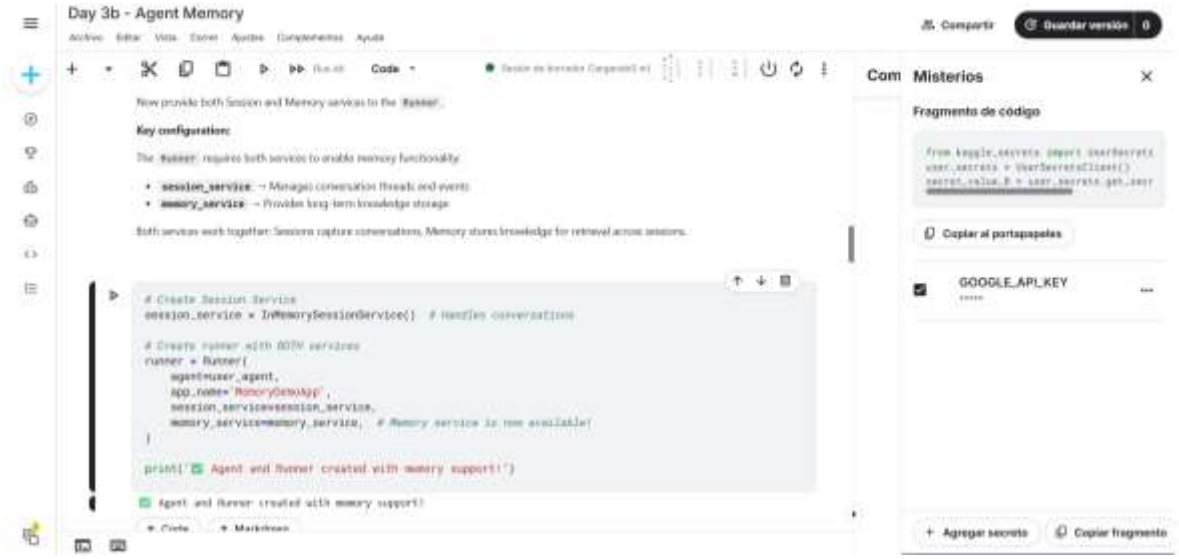
Section 2: Memory Workflow

Aprendí que primero se inicializa el servicio de memoria, luego se almacenan los datos de la sesión y finalmente se recupera la información cuando es necesaria.

Section 3: Initialize MemoryService

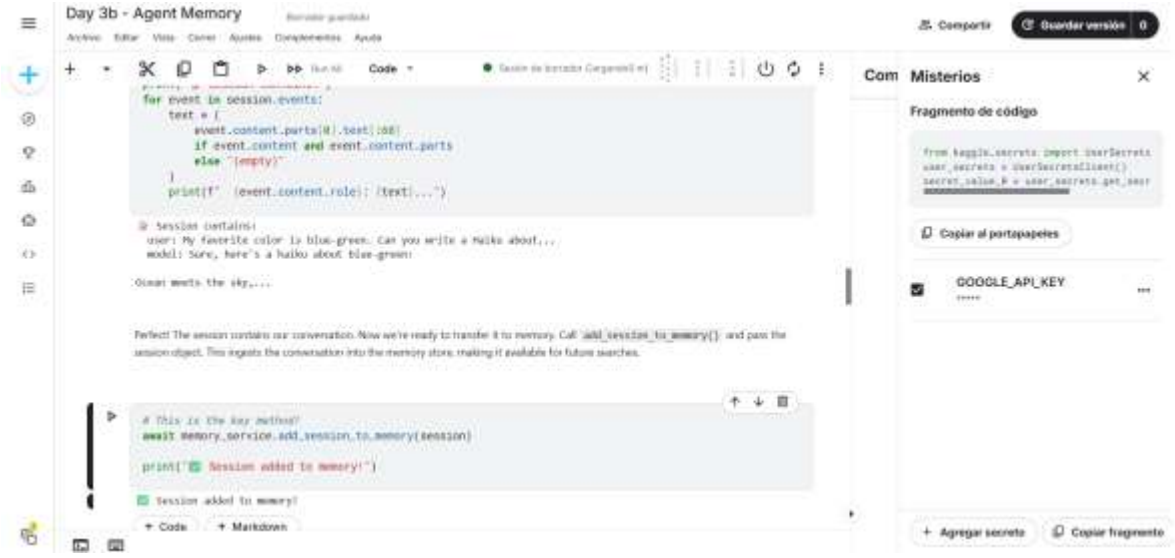
Comprendí cómo se inicializa el servicio de memoria y cómo se integra al agente mediante el Runner. También entendí que la memoria no se usa automáticamente, sino que debe configurarse para almacenar y recuperar información.





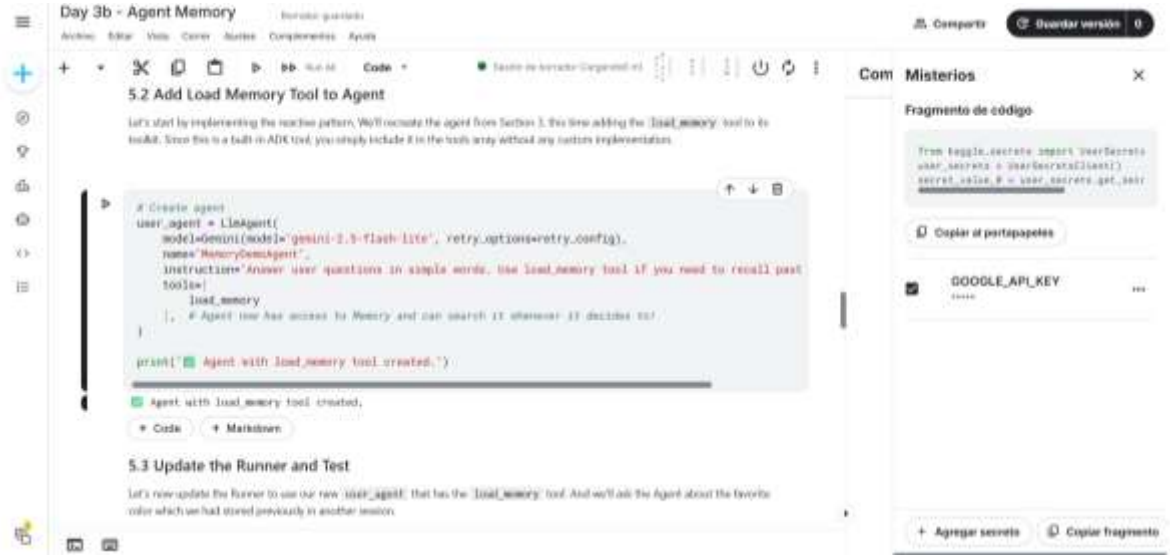
Section 4: Ingest Session Data into Memory

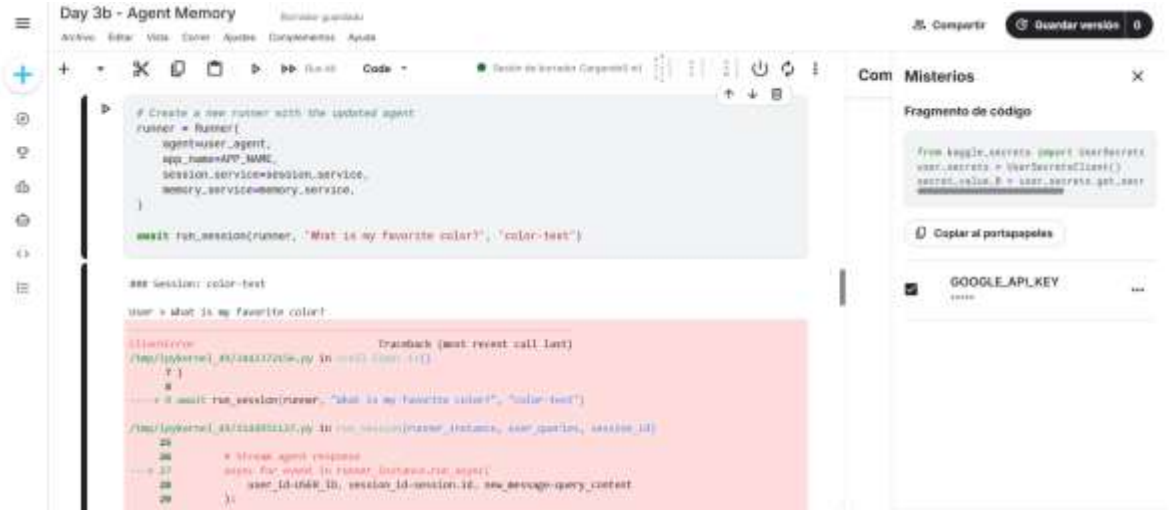
Ejecuté un ejemplo en el que el agente sostiene una conversación y, posteriormente, esa información se transfiere a la memoria, lo que me permitió observar cómo los datos de la sesión pueden almacenarse para su uso posterior.

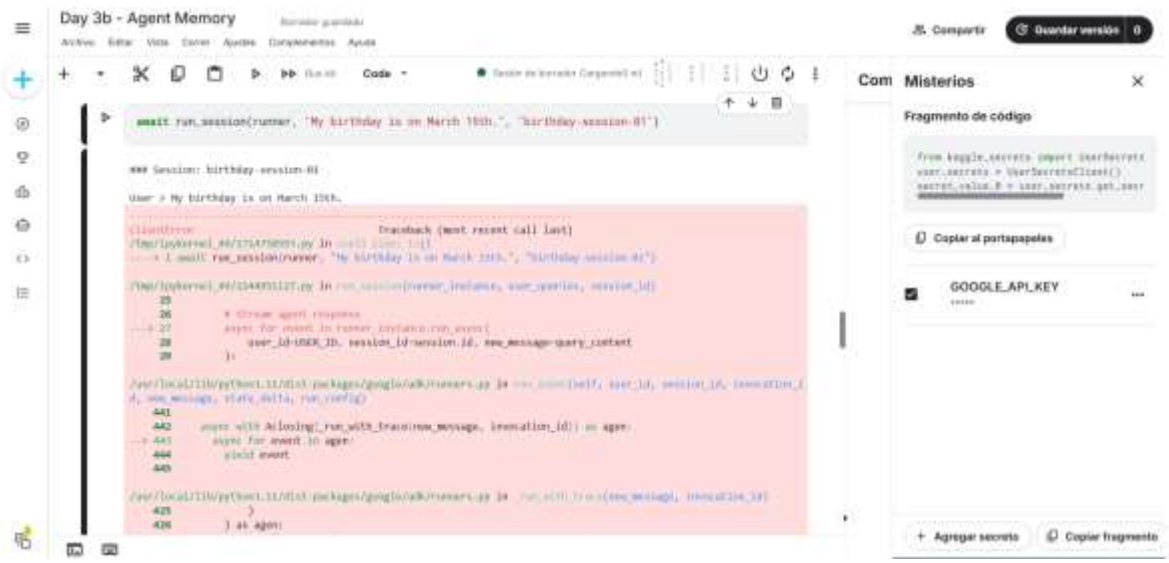


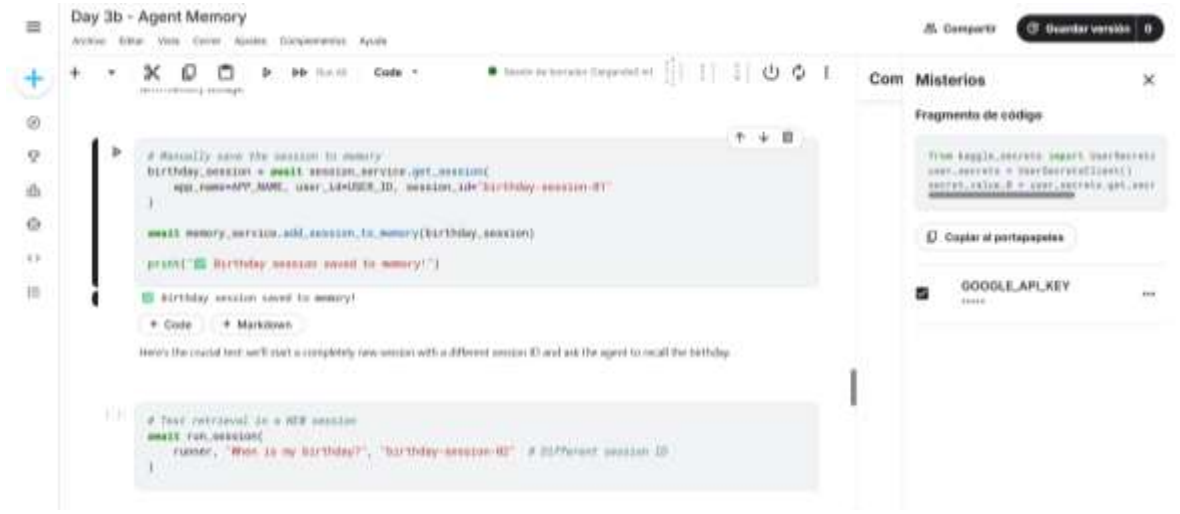
Section 5: Enable Memory Retrieval in Your Agent

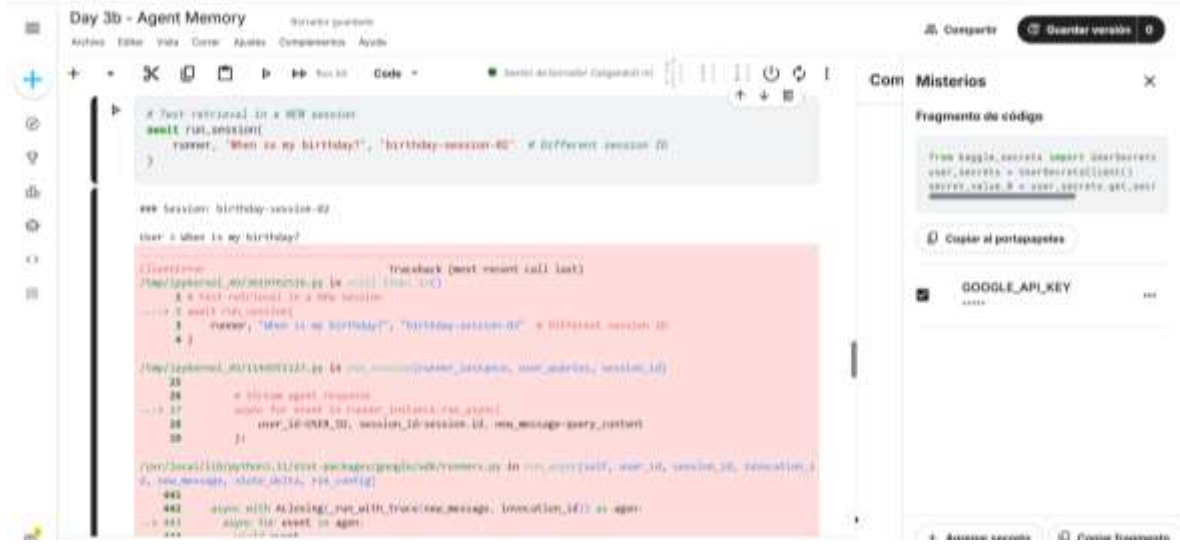
Analicé cómo habilitar la recuperación de memoria en el agente mediante *load_memory*, lo que permite consultar información previamente almacenada. Asimismo, verifiqué cómo se recuperan estos datos en nuevas sesiones y comprendí el proceso tanto de almacenamiento como de consulta.

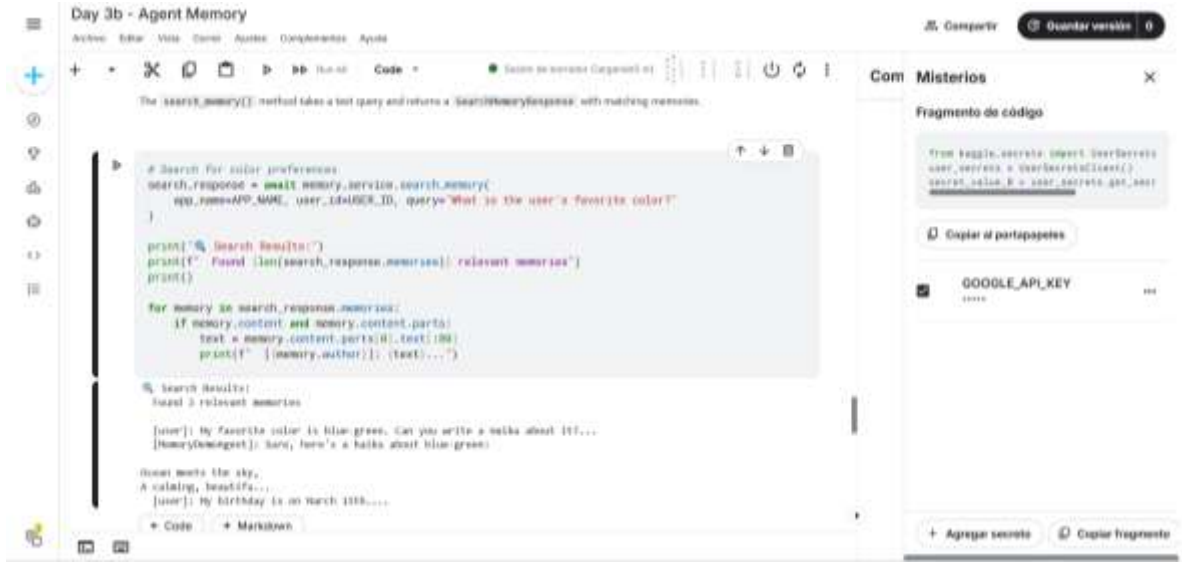

Day 3b - Agent Memory
Compartir
Guardar versión
Code
5.2 Add Load Memory Tool to Agent
Let's start by implementing the reactive pattern. We'll recreate the agent from Section 1, this time adding the load_memory tool to its toolkit. Since this is a built-in ADK tool, you simply include it in the tools array without any custom implementation.
# Create agent
user_agent = LlmAgent(
model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
name="MemoryDemoAgent",
instruction="Answer user questions in simple words. Use load_memory tool if you need to recall past
tools=[
load_memory
], # Agent now has access to Memory and can search it whenever it decides to!
)
print("✅ Agent with load_memory tool created.")
✅ Agent with load_memory tool created.
+ Code
+ Markdown
5.3 Update the Runner and Test
Let's now update the Runner to use our new user_agent that has the load_memory tool. And we'll ask the Agent about the favorite color which we had stored previously in another session.
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento


Day 3b - Agent Memory
Compartir
Guardar versión
Code
# Create a new runner with the updated agent
runner = Runner(
agent=user_agent,
app_name=APP_NAME,
session_service=session_service,
memory_service=memory_service,
)
await run_session(runner, "What is my favorite color?", "color-test")
Session: color-test
User > What is my favorite color?
Traceback (most recent call last)
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY


Day 3b - Agent Memory
Compartir
Guardar versión
Code
await run_session(runner, "My birthday is on March 15th.", "birthday-session-01")
Session: birthday-session-01
User > My birthday is on March 15th.
Traceback (most recent call last)
Com
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
+ Agregar secreto
Copiar fragmento

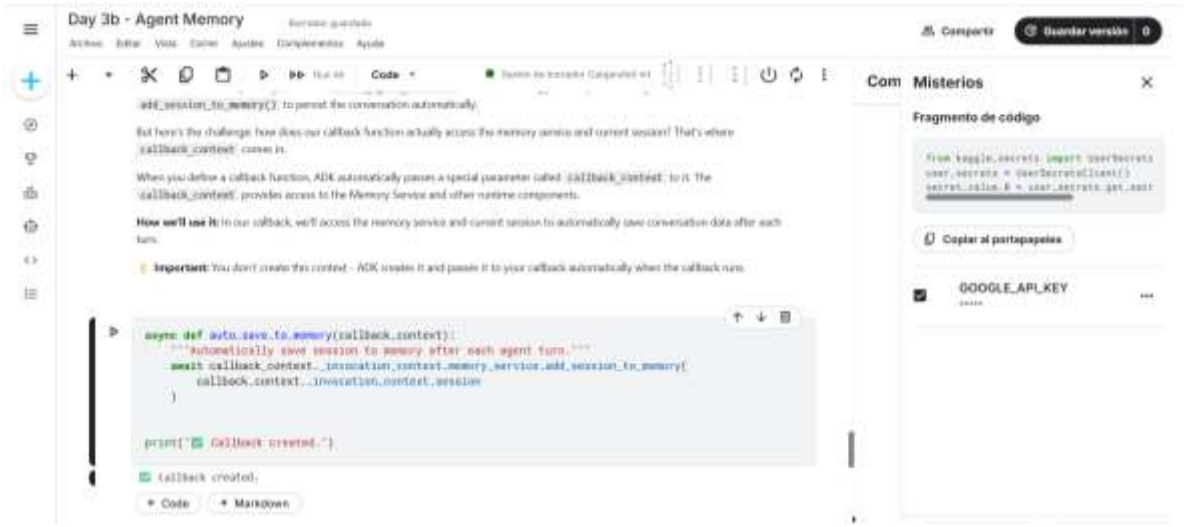

Day 3b - Agent Memory
Misterios
Fragmento de código
GOOGLE_API_KEY
Birthday session saved to memory!
+ Code
+ Markdown

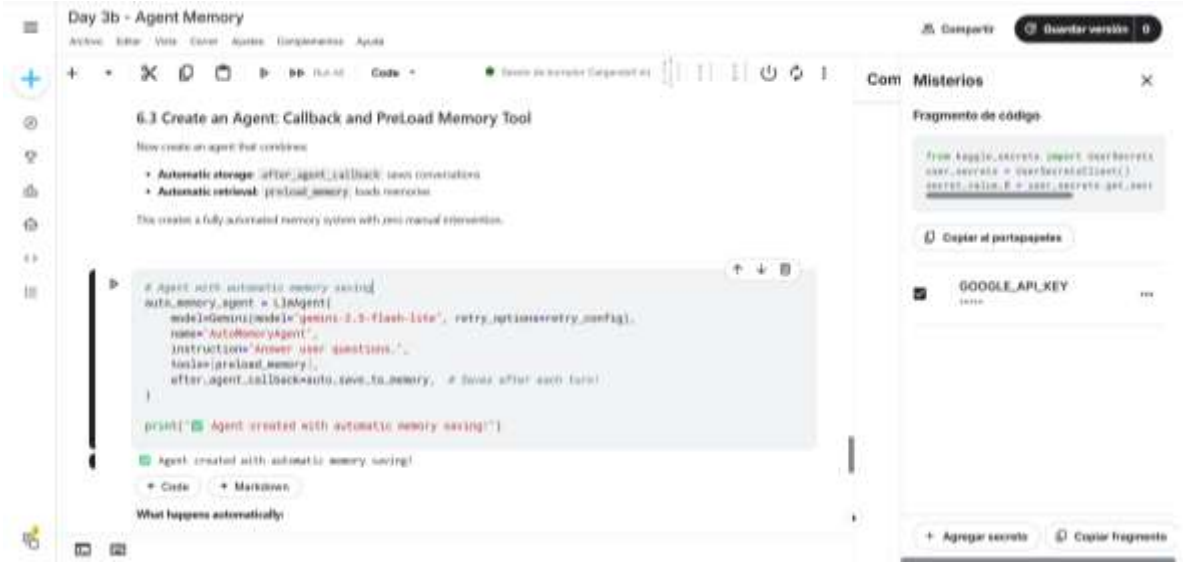

Day 3b - Agent Memory
Misterios
Fragmento de código
GOOGLE_API_KEY

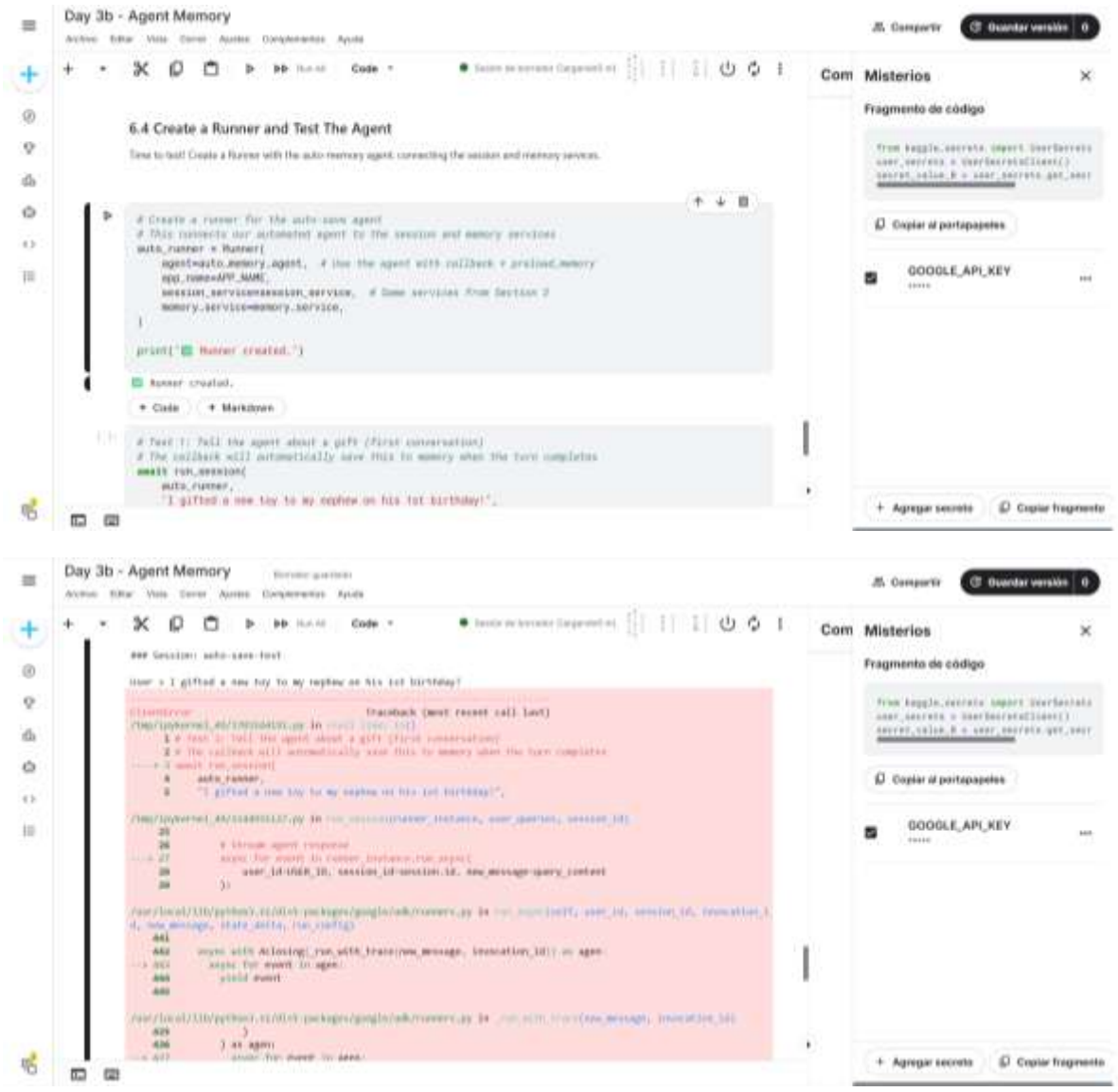

Day 3b - Agent Memory
Misterios
Fragmento de código
GOOGLE_API_KEY
+ Code
+ Markdown

Section 6: Automating Memory Storage

Ejecuté el proceso de automatización del almacenamiento en memoria con callbacks, observando cómo el sistema guarda y recupera la información automáticamente sin intervención manual.



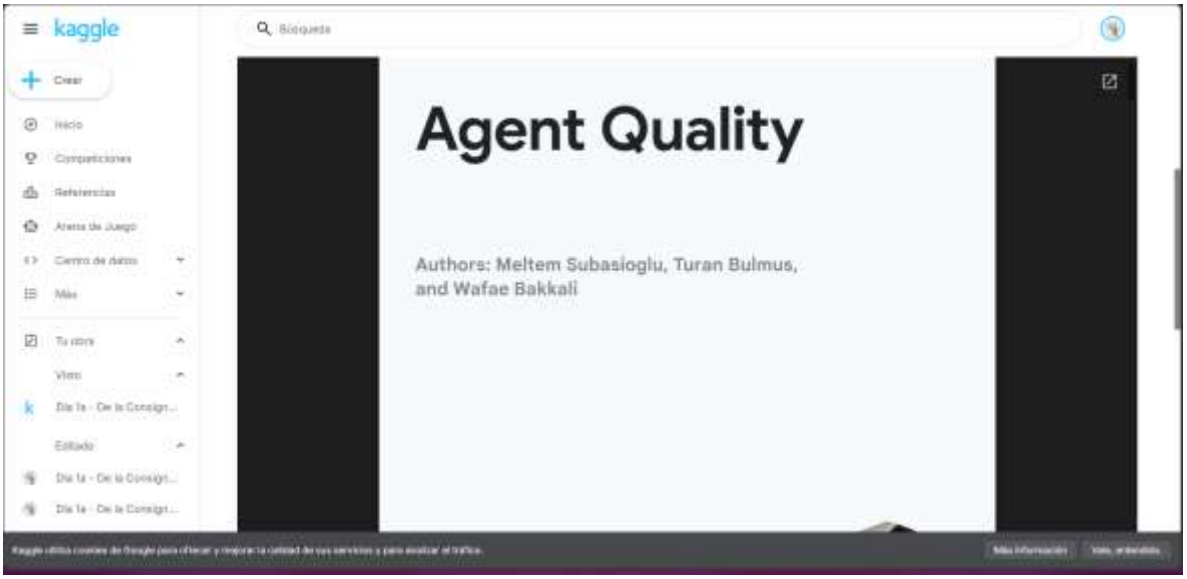




DIA 4

En el Día 4, **“Calidad del agente”**, Aprendí a evaluar el rendimiento de los agentes mediante el uso de registros, trazas y métricas, y realicé prácticas para analizar su desempeño y el uso de sus herramientas. Además, escuché un podcast que resume los conceptos principales, lo que me ayudó a comprender con mayor claridad cómo medir y mejorar la calidad de un agente.

Leí el informe técnico sobre la calidad del agente para complementar el podcast.

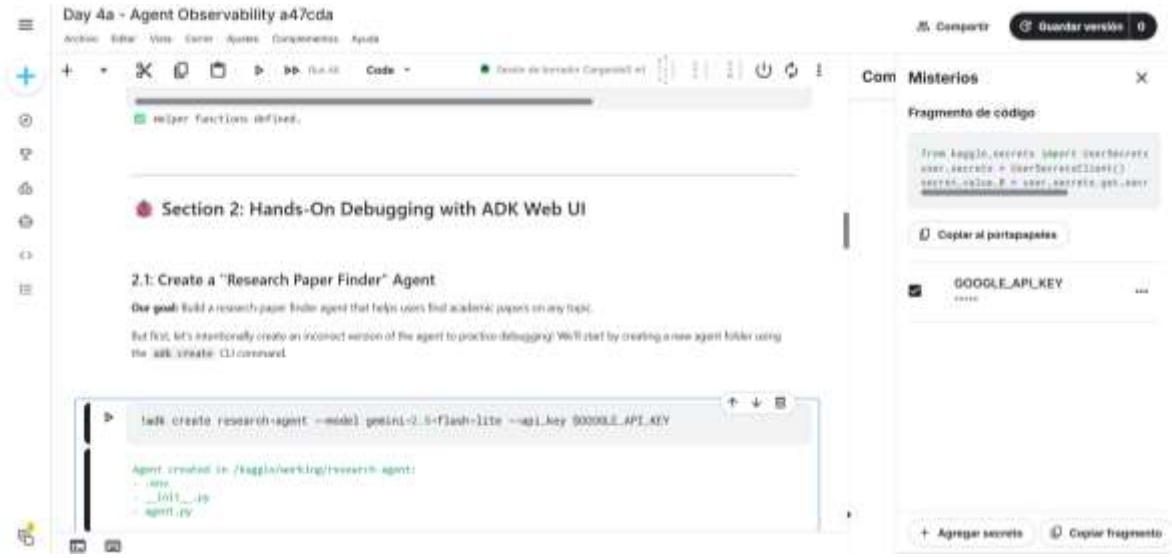


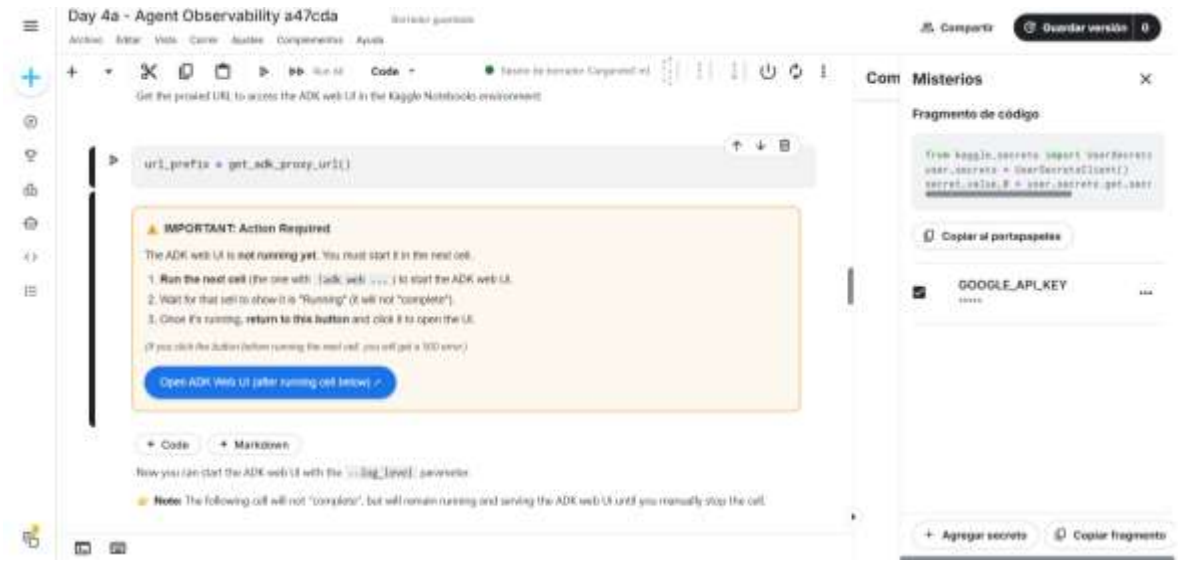
Section 1: Setup

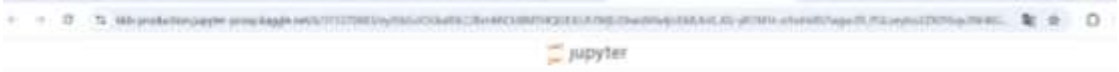
En esta sección no fue necesario repetir la instalación de dependencias ni la configuración de la API key, ya que el entorno ya se encontraba previamente preparado. Por ello, me limité a ejecutar las partes nuevas del laboratorio, como el logging, la limpieza de archivos y los ajustes del entorno, enfocándome en dejar listo el sistema para la observabilidad del agente.

Section 2: Hands-On Debugging with ADK Web UI

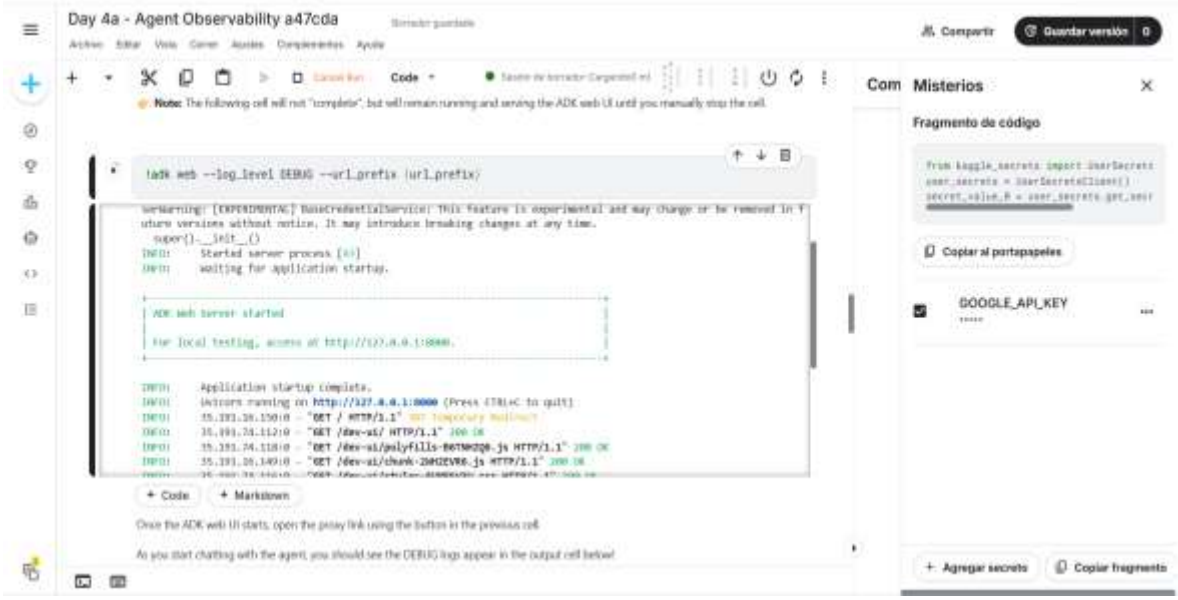
Ejecuté un agente en la interfaz web de ADK para comprobar su funcionamiento, donde analicé su comportamiento al buscar artículos y detectar errores en los resultados. Asimismo, revisé los **logs** y eventos para identificar la causa del problema sin necesidad de realizar modificaciones al código.

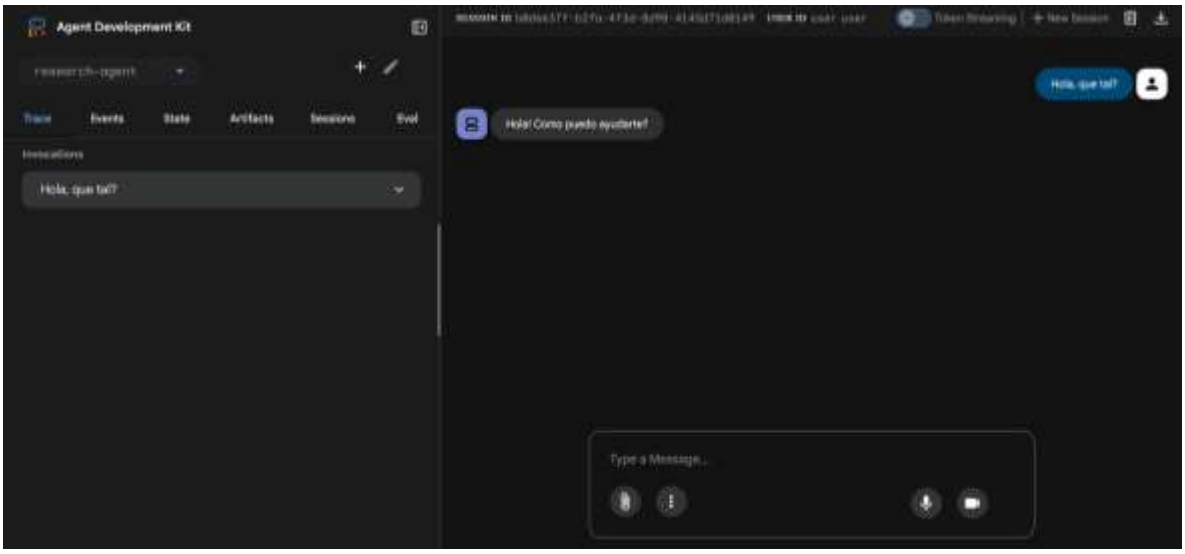


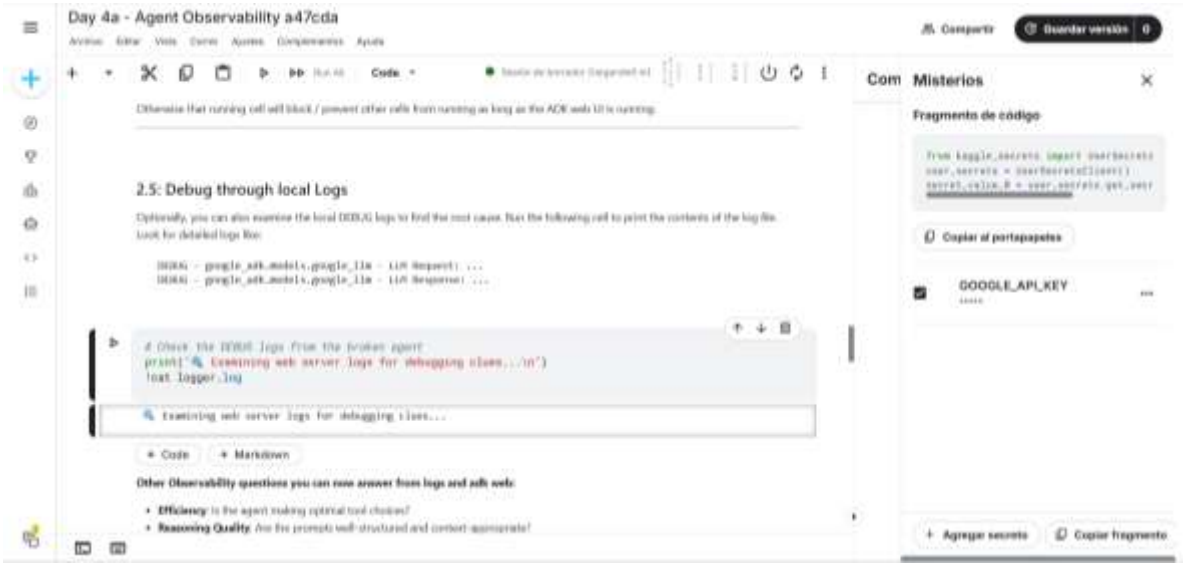




500: Error interno del servidor

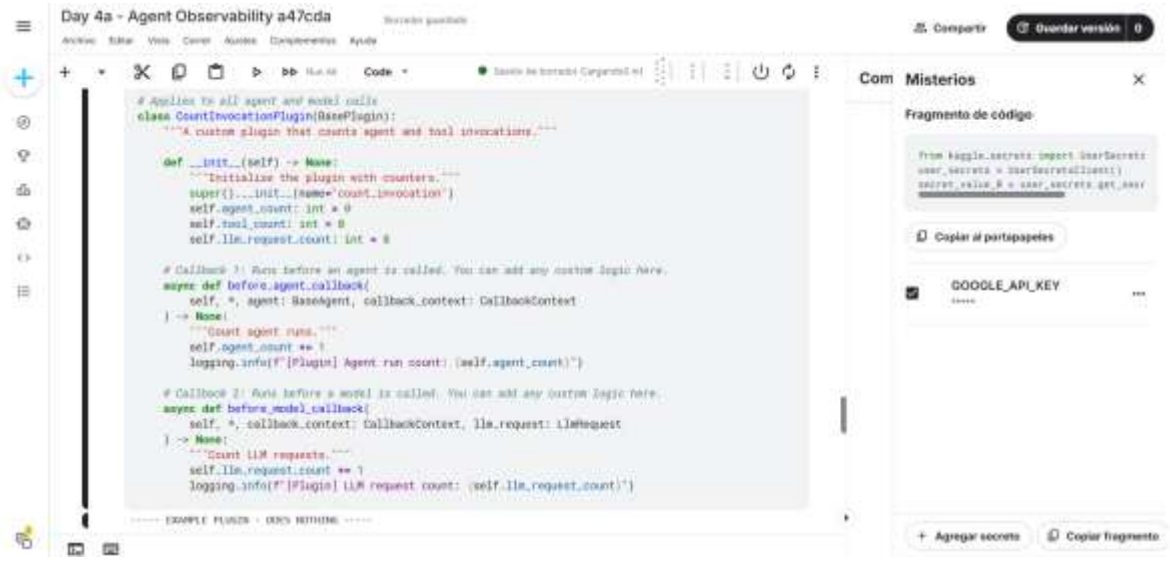




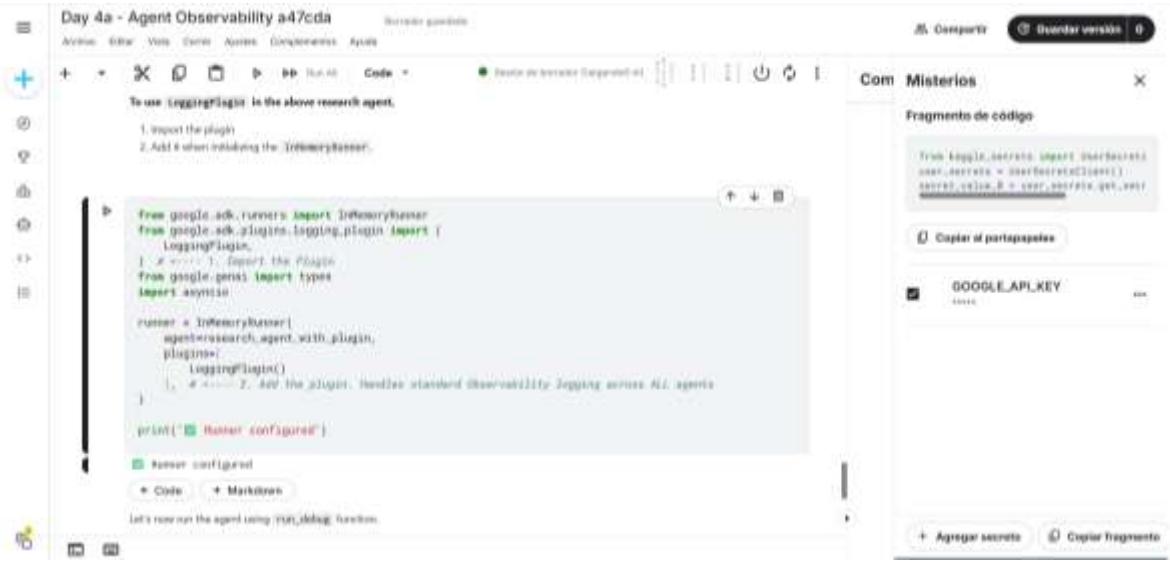


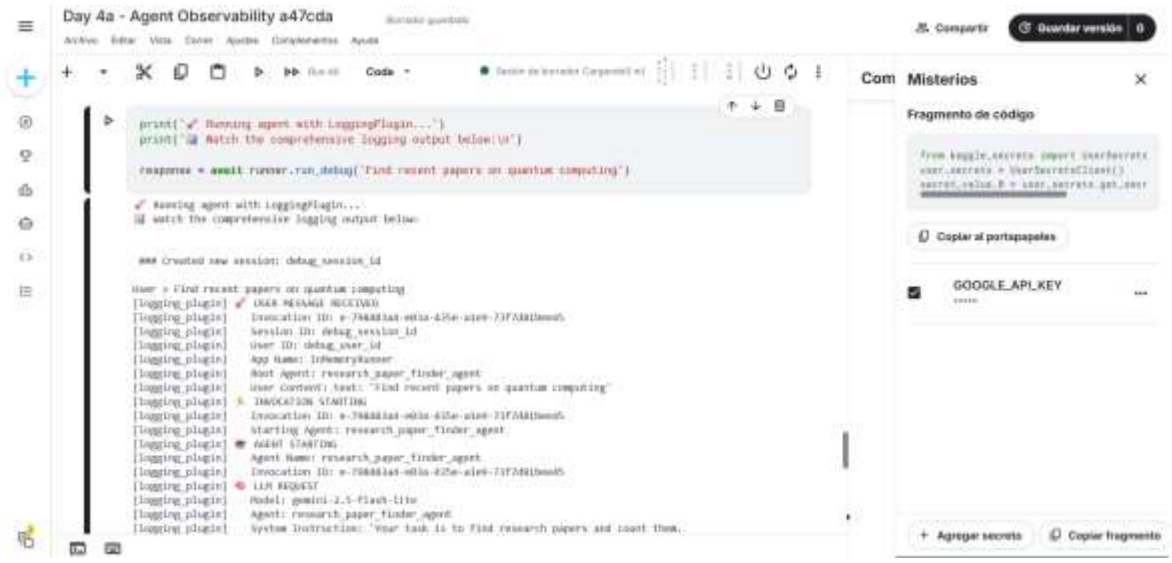
Section 3: Logging in production

Comprendí el uso de **LoggingPlugin** para generar registros automáticos en entornos de producción, analizando el comportamiento del agente, sus respuestas y el uso de herramientas, lo que me permitió monitorear y entender su funcionamiento en escenarios reales.


Day 4a - Agent Observability a47cda
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento


Day 4a - Agent Observability a47cda
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento


Day 4a - Agent Observability a47cda
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

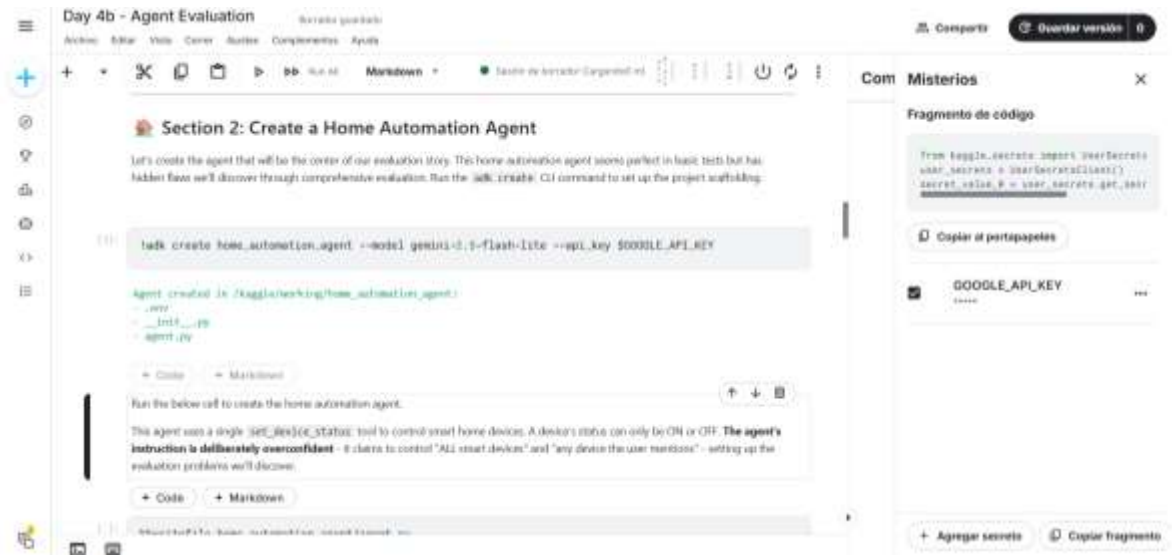


Section 1: Setup

Como ya tenemos hacho lo que ocuparemos omitiré esta parte.

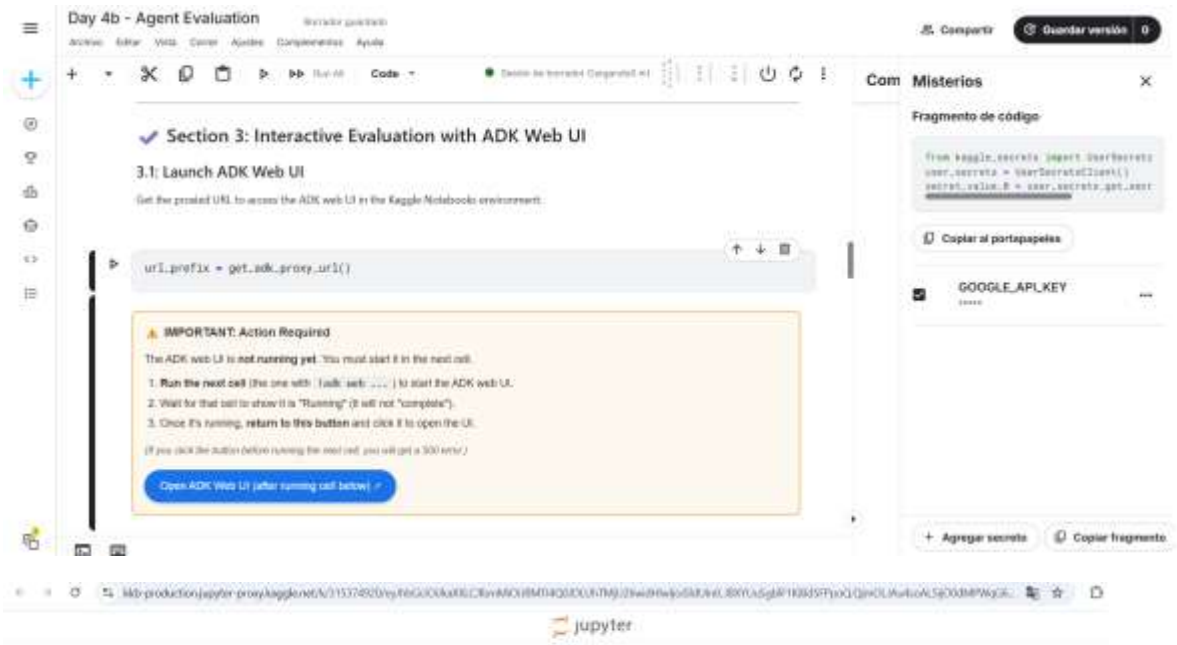
Section 2: Create a Home Automation Agent

Comprendí la creación de un agente de automatización del hogar, observando cómo se genera mediante un comando y cómo emplea una herramienta para controlar dispositivos, como encenderlos y apagarlos. Además, identifiqué que incluye fallas intencionales que serán evaluadas posteriormente.

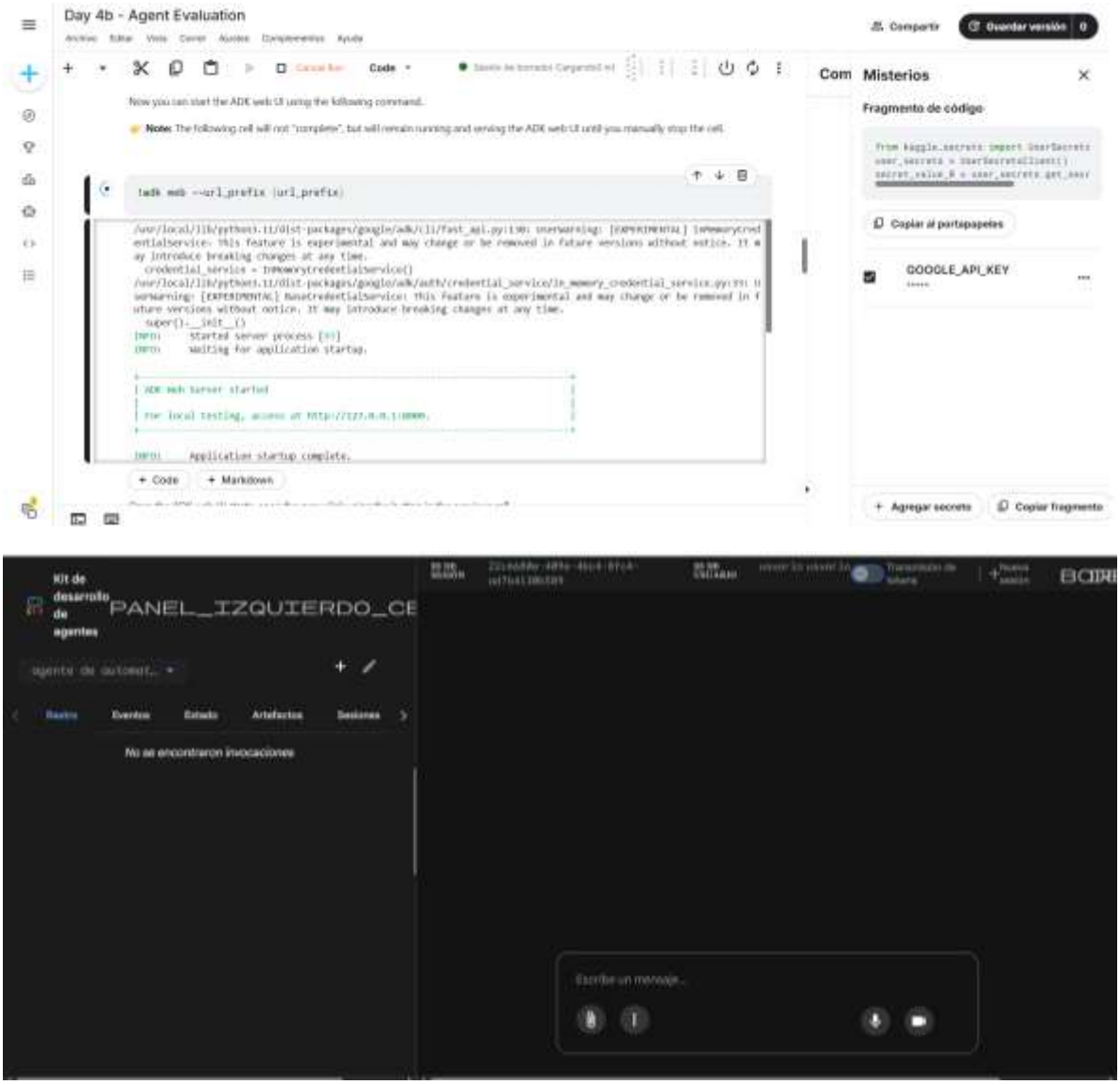


Section 3: Interactive Evaluation with ADK Web UI

Generé conversaciones, guardé algunas como casos de prueba y ejecuté evaluaciones para contrastar sus respuestas con los resultados esperados.

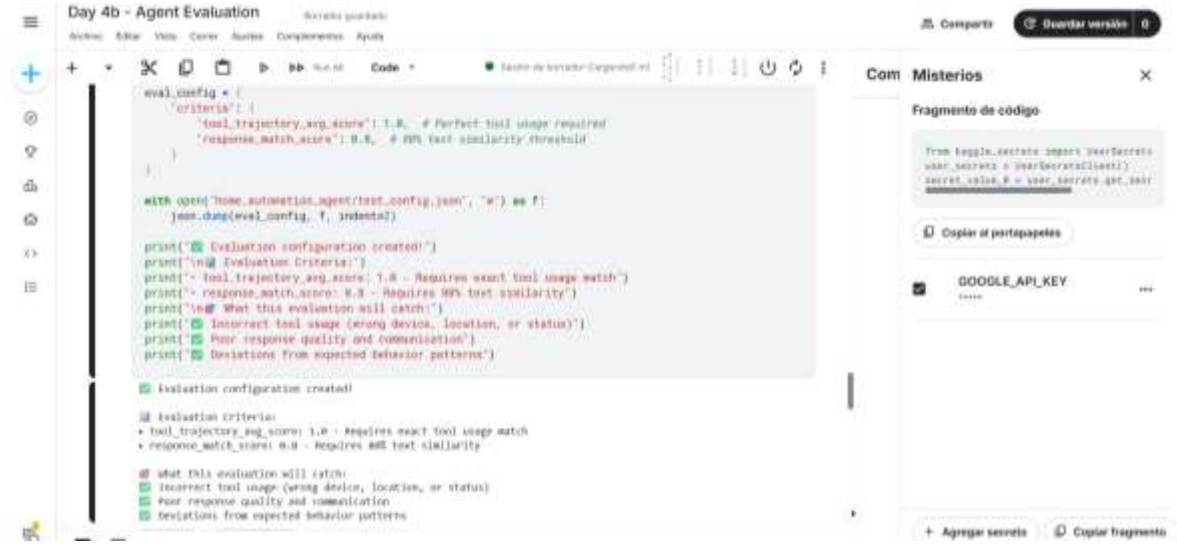


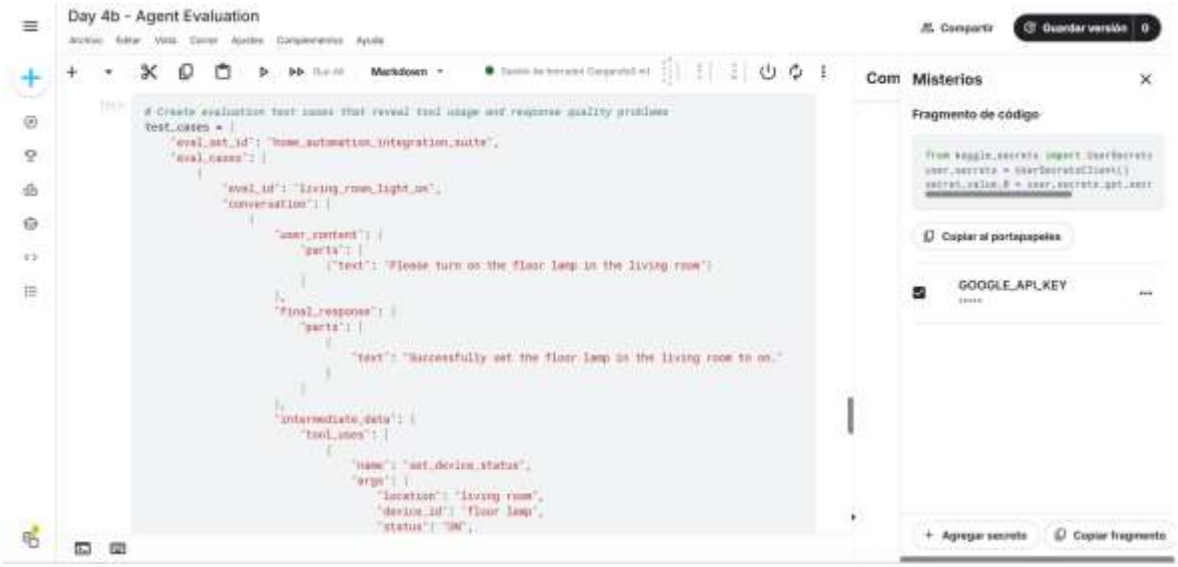
500: Error interno del servidor



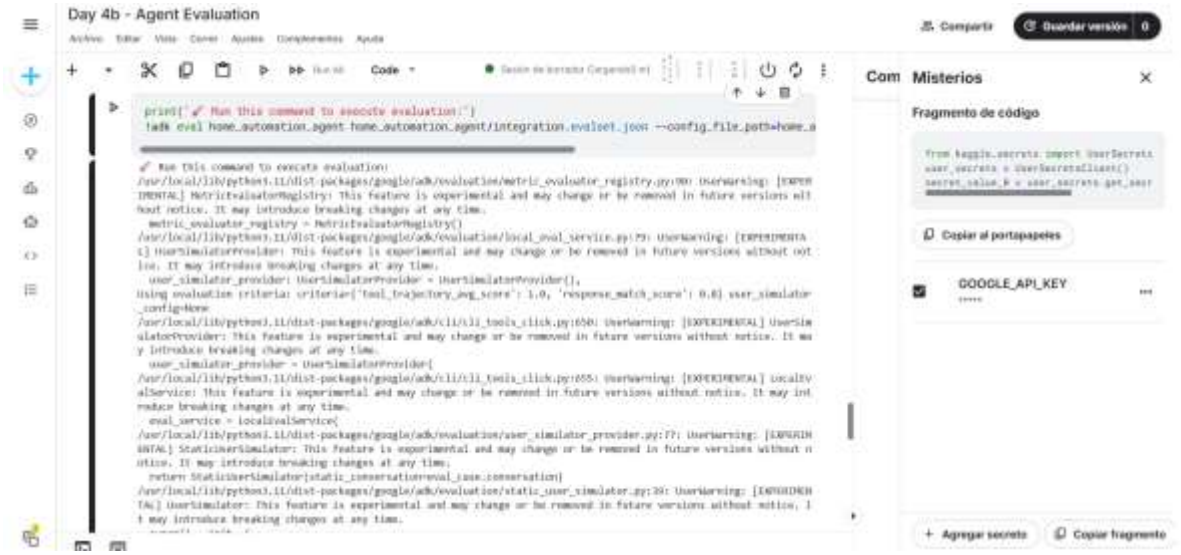
Section 4: Systematic Evaluation

Llevé a cabo la evaluación del agente mediante la creación de casos de prueba, comparando sus respuestas con los resultados esperados y analizando su desempeño para identificar posibles errores. Asimismo, verifiqué si cumplía con los criterios establecidos en distintos escenarios.


Day 4b - Agent Evaluation
Compartir
Guardar versión
Code
eval_config = {
"criteria": {
"tool_trajectory_avg_score": 1.0, # Perfect tool usage required
"response_match_score": 0.8, # 80% text similarity threshold
}
}
with open('home_automation_agent/test_config.json', "w") as f:
json.dump(eval_config, f, indent=2)
print("✅ Evaluation configuration created!")
print("\n📊 Evaluation Criteria:")
print("• tool_trajectory_avg_score: 1.0 - Requires exact tool usage match")
print("• response_match_score: 0.8 - Requires 80% text similarity")
print("\n🎯 What this evaluation will catch:")
print("✅ Incorrect tool usage (wrong device, location, or status)")
print("✅ Poor response quality and communication")
print("✅ Deviations from expected behavior patterns")
✅ Evaluation configuration created!
📊 Evaluation Criteria:
• tool_trajectory_avg_score: 1.0 - Requires exact tool usage match
• response_match_score: 0.8 - Requires 80% text similarity
🎯 What this evaluation will catch:
✅ Incorrect tool usage (wrong device, location, or status)
✅ Poor response quality and communication
✅ Deviations from expected behavior patterns
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento


Day 4b - Agent Evaluation
Compartir
Guardar versión
Markdown
# Create evaluation test cases that reveal tool usage and response quality problems
test_cases = {
"eval_set_id": "home_automation_integration_suite",
"eval_cases": [
{
"eval_id": "living_room_light_on",
"conversation": [
{
"user_content": {
"parts": [
{"text": "Please turn on the floor lamp in the living room"}
]
},
"final_response": {
"parts": [
{
"text": "Successfully set the floor lamp in the living room to on."
}
]
},
"intermediate_data": {
"tool_uses": [
{
"name": "set_device_status",
"args": {
"location": "living room",
"device_id": "floor lamp",
"status": "ON",
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

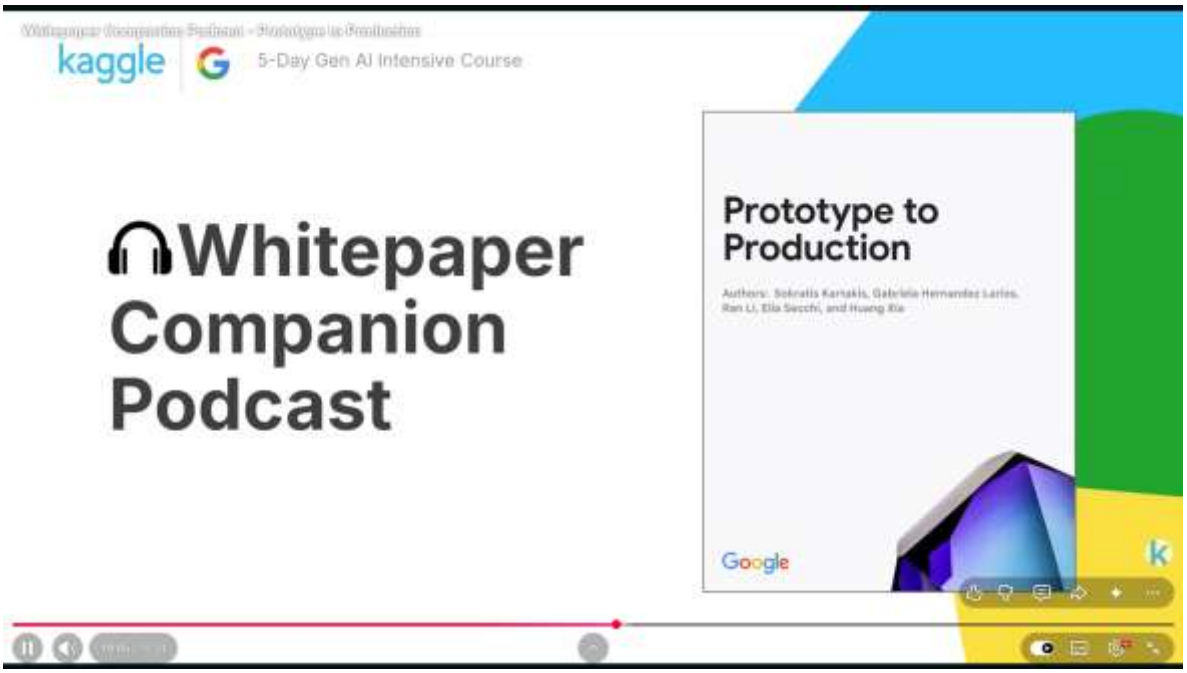

Day 4b - Agent Evaluation
Compartir
Guardar versión
Code
with open('home_automation_agent/integration.evalset.json', "w") as f:
json.dump(test_cases, f, indent=2)
print("✅ Evaluation test cases created")
print("\n📝 Test scenarios:")
for case in test_cases['eval_cases']:
user_msg = case['conversation'][0]['user_content']['parts'][0]['text']
print(f"• {case['eval_id']}: {user_msg}")
print("\n🎯 Expected results:")
print("• basic_device_control: Should pass both criteria")
print(
"• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters"
)
print(
"• poor_response_quality_test: May fail response_match if response differs too much"
)
✅ Evaluation test cases created
📝 Test scenarios:
• living_room_light_on: Please turn on the floor lamp in the living room
• kitchen_on_off_sequence: Switch on the main light in the kitchen.
🎯 Expected results:
• basic_device_control: Should pass both criteria
• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters
• poor_response_quality_test: May fail response_match if response differs too much
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento



DIA 5

En el Día 5, **“De prototipo a producción”**, comprendí cómo los agentes de IA pasan a entornos reales, abordando su despliegue, escalamiento y comunicación mediante A2A en servicios en la nube como Vertex AI Agent Engine.

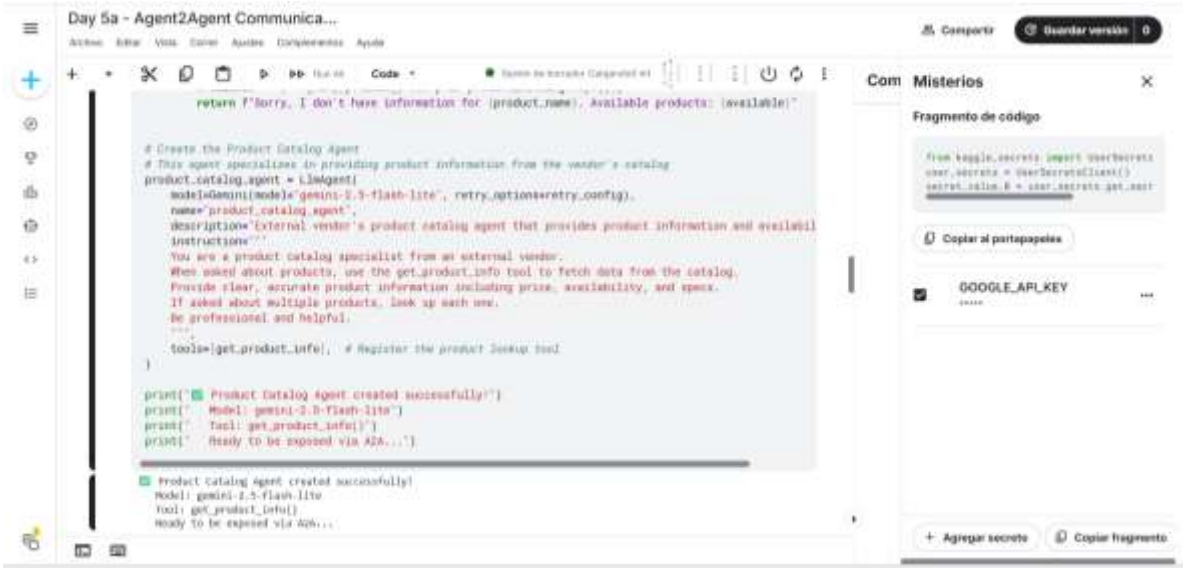
Escuché el episodio resumen del podcast de esta unidad, donde se presentan de manera clara y sencilla los conceptos principales.



Leí el informe técnico “Del prototipo a la producción” para complementar el podcast y reforzar mi comprensión sobre el paso de los agentes hacia entornos reales.

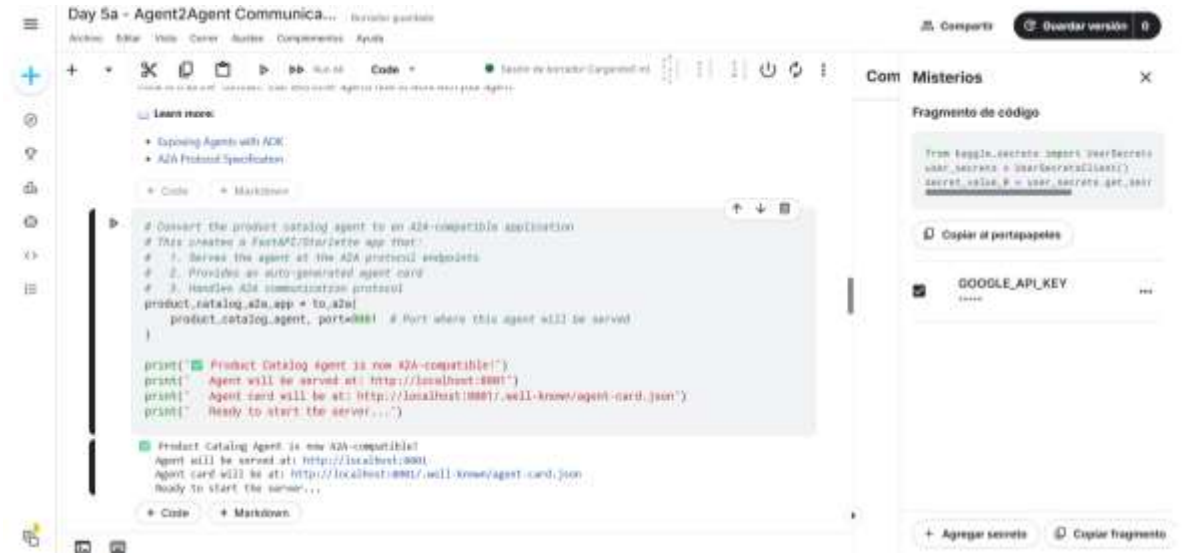
Section 1: Create the Product Catalog Agent (To Be Exposed)

Comprendí la creación de un agente de catálogo de productos que puede exponerse mediante A2A, lo que permite que otros agentes accedan a su información sin necesidad de modificar su implementación.



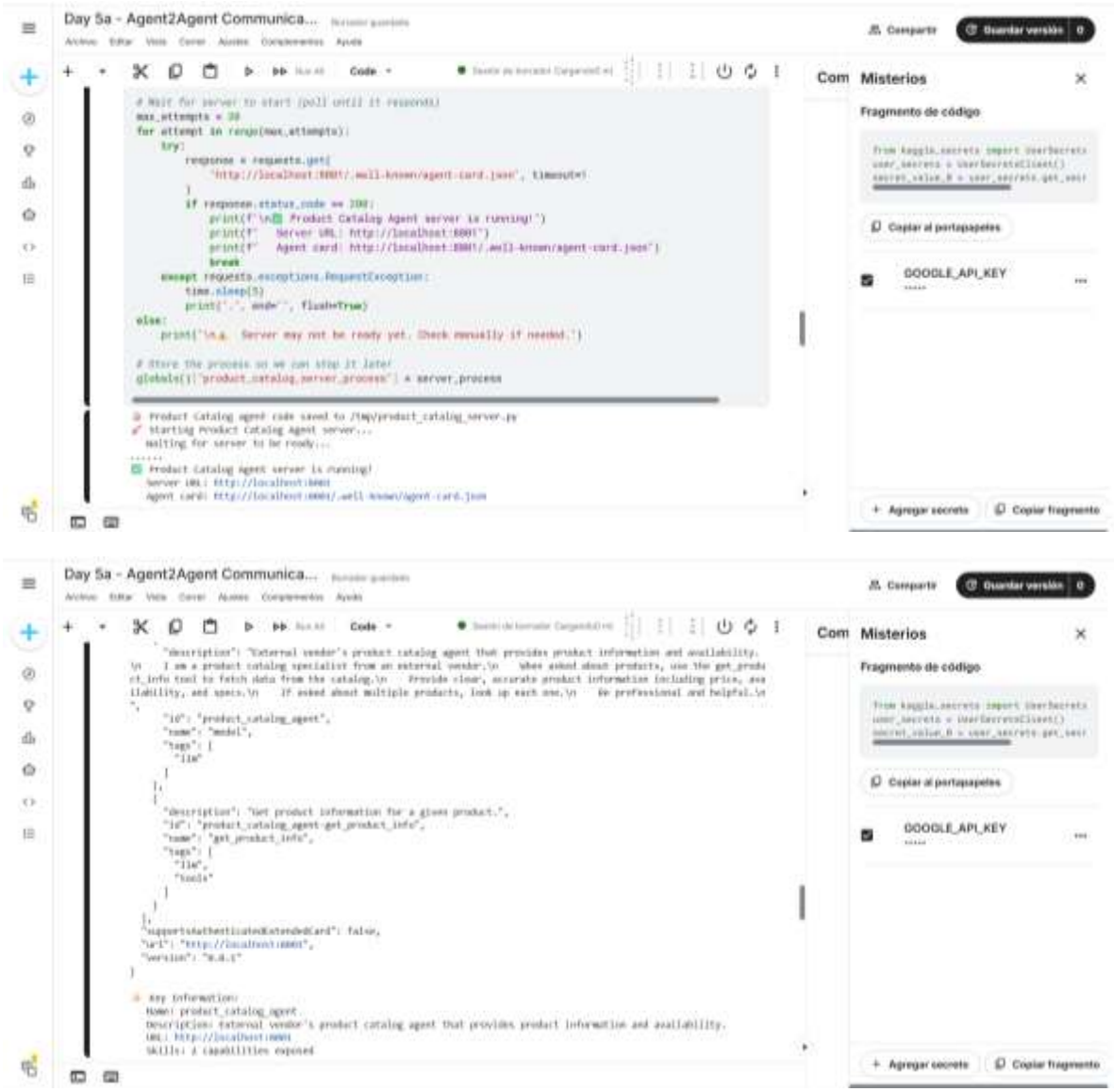
Section 2: Expose the Product Catalog Agent via A2A

Ejecuté la exposición del agente mediante A2A con to_a2a(), observando la generación automática del agent card y su acceso por otros agentes.



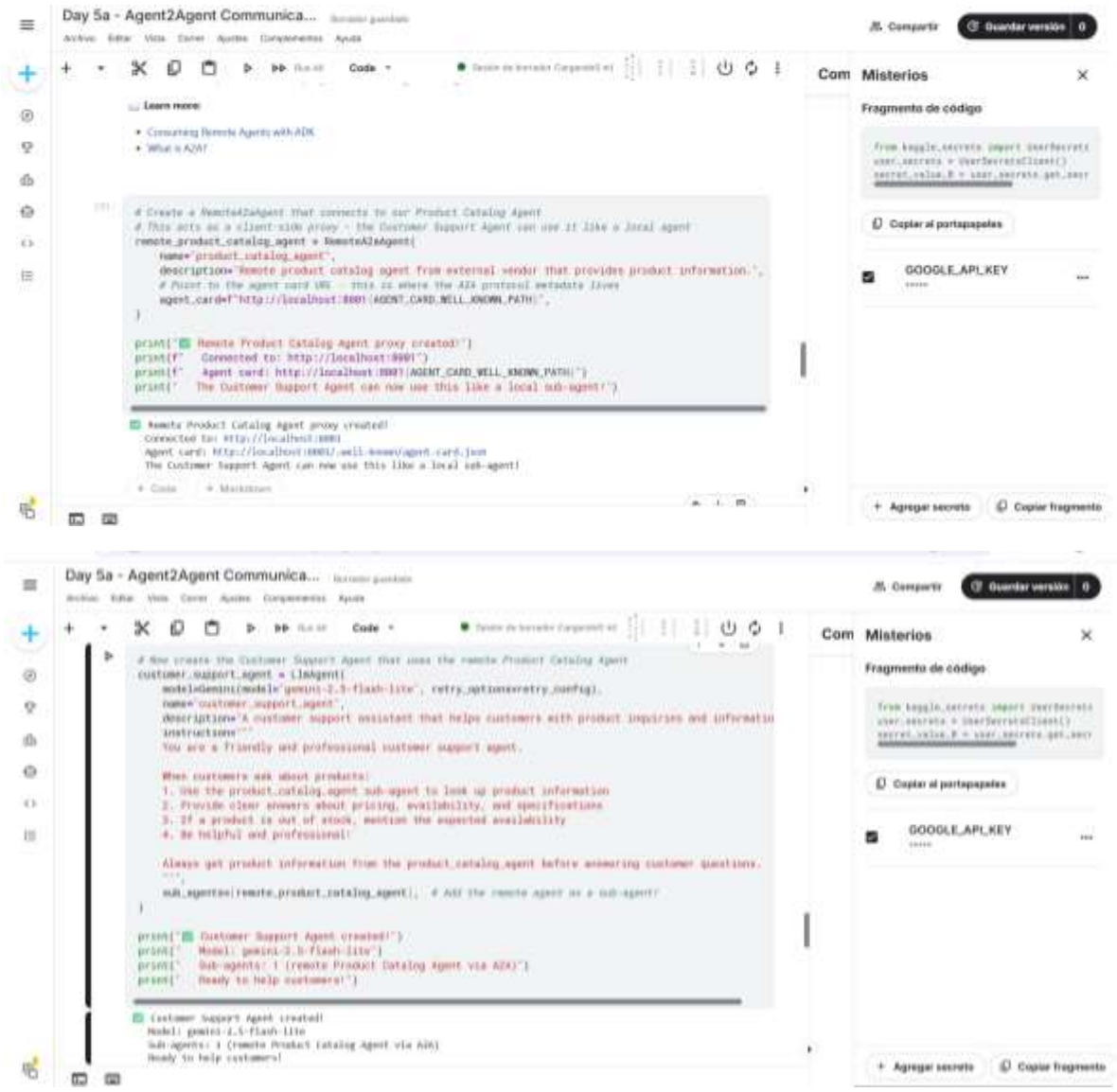
Section 3: Start the Product Catalog Agent Server

En esta sección ejecuté el inicio del servidor del agente de catálogo de productos utilizando uvicorn, permitiendo que funcione en segundo plano y pueda recibir solicitudes de otros agentes.



Section 4: Create the Customer Support Agent (Consumer)

Analicé la creación de un agente de soporte al cliente que consume al agente de catálogo mediante A2A utilizando *RemoteA2aAgent*, lo que permite usarlo como si fuera una herramienta local y observar la comunicación automática entre agentes.



Section 5: Test A2A Communication

Se probó la comunicación entre agentes mediante A2A, donde el agente de soporte consulta información al agente de catálogo y devuelve la respuesta al usuario. También se observó cómo todo el proceso ocurre de forma transparente sin interacción directa con el agente remoto.

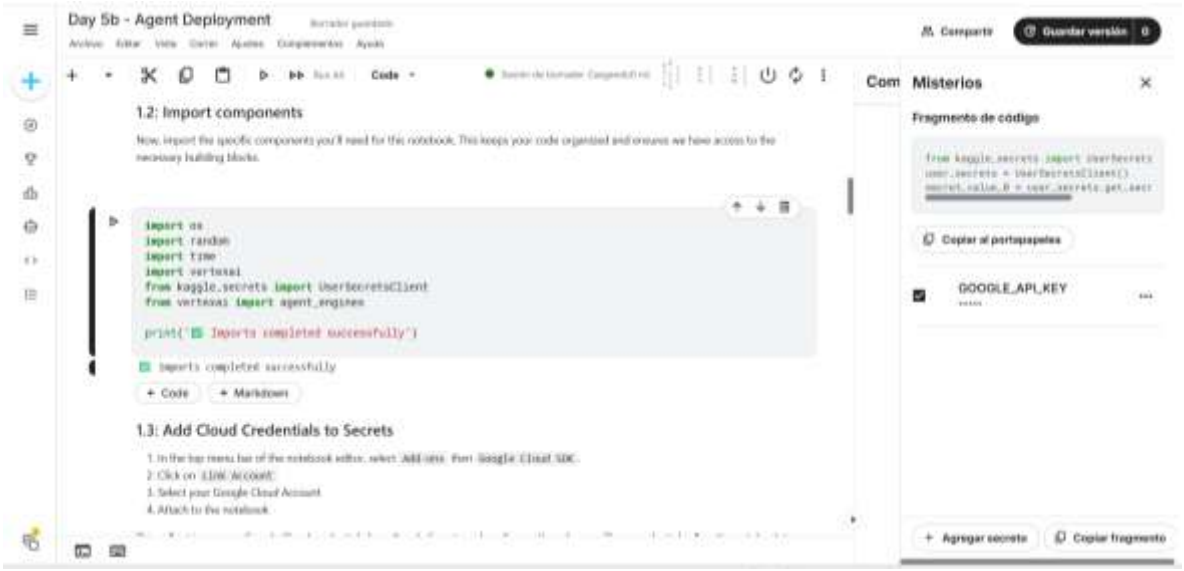

Day 5a - Agent2Agent Communica...
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

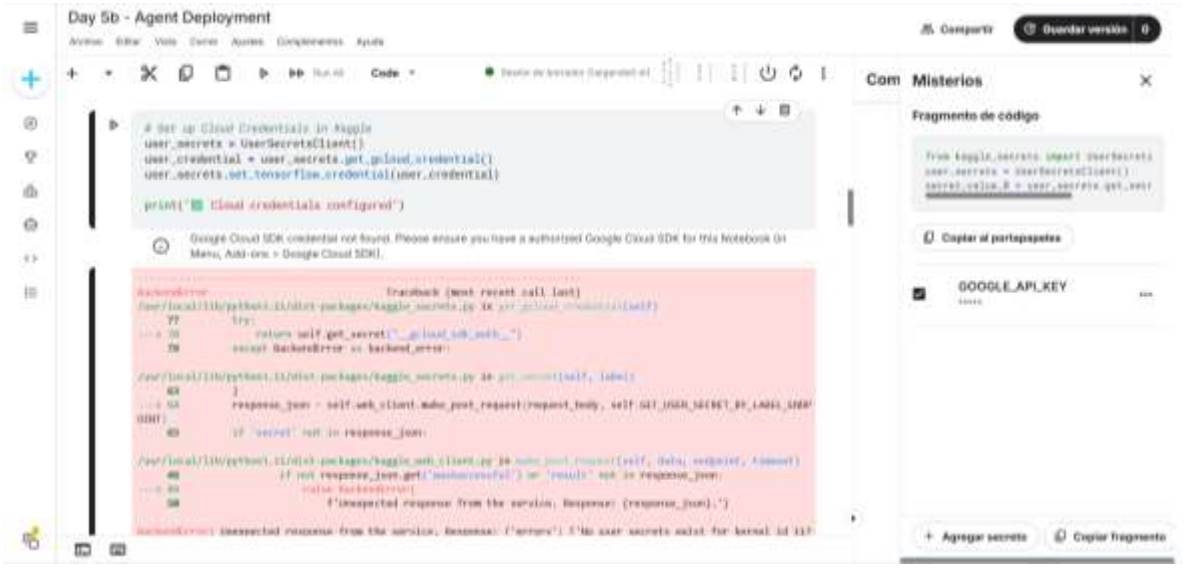

Day 5a - Agent2Agent Communica...
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

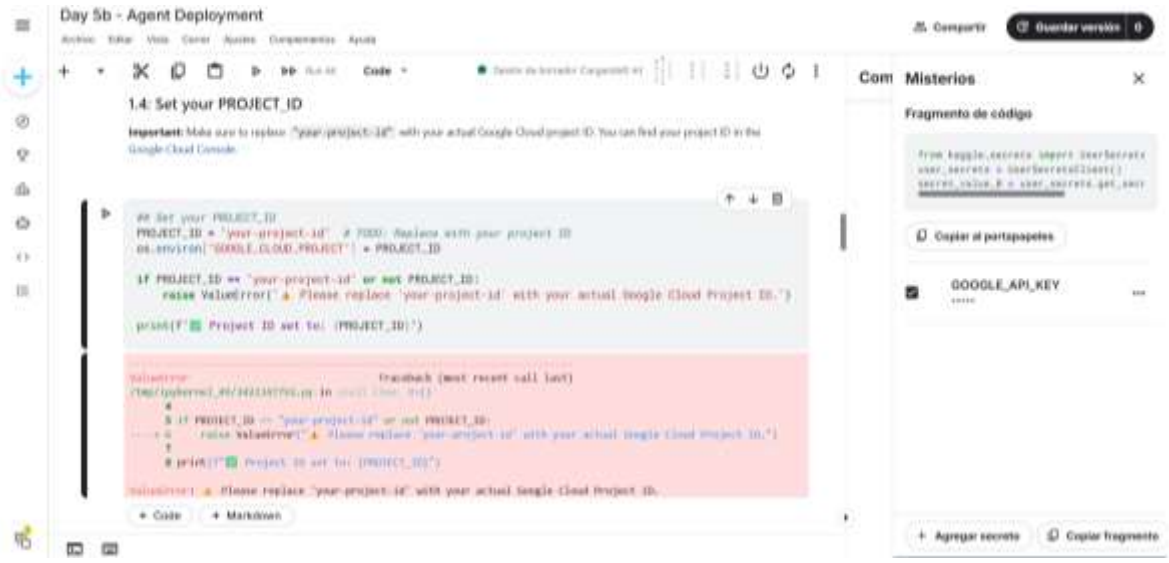

Day 5a - Agent2Agent Communica...
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento

Section 1: Setup

En esta sección no llevé a cabo la configuración en Google Cloud, ya que se requiere registrar una tarjeta y habilitar facturación. Por ello, omití la ejecución y me enfoqué solo en revisar la parte teórica.

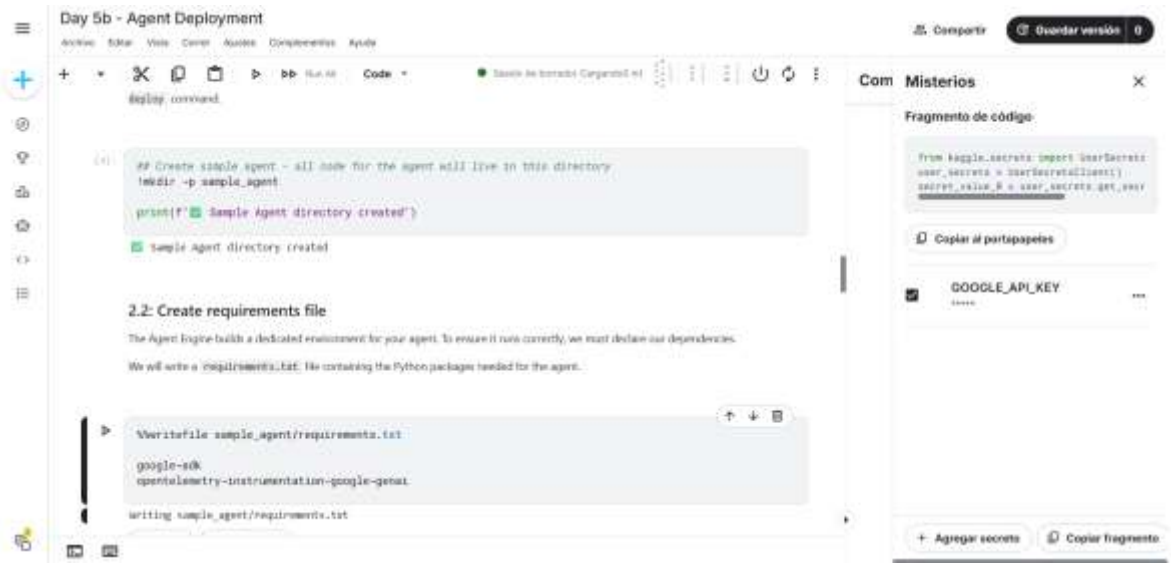


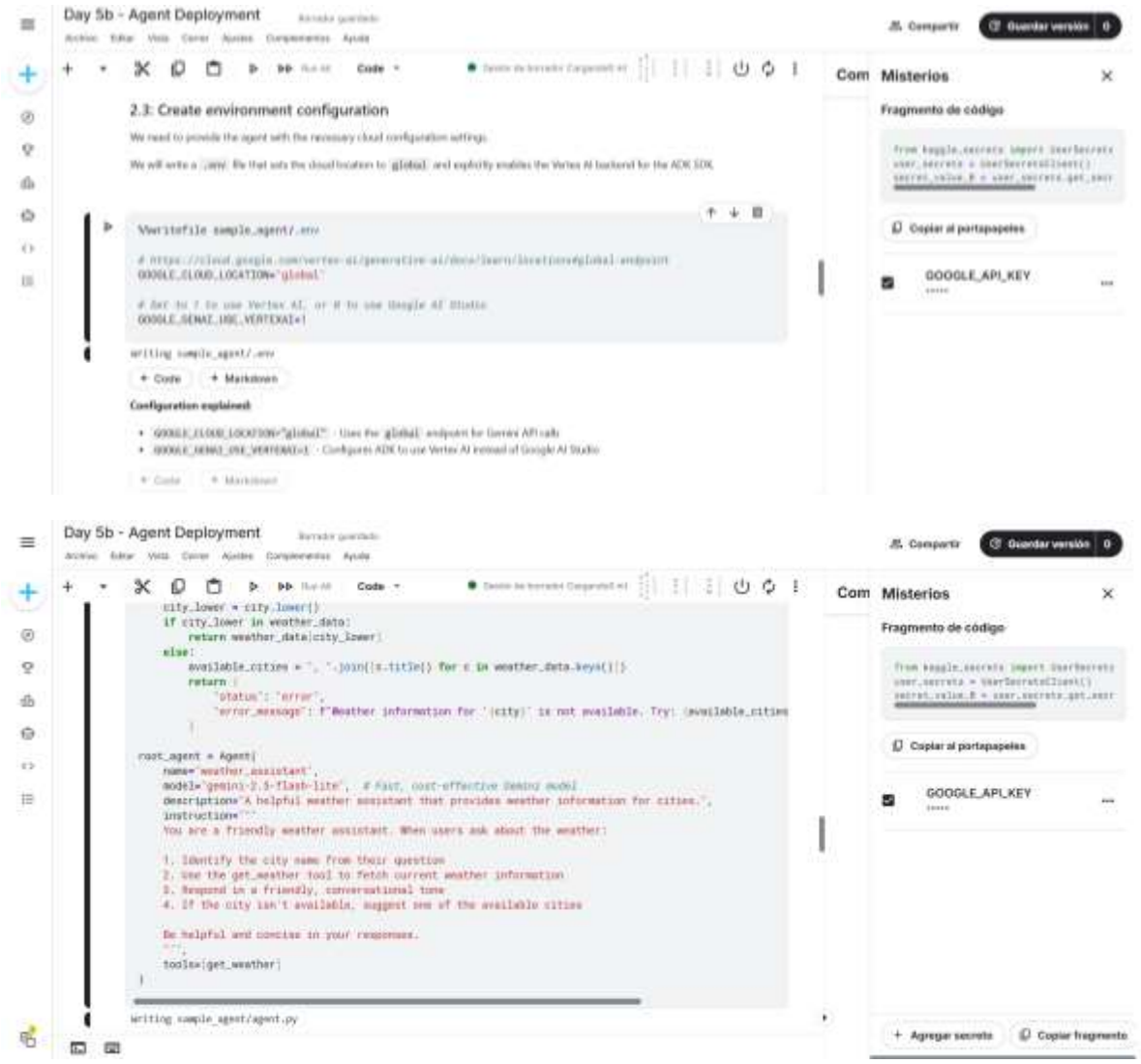




Section 2: Create Your Agent with ADK

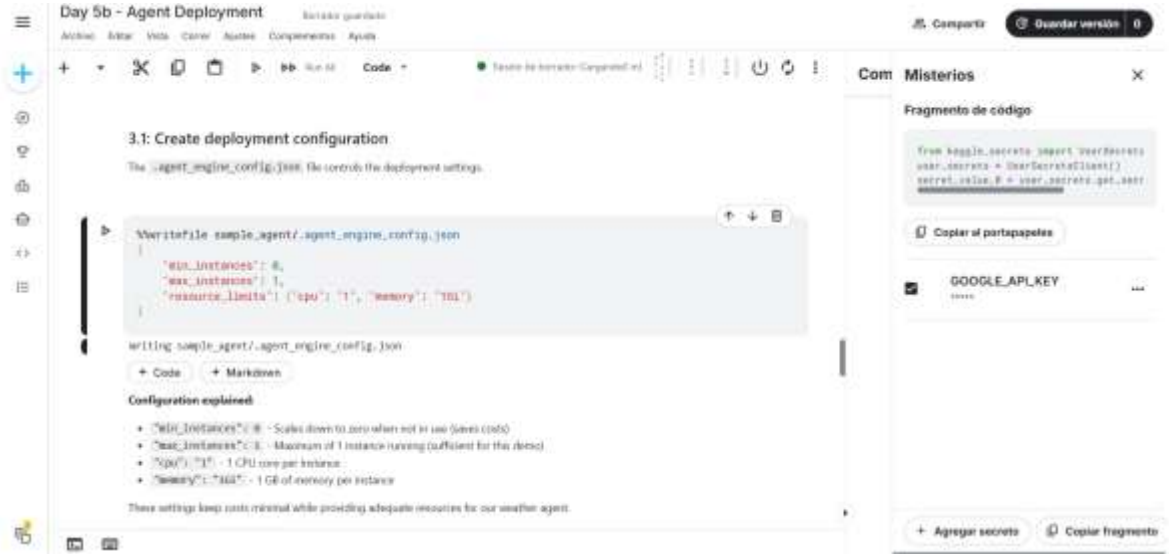
Se presenta la creación de un agente de clima utilizando ADK, organizando los archivos necesarios como el código, las dependencias y la configuración dentro de una carpeta. Asimismo, se observa cómo se define el agente con su modelo, instrucciones y una herramienta para consultar el clima, dejándolo preparado para su despliegue en un entorno de producción.

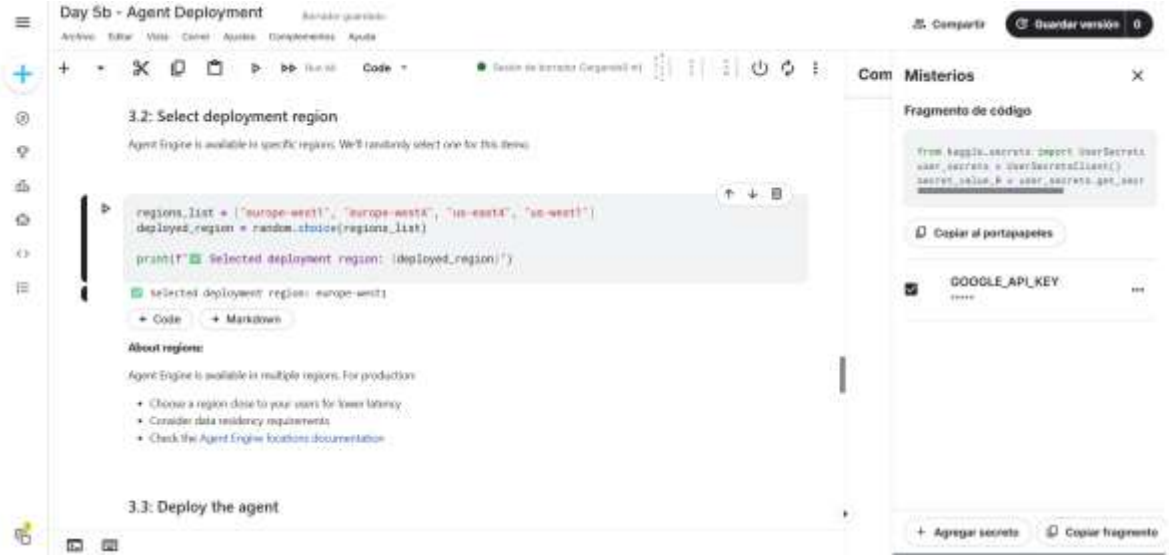


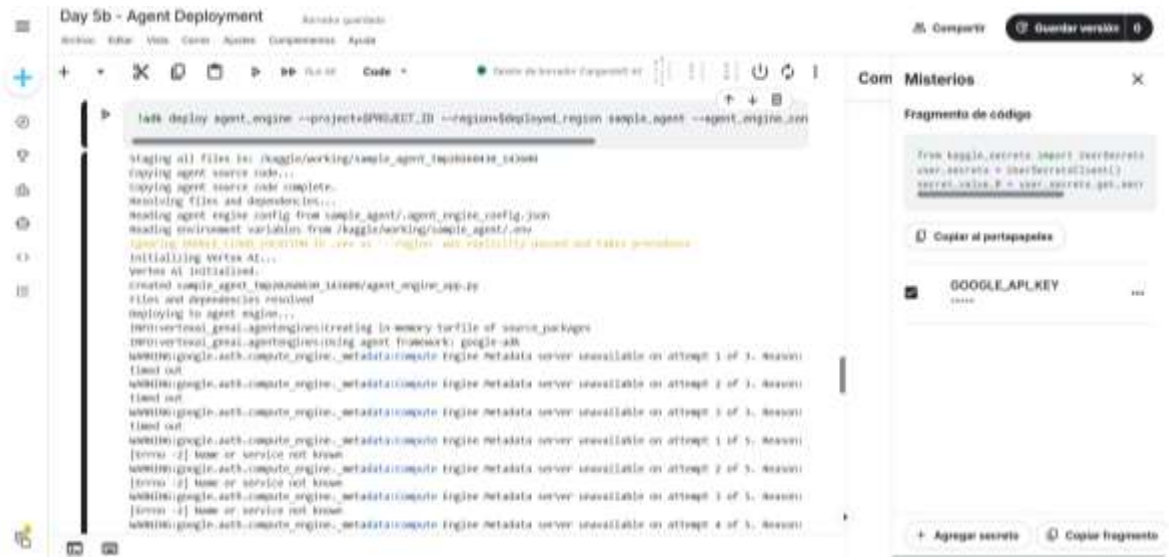


Section 3: Deploy to Agent Engine

En esta sección se muestra cómo se configura y despliega el agente en Vertex AI Agent Engine, dejándolo listo para ejecutarse en la nube.

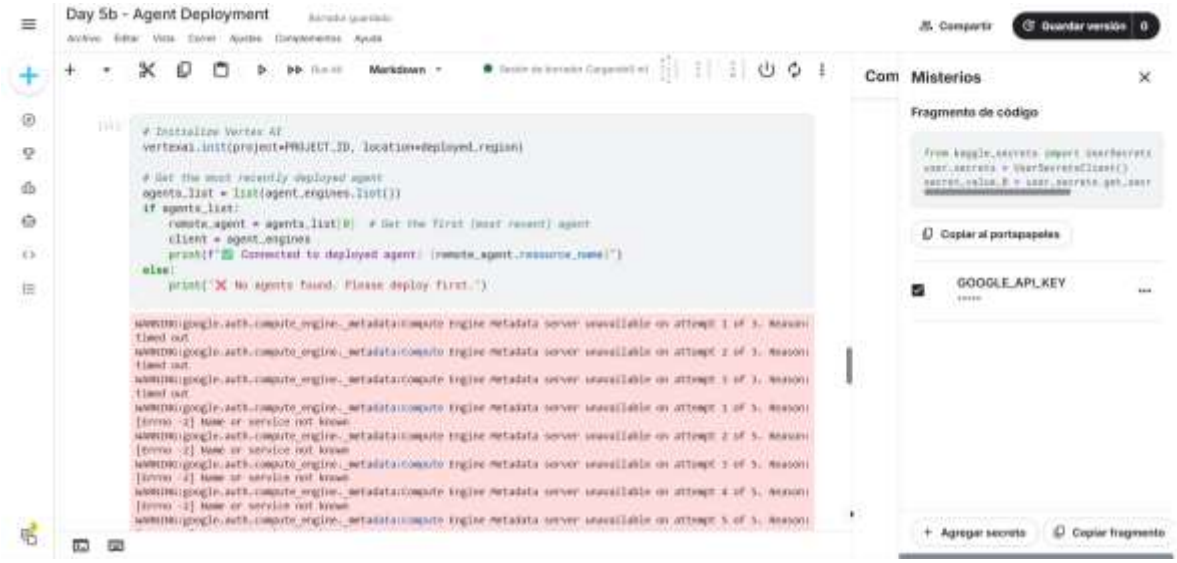

Day 5b - Agent Deployment
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
3.1: Create deployment configuration
The .agent_engine_config.json file controls the deployment settings.
%%writefile sample_agent/.agent_engine_config.json
{
"min_instances": 0,
"max_instances": 1,
"resource_limits": {"cpu": "1", "memory": "1Gi"}
}
Writing sample_agent/.agent_engine_config.json
+ Code
+ Markdown
Configuration explained:
"min_instances": 0 - Scales down to zero when not in use (saves costs)
"max_instances": 1 - Maximum of 1 instance running (sufficient for this demo)
"cpu": "1" - 1 CPU core per instance
"memory": "1Gi" - 1 GB of memory per instance
These settings keep costs minimal while providing adequate resources for our weather agent.

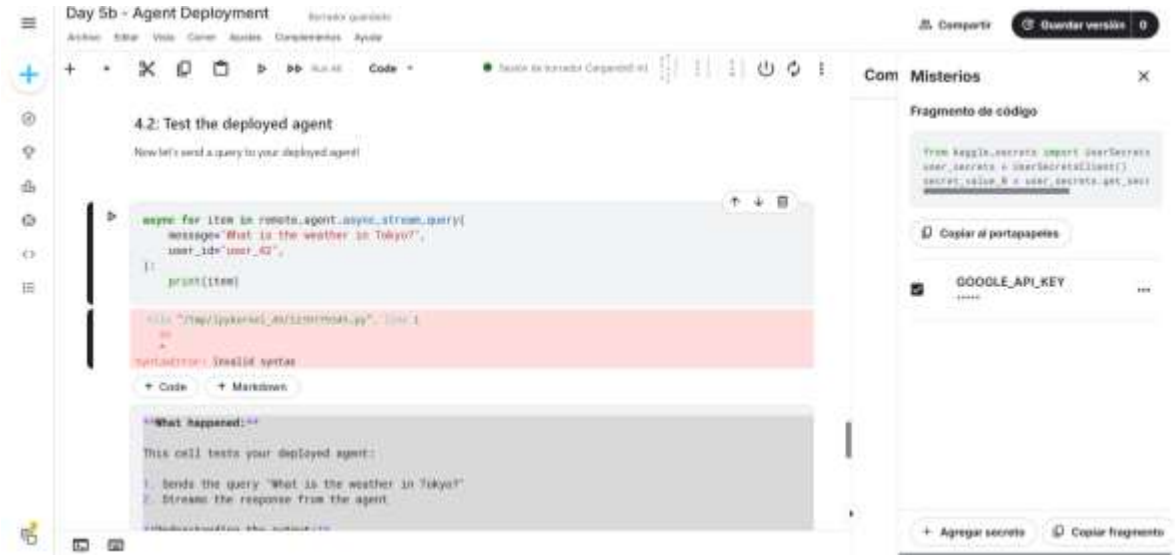

Day 5b - Agent Deployment
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
3.2: Select deployment region
Agent Engine is available in specific regions. We'll randomly select one for this demo.
regions_list = ["europe-west1", "europe-west4", "us-east4", "us-west1"]
deployed_region = random.choice(regions_list)
print(f"☑ Selected deployment region: {deployed_region}")
☑ Selected deployment region: europe-west1
+ Code
+ Markdown
About regions:
Agent Engine is available in multiple regions. For production:
Choose a region close to your users for lower latency
Consider data residency requirements
Check the Agent Engine locations documentation
3.3: Deploy the agent


Day 5b - Agent Deployment
Compartir
Guardar versión
Misterios
Fragmento de código
Copiar al portapapeles
GOOGLE_API_KEY
Agregar secreto
Copiar fragmento
!adk deploy agent_engine --project=$PROJECT_ID --region=$deployed_region sample_agent --agent_engine_con
Staging all files in: /kaggle/working/sample_agent_tmp
Copying agent source code...
Copying agent source code complete.
Resolving files and dependencies...
Reading agent engine config from sample_agent/.agent_engine_config.json
Reading environment variables from /kaggle/working/sample_agent/.env
Initializing Vertex AI...
Vertex AI initialized.
Files and dependencies resolved
Deploying to agent engine...
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 1 of 3. Reason:
timed out
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 2 of 3. Reason:
timed out
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 3 of 3. Reason:
timed out
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 1 of 5. Reason:
[Errno -2] Name or service not known
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 2 of 5. Reason:
[Errno -2] Name or service not known
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 3 of 5. Reason:
[Errno -2] Name or service not known
WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 4 of 5. Reason:

Section 4: Retrieve and Test Your Deployed Agent

Examiné el proceso para acceder al agente ya desplegado y enviarle consultas de prueba, observando cómo genera sus respuestas y cómo utiliza sus herramientas para obtener el resultado final.





Section 5: Long-Term Memory with Vertex AI Memory Bank

Analicé el concepto de memoria a largo plazo en agentes, comprendiendo cómo pueden conservar información entre distintas conversaciones y cómo se integra este mecanismo, aunque no lo llevé a la práctica.

Sección 6: Cleanup

Revisé la importancia de eliminar recursos en la nube para evitar costos innecesarios y entendí el proceso para borrar un agente desplegado, aunque no lo ejecuté debido a que no realicé el despliegue.

Conclusión

A lo largo de este curso adquirí una comprensión general sobre el funcionamiento de los agentes de inteligencia artificial, desde su creación hasta su aplicación en escenarios más cercanos a la realidad. Aprendí cómo interactúan con herramientas, cómo gestionan la memoria y de qué manera pueden colaborar entre sí, así como los métodos para evaluarlos y verificar su correcto desempeño.

También entendí que los agentes no se limitan a pruebas básicas, sino que pueden escalar y utilizarse en entornos más complejos, incluso conectándose entre ellos. Aunque en varias secciones me enfoqué únicamente en observar los procesos, esto me permitió tener una visión clara del flujo completo y de cómo evolucionan desde implementaciones simples hasta soluciones más robustas y útiles en contextos reales.

En general, este curso me ayudó a comprender mejor el desarrollo de estos sistemas y la relevancia de cada una de sus etapas, desde su creación hasta su implementación final, brindándome una base sólida para continuar aprendiendo sobre inteligencia artificial.